Промышленные графы знаний

Семантические технологии и графы знаний

Дмитрий Ильич Муромцев

кандидат технических наук, доцент университета ИТМО, руководитель международной лаборатории "Интеллектуальные методы обработки информации и семантические технологии"

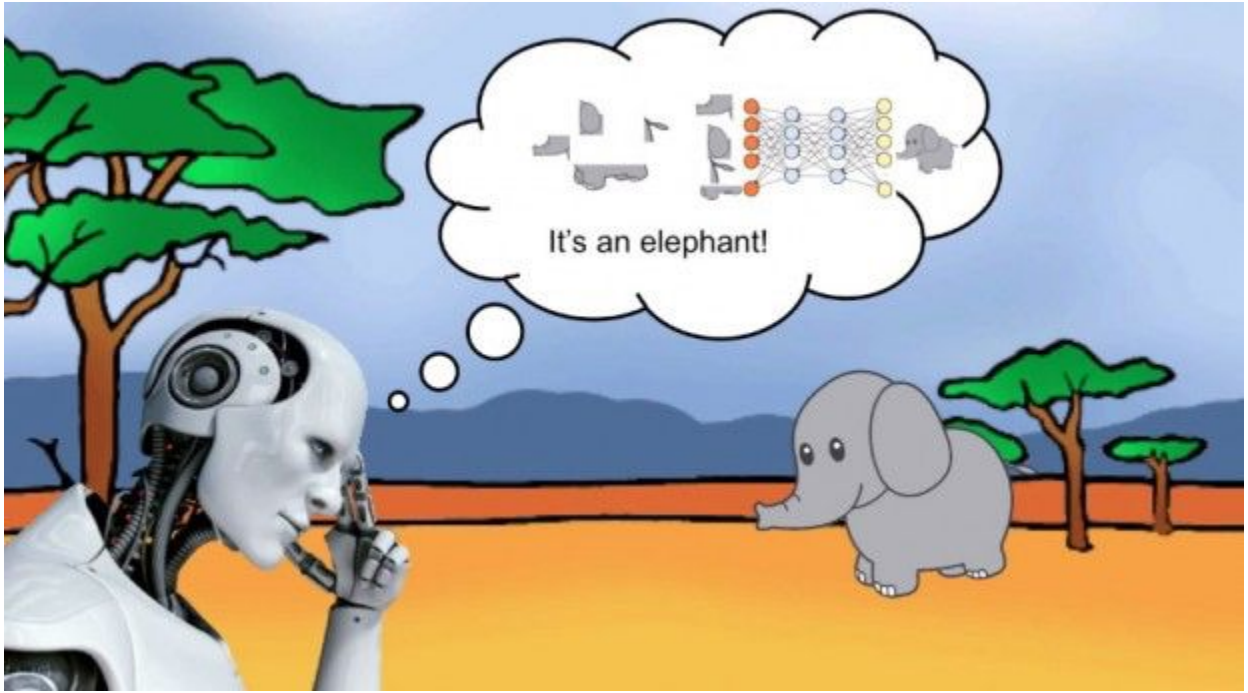
07 февраля 2022 года

1. Введение в представление знаний

Некоторые вопросы, которые возникают в представлении знаний с точки зрения ИИ

- Как люди представляют знания?
- Какова природа знаний и как мы их представляем?
- Должна ли схема представления связываться с частной областью знаний, или она должна быть общецелевой?
- Насколько выразительна данная схема представления?
- Должна ли быть схема декларативной или процедурной?

Зачем это нужно?



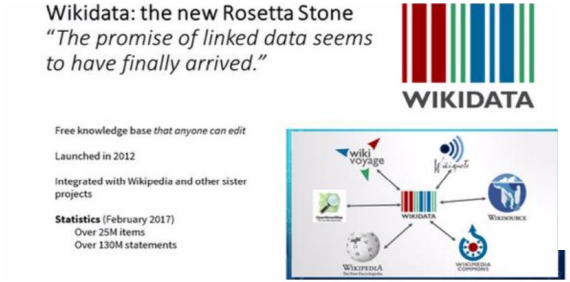
Предпосылки появления

- Первые исследования в области искусственного интеллекта
- Разнообразие трудноформализуемых задач
- Развитие когнитивной психологии, формальных логик и человеко-машинных интерфейсов и баз знаний

Что же такое знания?

- «Знания — это закономерности предметной области (принципы, связи, законы), полученные в результате практической деятельности и профессионального опыта, позволяющие специалистам ставить и решать задачи в этой области»
- «Знания — это хорошо структурированные данные или дан ные о данных, или метаданные»
- «Знания — формализованная информация, на которую ссылаются или используют в процессе логического вывода»

Где и кем используются



- Все больше и больше знаний доступно в взаимосвязанной, структурированной, логически доступной форме

Что такое граф знаний?

- Термин граф знаний ввела в обращение компания Google
- Google Knowledge Graph

  https://googleblog.blogspot.com/2012/05/introducing-knowledge-graph-things-not.html

Google Knowledge Graph

Ньютон, Исаак — Википедия
https://ru.wikipedia.org/wiki/Ньютон,_Исаак
Сэр Иса́к **Нью́тон** (англ. Isaac Newton /ˈnjuːtən/, 25 декабря 1642 года — 20 марта 1727 года по юлианскому календарю, действовавшему в Англии до ...

Дата смерти: 20 (31) марта 1727 (84 года) **Научный руководитель**: Исаак Барроу
Дата рождения: 25 декабря 1642 (**4 января 1...** **Место смерти**: Кенсингтон, Великобритания;

Биография · Личные качества · Научная деятельность

Ньютон (единица измерения) — Википедия
https://ru.wikipedia.org/wiki/Ньютон_(единица_измерения)
Нью́тон (русское обозначение: Н; международное: N) — единица измерения силы в Международной системе единиц (СИ). **Ньютон** — производная ...
История · Связь с другими ... · Кратные и дольные ...

Исаак Ньютон - Биография - Кафедра физхимии ЮФУ
www.physchem.chimfak.rsu.ru/Source/History/Persones/Newton.html
В 12 лет **Ньютон** начал учиться в Грантемской школе, в 1661 г. поступил в колледж св. Троицы (Тринити-колледж) Кембриджского университета в ...

Исаак Ньютон - биография, законы физики, семья, личная жизнь ...
https://24smi.org/celebrity/3876-isaak-niuton.html
★★★★☆ Рейтинг: 4 - 44 голоса
Биография Исааке **Ньютоне**, его открытия, женщины, интересные факты, книги, личная жизнь, жена, дети, рост, возраст, фото и последние новости о ...

Исаак Ньютон

Физик

Сэр Иса́к Нью́тон — английский физик, математик, механик и астроном, один из создателей классической физики. Автор фундаментального труда «Математические начала натуральной философии», в котором он изложил закон всемирного тяготения и три закона механики, ставшие основой классической механики.
Википедия

Родился: 4 января 1643 г., Усадьба Вулсторп, Великобритания

Умер: 31 марта 1727 г., Кенсингтон, Лондон, Великобритания

Полное имя: Sir Isaac Newton

Открытия: Законы Ньютона, Классическая теория тяготения Ньютона

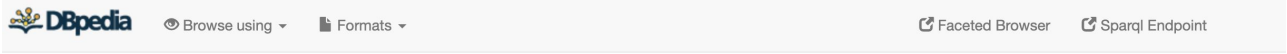
Образование: Тринити-колледж (1667 г.–1668 г.), ЕЩЁ

DBpedia

- Основы и концепция открытых для пользователей графов знаний были впервые реализованы в 2007 в базе знаний DBpedia, созданной в результате семантической обработки инфобоксов статей в Wikipedia

https://wiki.dbpedia.org/

DBpedia — http://dbpedia.org/page/Lukoil



About: Лукойл

An Entity of Type : ÐÑÐ±Ð»Ð¸ÑÐ½Ð°Ñ Ð°Ð¾Ð¼Ð¿Ð¿Ð¿Ð½Ð¸Ñ, from Named Graph : http://dbpedia.org, within Data Space : dbpedia.org

ПАО «ЛУКОЙЛ» — российская нефтяная компания, принадлежащая The Bank of New York Mellon. Официальное название — ПАО «Нефтяная компания „ЛУКОЙЛ“». Наименование компании происходит от первых букв названий городов нефтяников (Лангепас, Урай, Когалым) и слова «ойл» (от англ. oil — нефть). Основные виды деятельности компании — операции по разведке, добыче и переработке нефти и природного газа, реализации нефти и нефтепродуктов. Штаб-квартира «ЛУКОЙЛ» находится в Москве, на Сретенском бульваре. Также у компании имеется североамериканская штаб-квартира, расположенная в Ист-Мэдоу — пригороде Нью-Йорка.

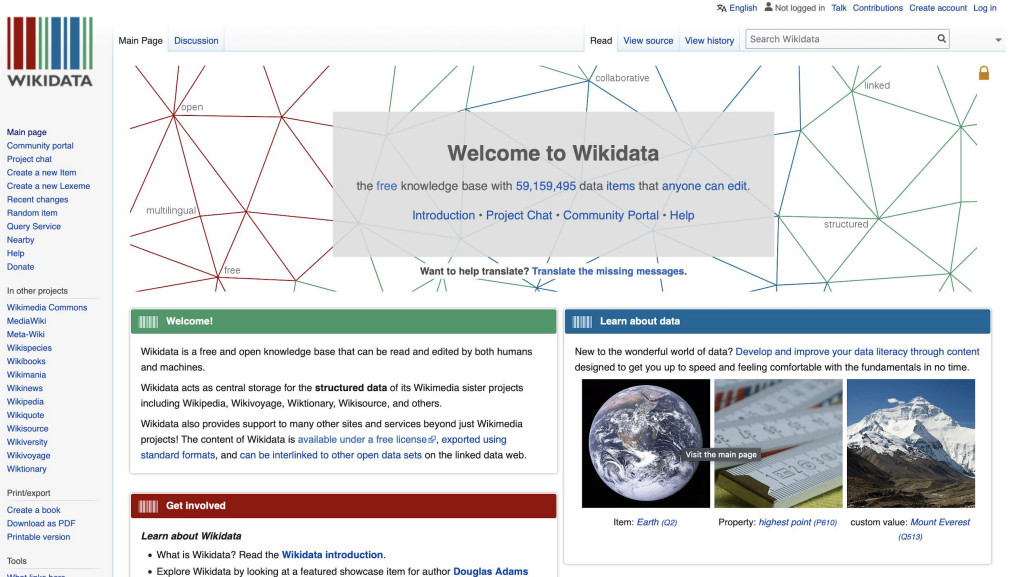
Property	Value
dbo:abstract	▪ ПАО «ЛУКОЙЛ» — российская нефтяная компания, принадлежащая The Bank of New York Mellon. Официальное название — ПАО «Нефтяная компания „ЛУКОЙЛ“». Наименование компании происходит от первых букв названий городов нефтяников (Лангепас, Урай, Когалым) и слова «ойл» (от англ. oil — нефть). Основные виды деятельности компании — операции по разведке, добыче и переработке нефти и природного газа, реализации нефти и нефтепродуктов. Вторая после «Газпрома» по объёмам выручки компания в России (по итогам 2014 года, по данным журнала «Эксперт»). До 2007 являлась крупнейшей по объёму добычи нефтяной компанией в России («Роснефть» опередила её после покупки активов «ЮКОСа»). По размеру доказанных запасов углеводородов «ЛУКОЙЛ», по собственным данным, по состоянию на 1 января 2011 года являлся третьей в мире частной нефтяной компанией (по запасам нефти — первой). Торговая марка «ЛУКОЙЛ» — один из двух российских брендов (наряду с «Балтикой»), вошедших в список 100 крупнейших мировых торговых марок, составленный в апреле 2007 года британской газетой Financial Times. Впрочем, по итогам аналогичного рейтинга, составленного в апреле 2009 года, в сотню ведущих марок «ЛУКОЙЛ» уже не попал. Штаб-квартира «ЛУКОЙЛ» находится в Москве, на Сретенском бульваре. Также у компании имеется североамериканская штаб-квартира, расположенная в Ист-Мэдоу — пригороде Нью-Йорка. (ru)
dbo:foundingYear	▪ 1991-01-01 (xsd:date)
dbo:industry	▪ dbr:Oil_and_gas_industry

DBpedia — запросы

uri	label
http://dbpedia.org/resource/Lukoil	"Lukoil"@en
http://dbpedia.org/resource/Lukoil	"Lukoil"@de
http://dbpedia.org/resource/Lukoil	"LUKoil"@es
http://dbpedia.org/resource/Lukoil	"Lukoil"@fr
http://dbpedia.org/resource/Lukoil	"LUKoil"@it
http://dbpedia.org/resource/Lukoil	"ルクオイル"@ja
http://dbpedia.org/resource/Lukoil	"LUKoil"@nl
http://dbpedia.org/resource/Lukoil	"Lukoil"@pl
http://dbpedia.org/resource/Lukoil	"LUKoil"@pt
http://dbpedia.org/resource/Lukoil	"Лукойл"@ru
http://dbpedia.org/resource/Lukoil	"卢克石油"@zh

Wikidata

- Если DBpedia использует данные Wikipedia для наполнения графа, то Wikidata предназначен для хранения знаний, которые будут использованы уже в Wikipedia.



https://www.wikidata.org/

Wikidata — данные

Lukoil (Q329347)

Russian oil company
LUKoil

edit

In more languages Configure

Language	Label	Description	Also known as
English	Lukoil	Russian oil company	LUKoil
Russian	ЛУКойл	Нефтяная компания	Лангепас-Урай-Когалымнефть ОАО «Лукойл»
Tatar	No label defined	No description defined	
Bashkir	Лукойл	No description defined	

All entered languages

Statements

instance of
- business — edit
  - 0 references
  - + add reference
- enterprise — edit
  - 0 references
  - + add reference

+ add value

logo image — edit

image — edit

Akademika Sakharova 2 Aug 2009 01.JPG
2,200 × 3,000; 1.6 MB

1 reference

+ add value

industry
- extraction of petroleum — edit
  - 1 reference
- petroleum industry — edit
  - 1 reference

+ add value

inception — 25 November 1991 — edit
- 0 references
- + add reference

+ add value

significant person — Vagit Alekperov — edit
- 1 reference

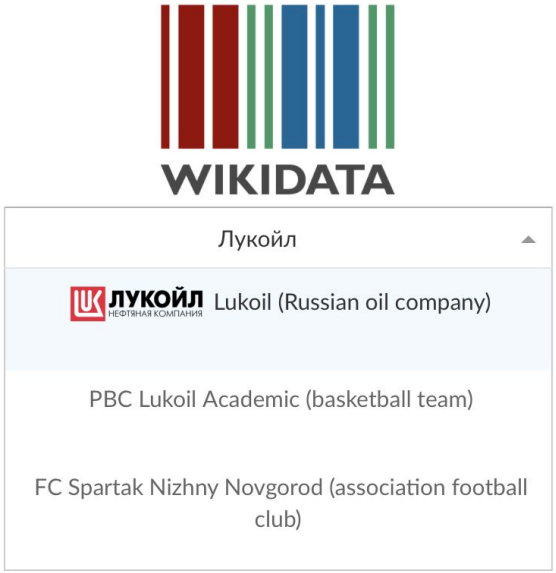
+ add value

founded by — Vagit Alekperov — edit
- 0 references

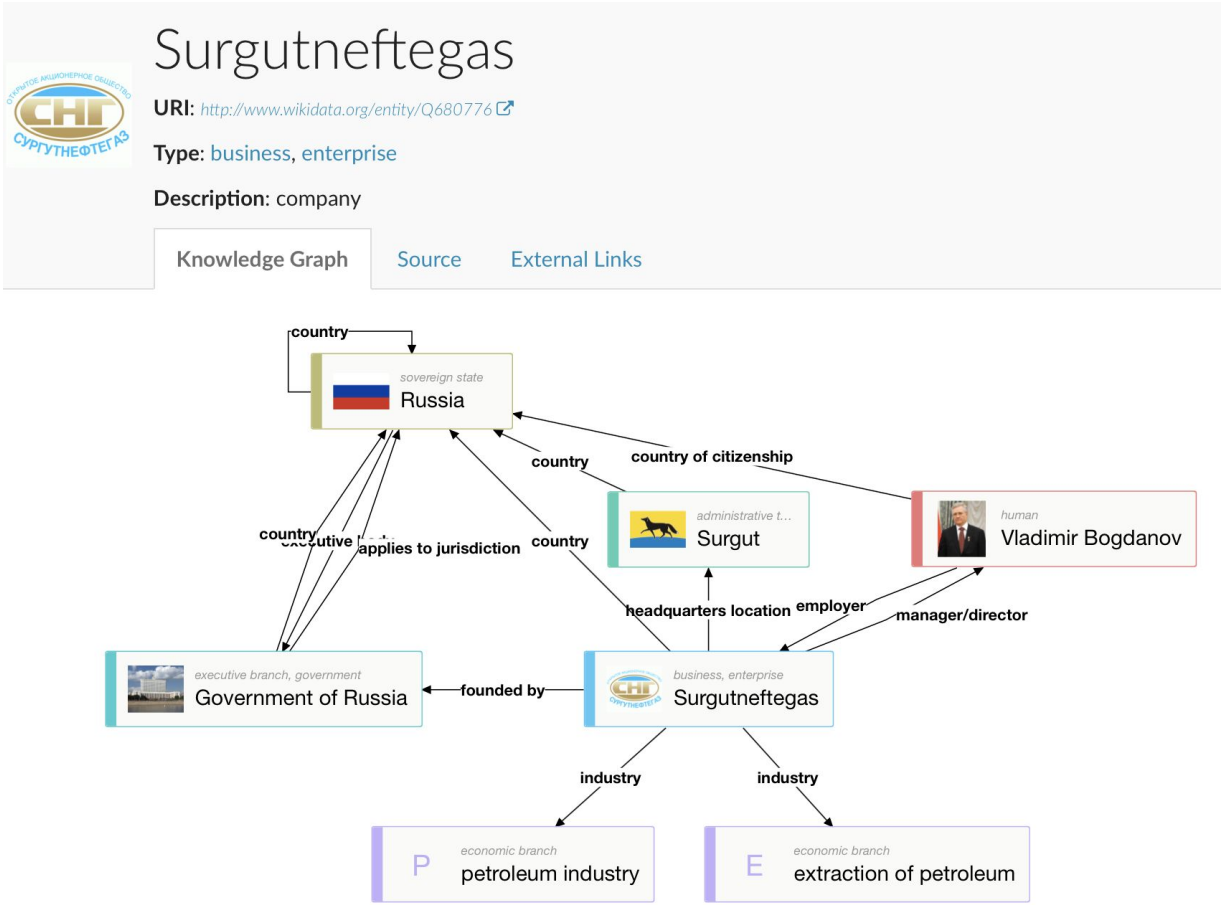
Приложения на основе Wikidata

metaphactory

https://wikidata.metaphacts.com/

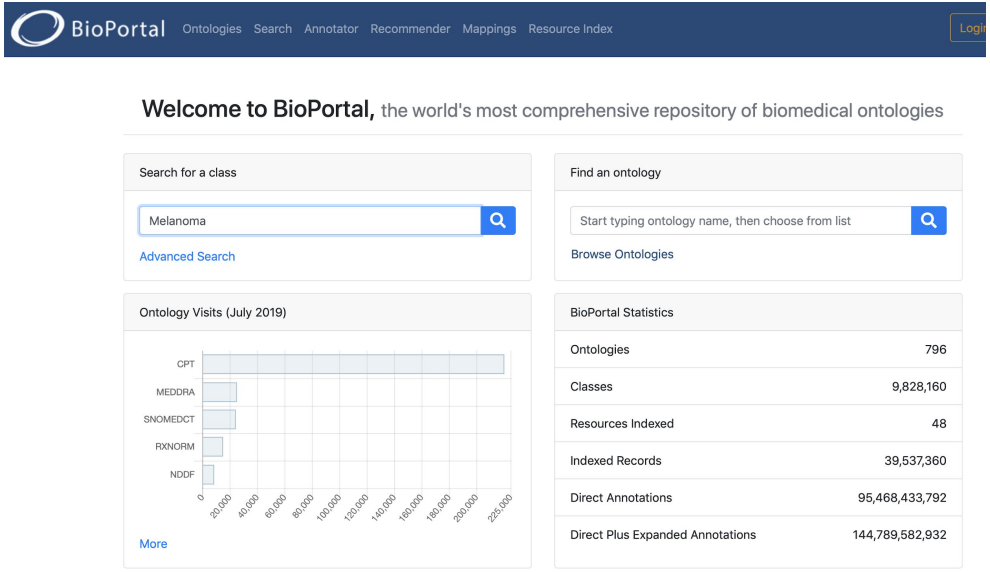


Metaphactory



BioPortal

- Один из наиболее интересных графов знаний, специализирующихся в конкретной предметной области



https://bioportal.bioontology.org

BioPortal

Medical Dictionary for Regulatory Activities Terminology (MedDRA)

Last uploaded: April 29, 2019

Summary | Classes | Properties | Notes | Mappings | Widgets

Details

Acronym	MEDDRA
Visibility	Public
Description	MedDRA is an international medical terminology with an emphasis on use for data entry, retrieval, analysis, and display. It applies to all phases of drug development, excluding animal toxicology, and to the health effects and malfunction of devices. An appendix includes concept descriptions which describe how a medical concept is interpreted, used, and classified within the MedDRA terminology. It is not intended as a medical definition. The concept descriptions are intended to aid the consistent and accurate use of MedDRA in coding, retrieval, and analysis.
Status	Production
Format	UMLS
Contact	MedDRA MSSO, mssohelp@meddra.org
Categories	Health
Groups	Cancer Biomedical Informatics Grid, Unified Medical Language System
License Information	The MedDRA ontology is maintained and distributed by the MedDRA MSSO. This ontology is freely accessible on this site for academic and other non-commercial uses. Users anticipating any commercial use of MedDRA must contact the MSSO to obtain a license.

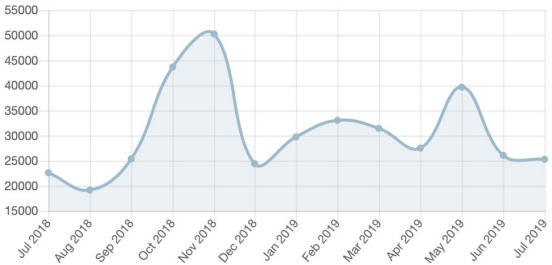
Submissions

Version	Released	Uploaded
2018AB (Parsed, Indexed, Metrics, Annotator, Error Diff)	11/05/2018	04/29/2019
2018AA (Archived)	05/07/2018	07/06/2018

Metrics

Classes	71,982
Individuals	0
Properties	16
Maximum depth	3
Maximum number of children	266
Average number of children	16
Classes with a single child	77
Classes with more than 25 children	418
Classes with no definition	71,759

Visits



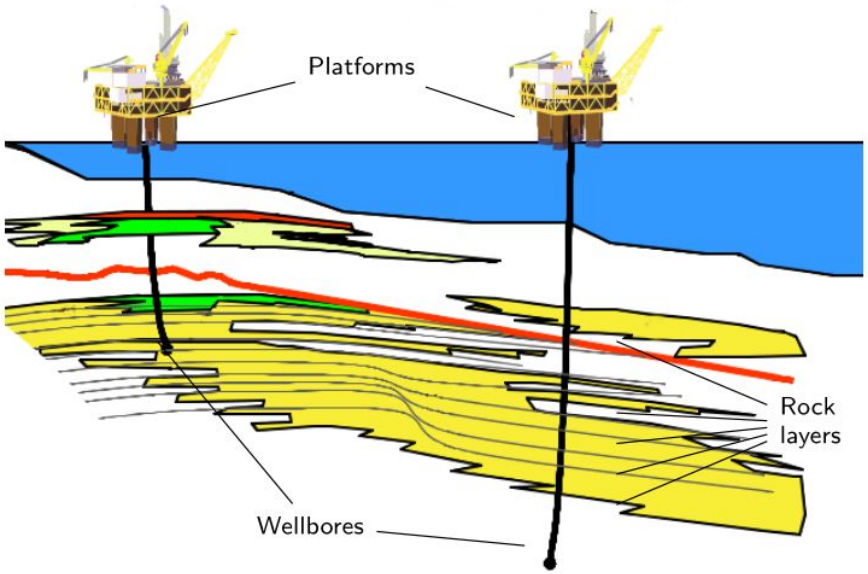
Примеры применения в индустрии

Statoil: интеграция данных на основе онтологий

Около 900 геологов и геофизиков изучают текущее состояние пластов и ищут новые залежи нефти и газа.

Задачи:

1) найти необходимые данные по скважинам, сейсмическим показаниям и т.д. в БД Statoil,
2) проанализировать данные с помощью соответствующих аналитических инструментов.



Источник: https://pdfs.semanticscholar.org/ed7b/716be12a7c42adaa20b9c42ba827b8924297.pdf

Statoil: Проблема

Около 30-70% времени геологи затрачивают на поиск и извлечение данных.

Причины:

- объем и запутанность схемы данных, а также ограниченность интерфейсов доступа,
- необходимость привлечения IT-инженеров удлиняет время на решение задачи.

Statoil: Источники данных

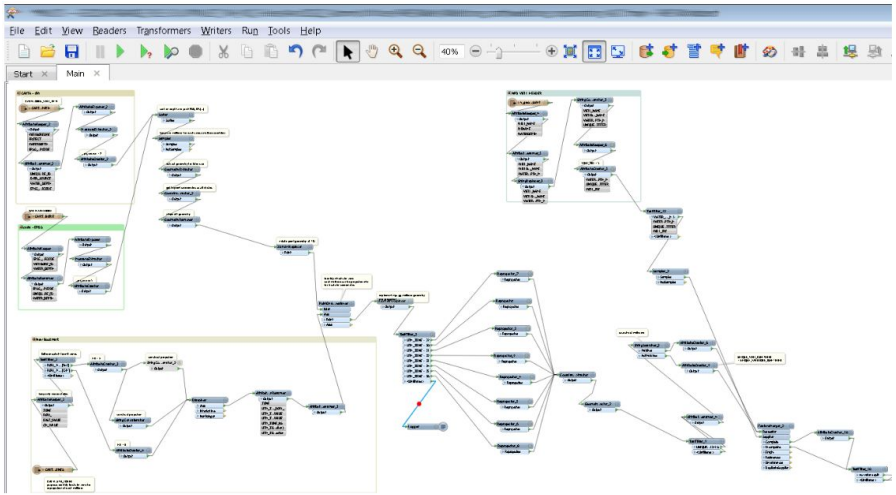
- 6 внутренних баз данных Statoil,
- 1 публичная база данных Норвежского министерства нефти и энергии.

Тысячи таблиц и views и десятки тысяч столбцов. Одни и те же данные представлены по разному, например, типы скважин и пластов.

Statoil: интерфейсы к источникам данных

- формы запросов с фиксированным набором параметров,
- геоинформационные системы,
- views, собранные ETL процедурами.

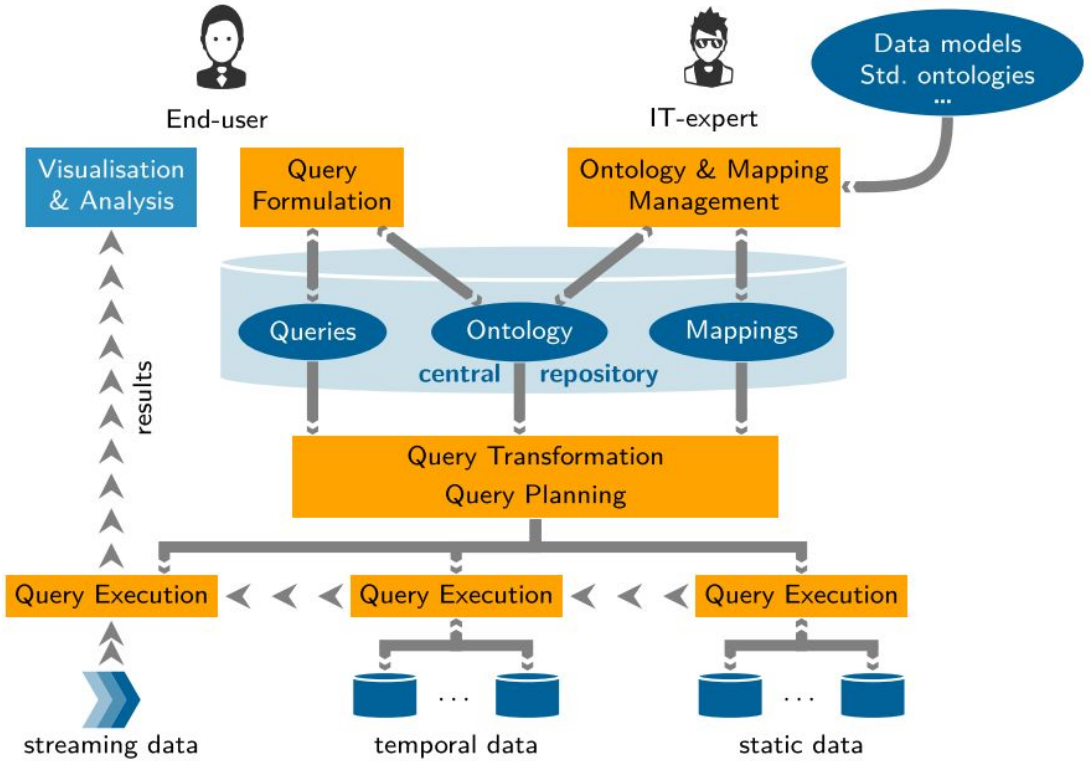
Время на подготовку views: от нескольких дней до нескольких недель.



Statoil: решение

Ontology Based Data Access (OBDA).

Реализация архитектуры виртуальной интеграции данных на основе онтологий и SPARQL.



Statoil: этапы внедрения

- онтология предметной области
  - верхнеуровневая онтология: 106 классов, 91 свойство, 520 аксиом,
  - онтологии баз данных (полуавтоматическая генерация).

- маппинги к базам данных
  - стандарт R2RML,
  - 1 федеративный маппинг (несколько БД в одном запросе),
  - 75 маппингов для верхнеуровневой онтологии,
  - 4000+ маппингов для онтологий баз данных (полуавтоматическая генерация).

Statoil: результаты

Время выполнения запросов (среднее):

- переписывание SPARQL -> SQL
  - 20 миллисекунд
- выполнение SQL
  - 36 секунд
- общее время
  - 58 секунд

Сложность запросов (кол-во символов):

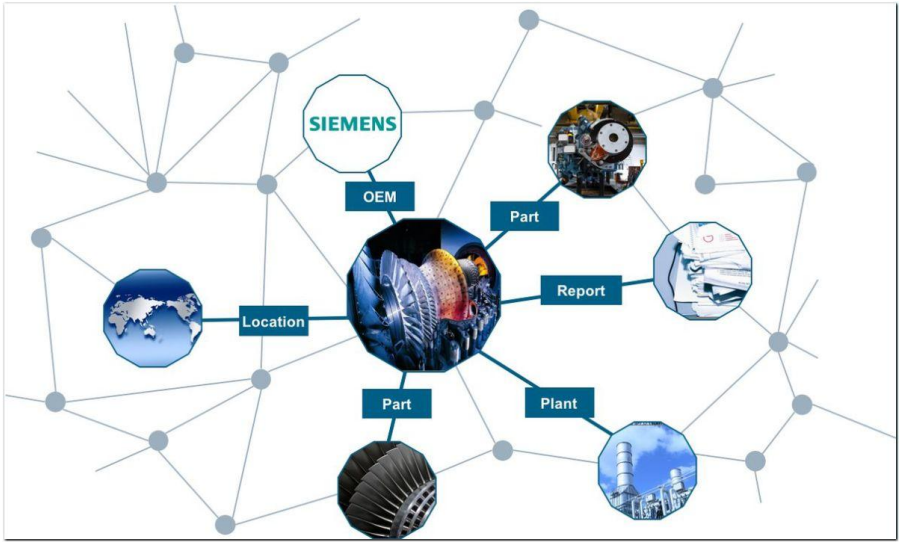
- SPARQL
  - среднее: 462,
  - медианое: 456,
  - максимальное: 1110.
- Сгенерированный SQL
  - среднее: 4283,
  - медианое: 2547,
  - максимальное: 37788.

В целом время выполнения запросов сопоставимо с такими же, но для подготовленных views. И значительно быстрее, чем подготовка нового view, с помощью ETL процедур.

Siemens: граф знаний - интеграционный хаб

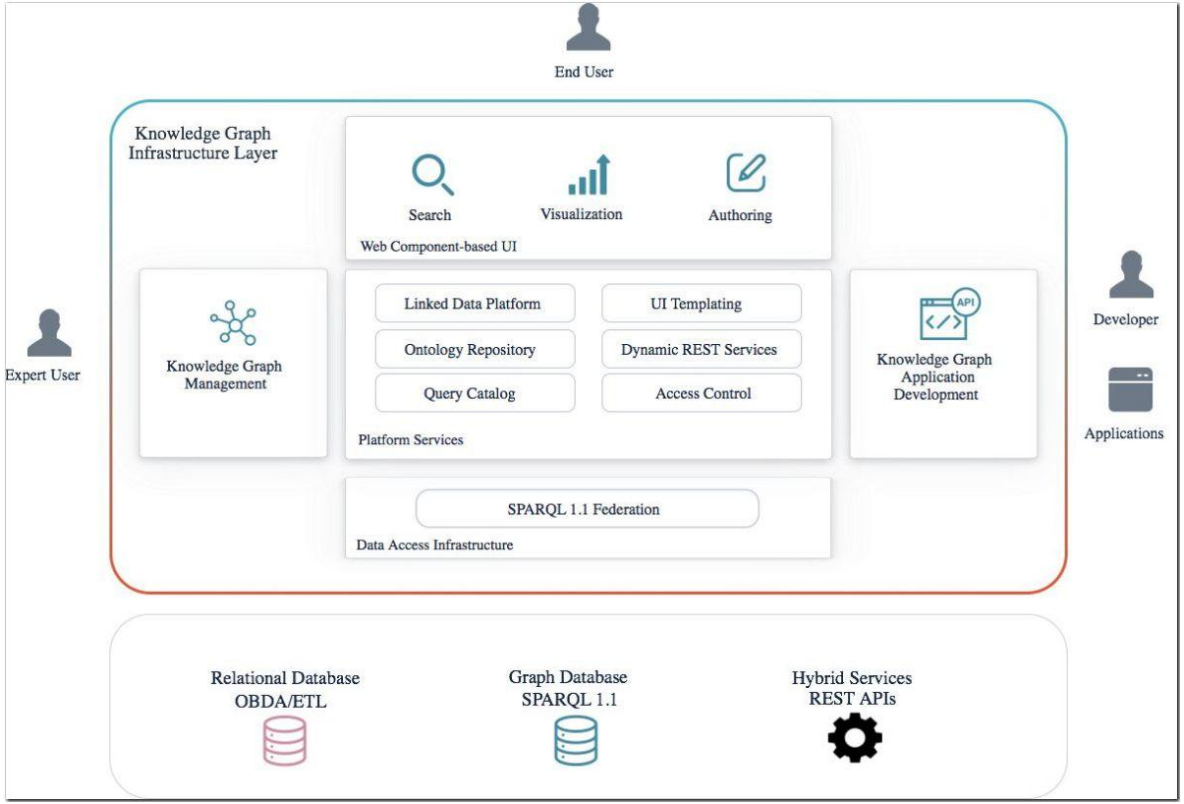
Примеры:

- виртуальная копия здания,
- риск менеджмент
  - финансы,
  - связи между сторонами,
  - местоположение, валюта и т.д.
- мониторинг процессов,
- обслуживание турбин и т.д.,
- мониторинг завода.

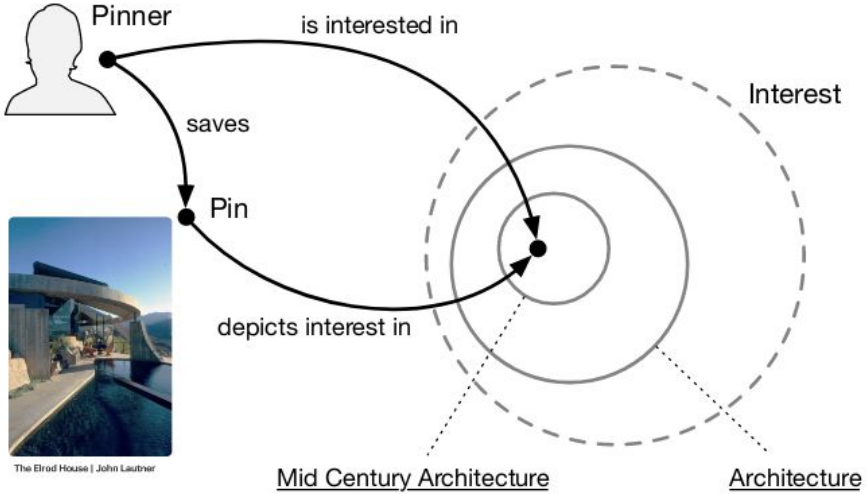


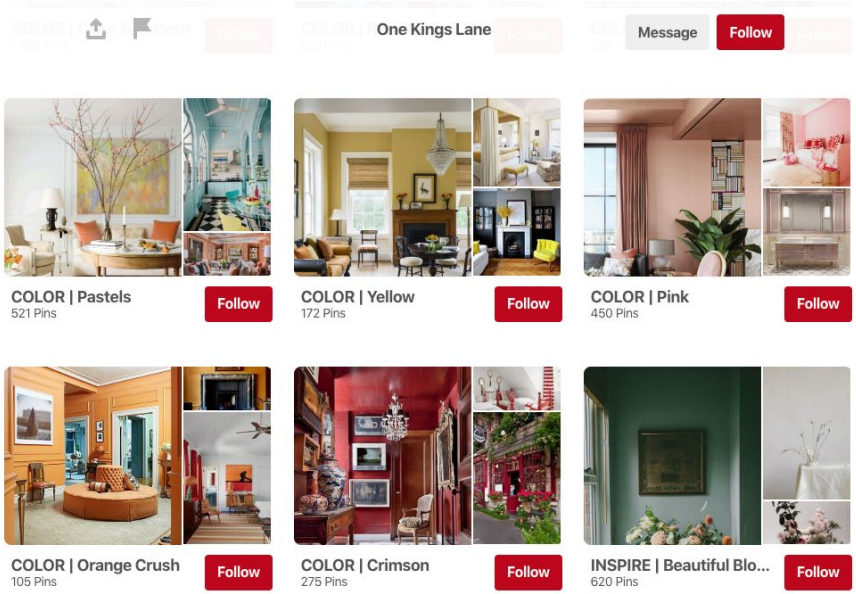
Источник: https://metaphacts.com/application-areas/engineering-and-manufacturing

Siemens: архитектура платформы



Pinterest: поиск и таргетированная реклама

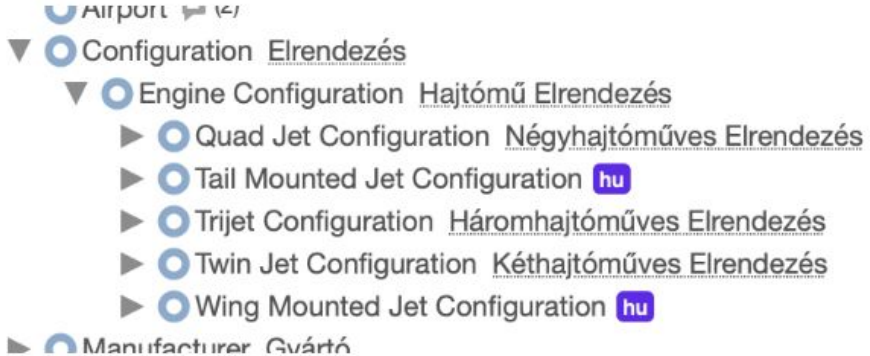




Источник: https://arxiv.org/pdf/1907.02106.pdf

Pinterest: таксономия интересов

- ~11 000 классов,
- до 12 уровней глубины иерархии,
- ~145 000 аксиом
  - ~25 000 логических,
  - ~95 000 аннотации.

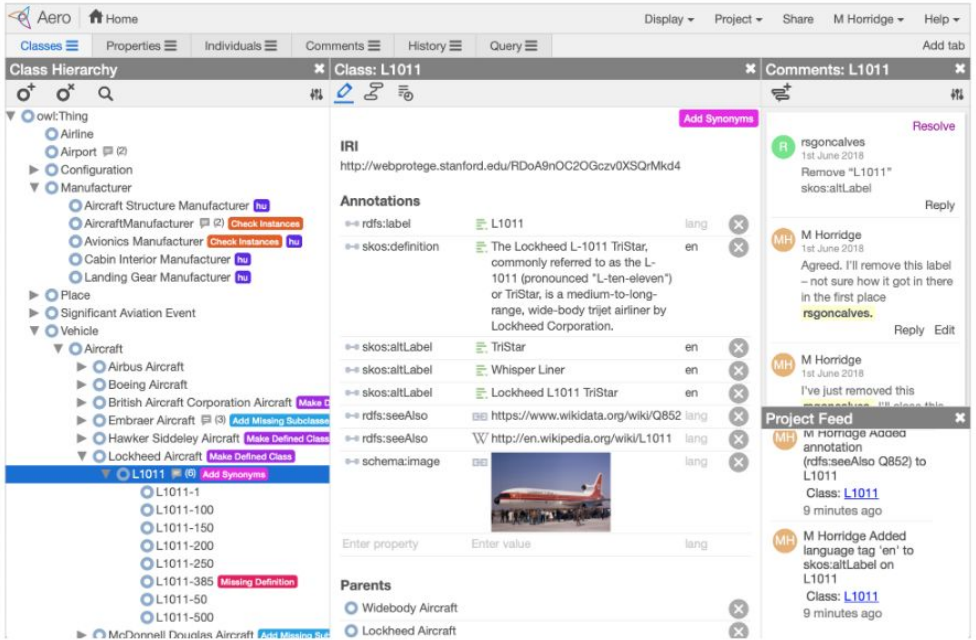


- ~38000 ревизий,
- ~2000 комментариев,
- ~1000 дискуссий.

Pinterest: эволюция процесса разработки

- от Excel таблиц,
- … внутренние инструменты,
- к WebProtege + расширения.

- 8 инженеров (без пред. опыта),
- 1 часовой тренинг,
- 2 месяца работы.

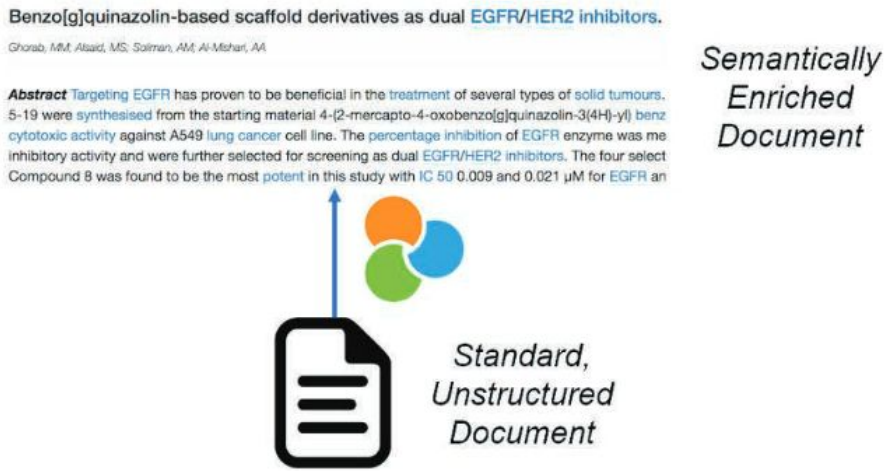


Pinterest: результаты

- Улучшили функциональность таргетированной рекламы
- … и поиска.
- Получили измеримый прирост в выручке (конкретных цифр нет).

SciBite: поиск научных статей

Применение онтологий и таксономий при извлечении информации из текстов (NLP) помогает обогатить эту информацию дополнительными фактами.



*The gene **PSEN1** was amplified by PCR using the primers listed below….*

*An assessment of the effect of a series of **Presenilin-1** mutations….*

*Genetic characterization of the early-onset Alzheimer's disease **AD3** locus*

***PSNL1** gene expression in human tissues at the tissue and cellular level*

Источник: https://www.scibite.com/wp-content/uploads/2018/04/SciBite-Use-Case-12-Semantics-in-Enterprise-Search.pdf

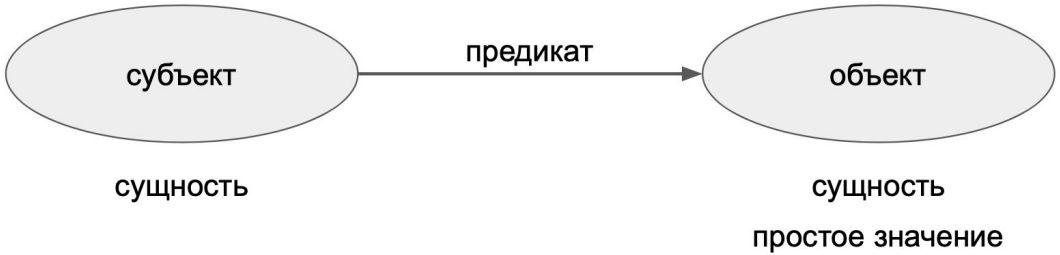
Другие примеры

- Zalando - семантический поиск товаров,
- GeoPhy - база данных недвижимости, алгоритмы оценки стоимости и т.д.
- и ещё больше: github.com/KMax/awesome-semantic-web-business
- и конечно Google, Apple, Amazon...

Примеры онтологий

Онтологии = основа графов знаний

- Формализация некоторой области знаний с помощью концептуальной схемы



Формы представления LPG vs RDF

Универсальные онтологии

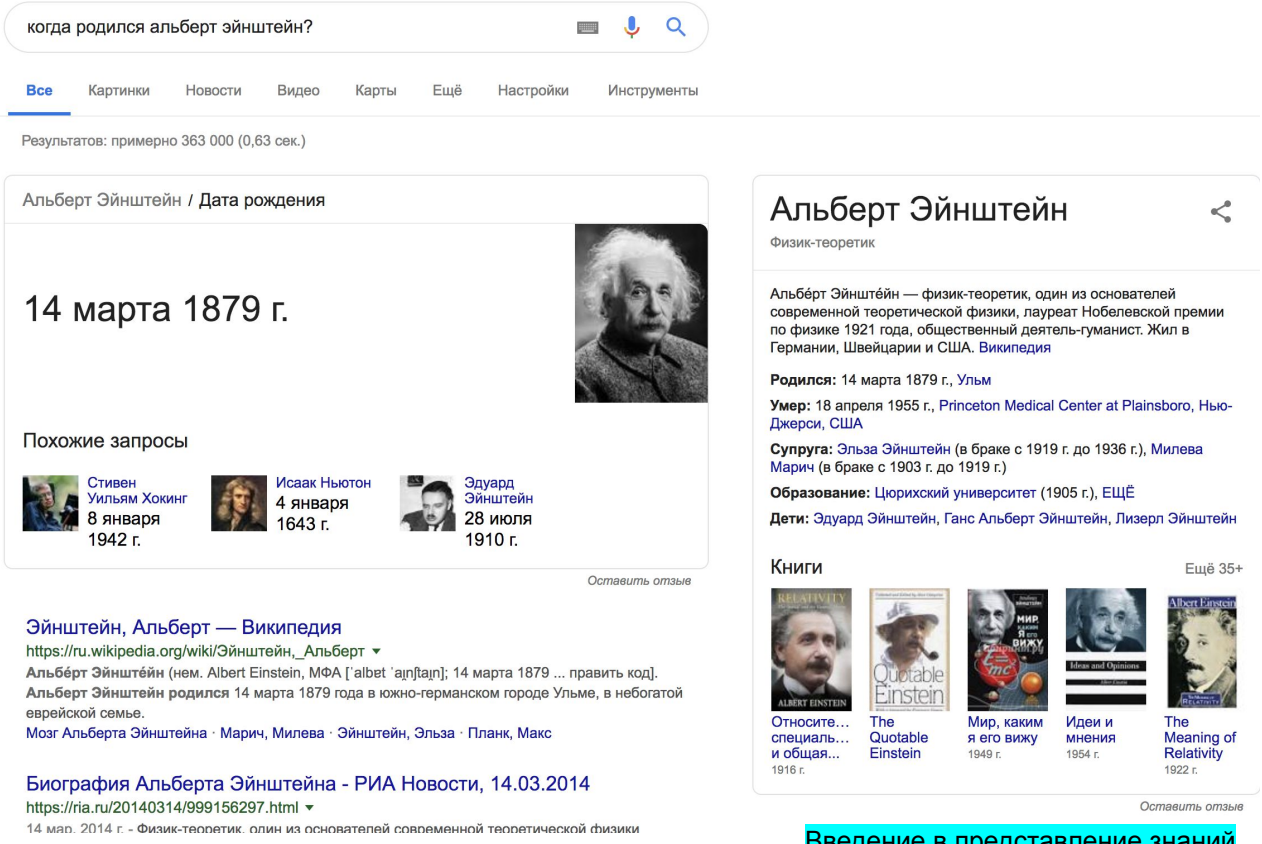
- SKOS Simple Knowledge Organization System https://www.w3.org/2004/02/skos/
- The Basic Formal Ontology (BFO) http://basic-formal-ontology.org
- DOLCE: a Descriptive Ontology for Linguistic and Cognitive
- Engineering http://www.loa.istc.cnr.it/old/DOLCE.htm

Специализированные онтологии

- FOAF Vocabulary http://xmlns.com/foaf/spec/
- The Organization Ontology https://www.w3.org/TR/vocab-org/
- Bibliographic Ontology http://bibliontology.com
- The CIDOC Conceptual Reference Model (CRM) http://www.cidoc-crm.org

Сценарии использования

1) Обогащение результатов поиска и ответы на простые вопросы



Введение в представление знаний

Сценарии использования

2) Ответы на сложные вопросы, заданные на естественном языке

AskNow

Which award did Albert Einstein receive?

Royal Society

The President, Council, and Fellows of the Royal Society of London for Improving Natural Knowledge, commonly known as the Royal Society, is a learned society for science and is possibly the oldest such society still in existence. Founded in November 1660, it was granted a royal charter by King Charles II as "The Royal Society". The Society is the United Kingdom's and Commonwealth of Nations' Academy of Sciences and fulfills a number of roles; promoting science and its benefits, recognising excellence in science, supporting outstanding science, providing scientific advice for policy, fostering international and global cooperation, education and public engagement. The society is governed by its Council, which is chaired by the Society's President, according to a set of statutes and standing orders. The members of Council and the President are elected from and by its Fellows, the basic members of the society, who are themselves elected by existing Fellows. As of 2016, there are about 1,600 fellows, allowed to use the postnominal title FRS (Fellow of the Royal

Nobel Prize in Physics

The Nobel Prize in Physics (Swedish: Nobelpriset i fysik) is a yearly award given by the Royal Swedish Academy of Sciences for those who conferred the most outstanding contributions for mankind in the field of physics. It is one of the five Nobel Prizes established by the will of Alfred Nobel in 1895 and awarded since 1901; the others being the Nobel Prize in Chemistry, Nobel Prize in Literature, Nobel Peace Prize, and Nobel Prize in Physiology or Medicine. The first Nobel Prize in Physics was awarded to a German physicist Wilhelm Röntgen in recognition of the extraordinary services he has rendered by the discovery of the remarkable rays (or x-rays). This award is administered by the Nobel Foundation and widely regarded as the most prestigious award that a scientist can receive in physics. It is presented in Stockholm at an annual ceremony on December 10, the anniversary of Nobel's death. Through 2016, a total of 203 individuals have been awarded the prize.

Matteucci Medal

The Matteucci Medal is an Italian award for physicists, named after Carlo Matteucci. It was established to award physicists for their fundamental contributions. Under an Italian Royal Decree dated July 10, 1870, the Italian Society of Sciences was authorized to receive a donation from Carlo Matteucci for the establishment of the Prize. Matteucci Medalists Source: Italian Society of Sciences] * 1868 Hermann Helmholtz * 1875 Henri Victor Regnault * 1876 William Thomson, 1st Baron Kelvin * 1877 Gustav Kirchhoff * 1878 Gustav Wiedemann * 1879 Wilhelm Eduard Weber * 1880 Antonio Pacinotti * 1881 Emilio Villari * 1882 Augusto Righi * 1887 Thomas Edison * 1888 Heinrich Rudolph Hertz * 1894 John Strutt, 3rd Baron Rayleigh * 1895 Henry Augustus Rowland * 1896 Wilhelm Conrad Röntgen and Philipp Lenard * 1901 Guglielmo Marconi * 1903 Albert Abraham Michelson * 1904 Marie Curie and Pierre Curie * 1905 Henri Poincaré * 1906 James Dewar * 1907 William Ramsay * 1908 Antonio Garbasso * 1909 Orso Mario Corbino * 1910 Heike

Max Planck Medal

The Max Planck medal is the highest award of the Deutsche Physikalische Gesellschaft (English: German Physical Society); the world's largest organization of physicists; for extraordinary achievements in theoretical physics. It is awarded annually. The winner is awarded with gold medal and hand-written parchment.

Show detailed

Copley Medal

The Copley Medal is a scientific award given by the Royal Society, London, for "outstanding achievements in research in any branch of science." It alternates between the physical and the biological sciences. Given every year, the medal is the oldest Royal Society medal still awarded, and probably the oldest surviving scientific award in the world, having first been given in 1731 to Stephen Gray, for "his new Electrical Experiments: – as an encouragement to him for the readiness he has always shown in obliging the Society with his discoveries and improvements in this part of Natural Knowledge".

Show detailed

Time 100: The Most Important People of the Century

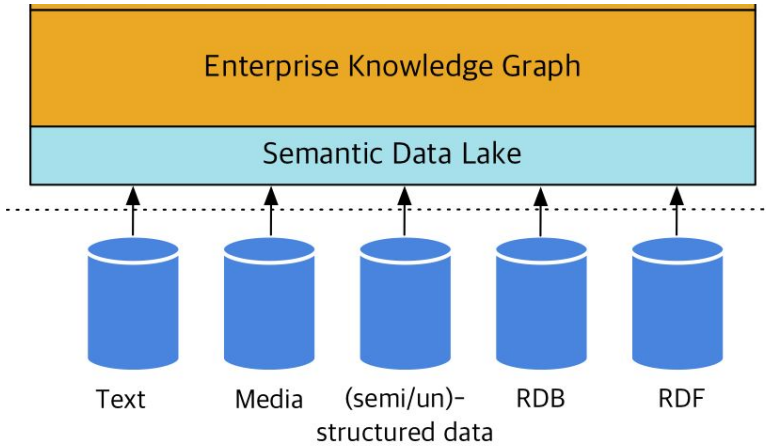
Time 100: The Most Important People of the Century is a compilation of the 20th century's 100 most influential people, published in Time magazine in 1999. The idea for such a list started on February 1, 1998, with a debate at a symposium in Hanoi, Vietnam. The panel participants were former CBS Evening News anchor Dan Rather, historian Doris Kearns Goodwin, former New York governor Mario Cuomo, then–Stanford Provost Condoleezza Rice, publisher Irving Kristol, and Time managing editor Walter Isaacson. In a separate issue on December 31, 1999, Time recognized Einstein as the Person of the Century.

Show detailed

Сценарии использования

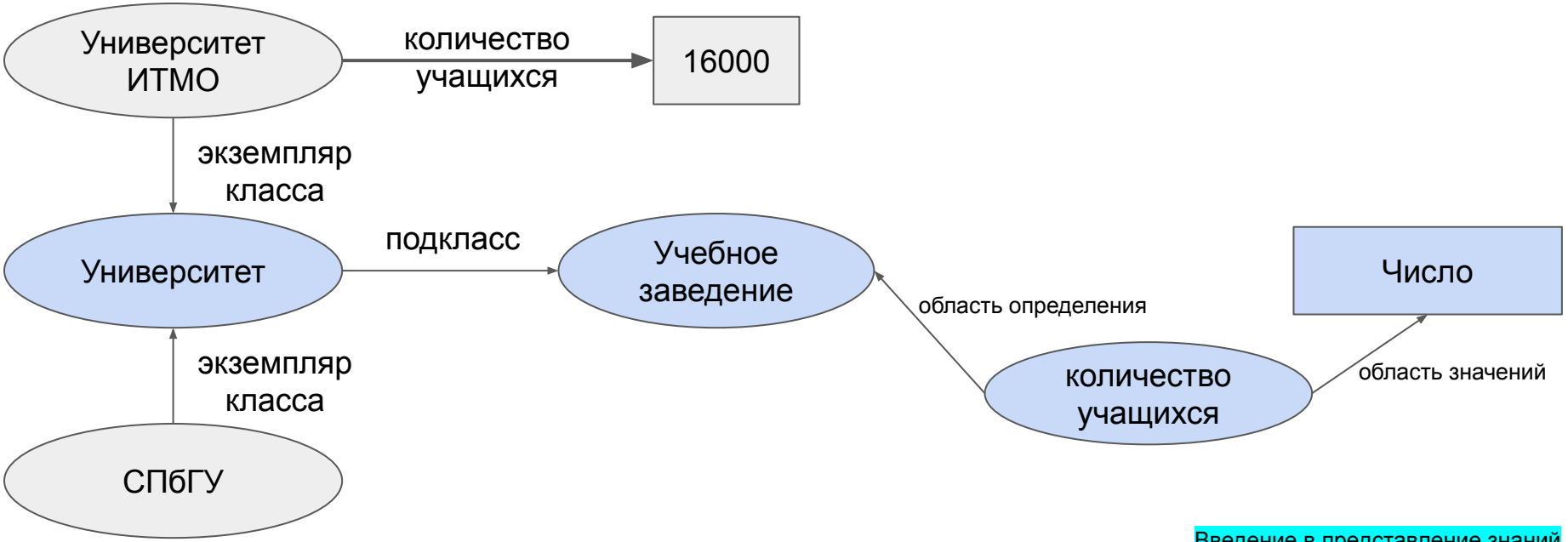
3) Интеграция данных из гетерогенных источников в один физический или виртуальный граф знаний

- Множество датасетов интегрируются с графом знаний
- Один запрос к графу пересылается нужным источникам



Отличия от RDB?

Университет	Количество учащихся
Университет ИТМО	16000
СПбГУ	N/A



Как же все устроено?

Продолжение после небольшого перерыва

2. Введение в RDF, RDFS, OWL.

Ресурс

… - это любая сущность: электронный документ, объект реального мира или даже абстрактное понятие

RDF (Resource Description Framework) позволяет

- кому угодно
- сказать что угодно
- о каком угодно ресурсе.

https://www.w3.org/RDF/

URI (Unified Resource Identifier)

- RDF позволяет кому угодно сказать что угодно о каком угодно ресурсе.
- Для нас необходимо каким-либо образом однозначно идентифицировать ресурс.
- В вебе уже существует технология глобальной идентификации ресурсов - URI https://www.w3.org/Addressing/URL/uri-spec.html

Решение: присвоить каждому ресурсу URI

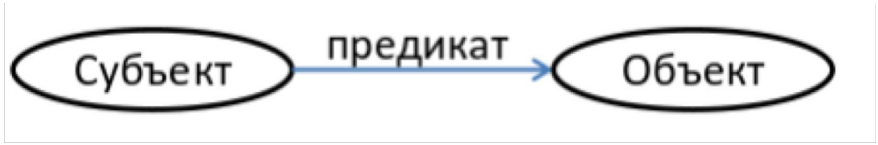
Частным случаем URI является всем известный URL.
Примеры URL: http://ifmo.ru, mailto:romanov@itmo.ru

Триплет

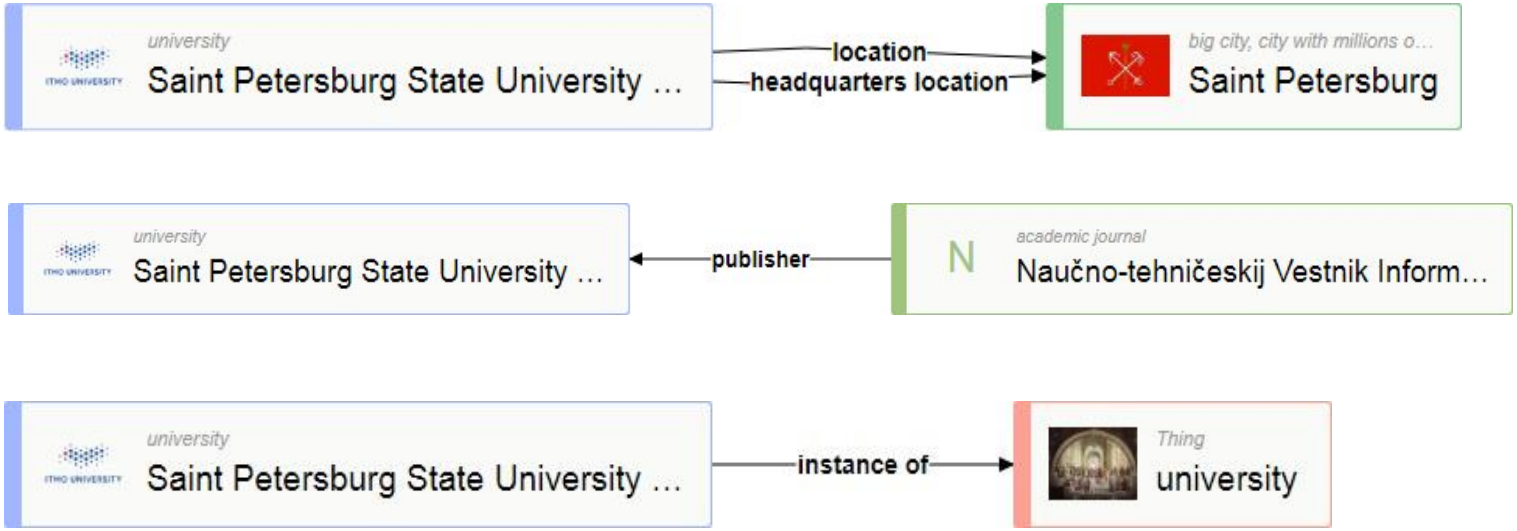
Высказывание о ресурсе представляется в виде триплета.

Триплет состоит из трех частей:

- субъект – описываемый ресурс
- предикат – свойство ресурса
- объект – значение свойства

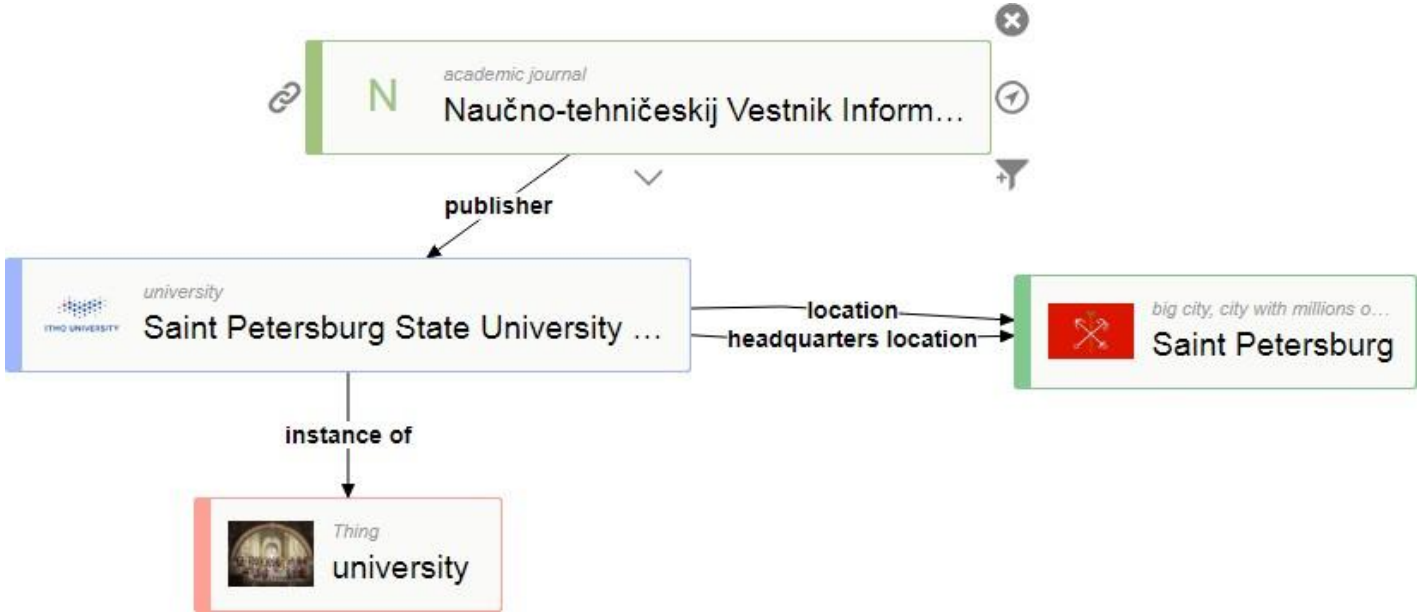


Триплеты


university
Saint Petersburg State University …
location
headquarters location
big city, city with millions o…
Saint Petersburg
university
Saint Petersburg State University …
publisher
academic journal
Naučno-tehničeskij Vestnik Inform…
university
Saint Petersburg State University …
instance of
Thing
university

RDF-граф

Триплеты из RDF-документа объединяются в RDF-граф.



RDF-датасет

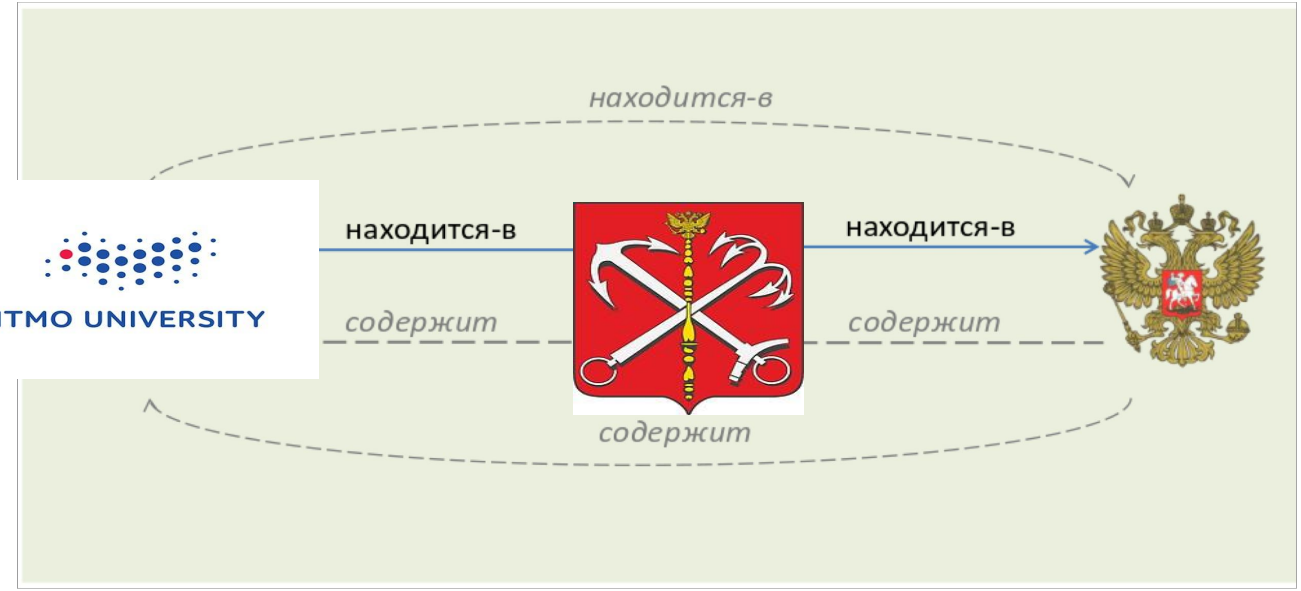
… - набор RDF-графов состоящий из

- *одного* графа по-умолчанию, у которого нет имени,
- и именованных графов.

Именованный граф — пара, состоящая из URI или анонимного узла, и RDF-графа.

Вывод триплетов

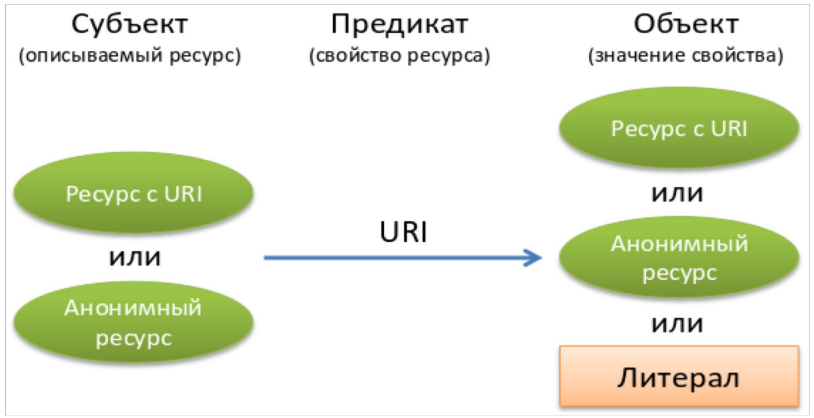
На основе онтологий с помощью логического вывода в RDF-графе вычисляется не существовавшие до этого триплеты



Состав триплета

Возможные значения элементов триплета:

- ресурс, задаваемый по URI,
- анонимный ресурс,
- литерал в формате Unicode.



Ресурс, задаваемый по URI

В качестве URI может выступать:

- URL ресурса (если этот ресурс — электр. документ)
  Например: Университет ИТМО - http://ifmo.ru

Подходит для документов, но **не** для электронных объектов реального мира.

Ресурс, задаваемый по URI

В качестве URI может выступать:

- URN — абстрактный URI, идентифицирующий ресурс, но не указывающий, где он расположен

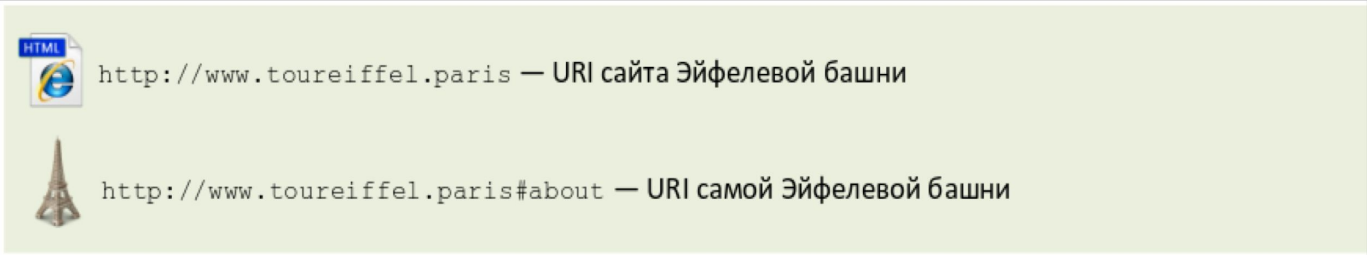
URI ресурса «Книга „Капитал. Том I“», определённого по коду ISBN:

urn:isbn:9660313837

Ресурс, задаваемый по URI

В качестве URI может выступать:

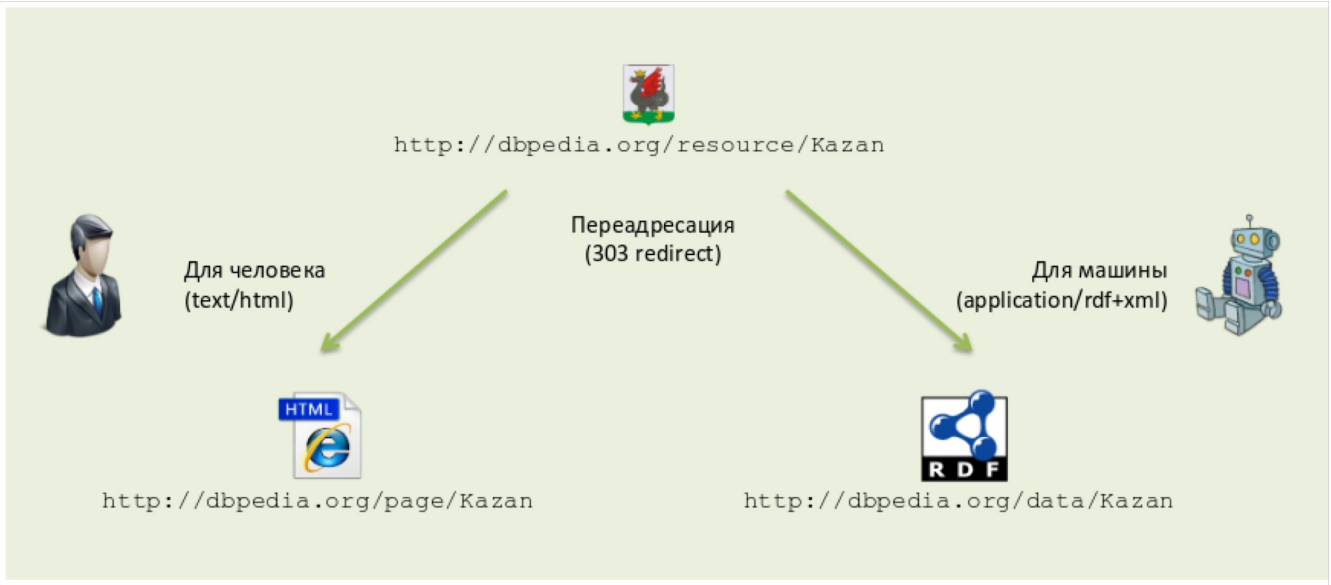
- URL фрагмента документа с описанием ресурса



Ресурс, задаваемый по URI

В качестве URI может выступать:

- Специальный URI с переадресацией



Литерал

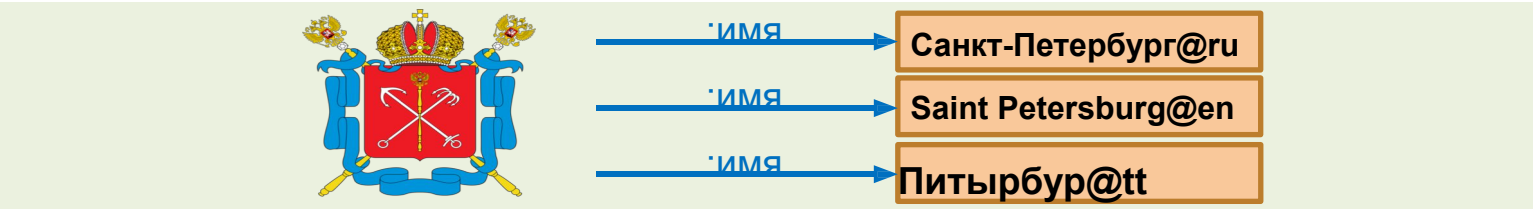
… - строка символов в кодировке Unicode.

Виды литералов:

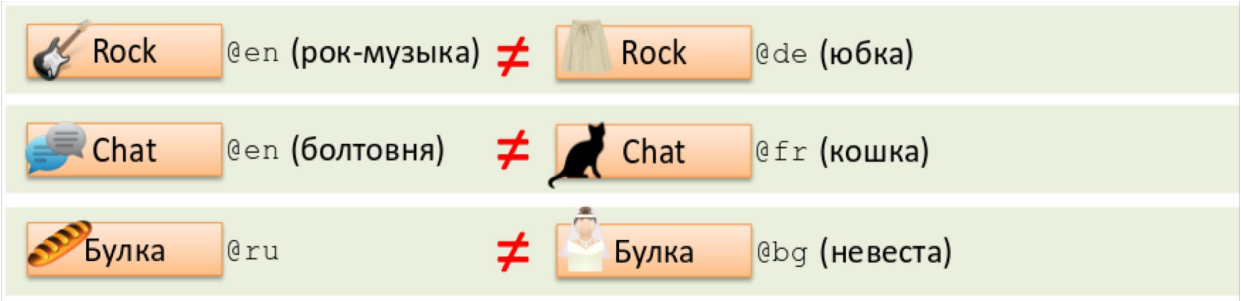
- Не типизированный литерал — просто строка символов;
- Литерал с меткой языка — текст на каком-то языке;
- Типизированный литерал — некое значение (число, дата, булевское значение и т.д.)

Литерал

Литерал с меткой языка — текст на каком-то языке. Состоит из: текста литерала в кодировке Unicode и метки языка в формате RFC 3066.



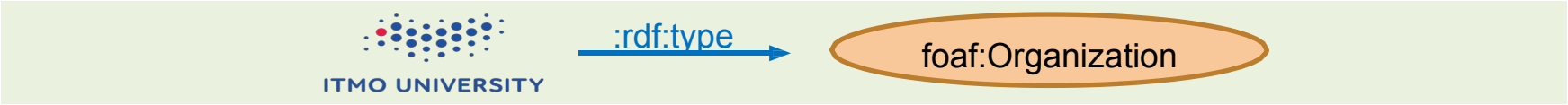
Литералы с одинаковым текстом, но разными кодами языка не равны друг другу.



Класс

… - это категория ресурсов. Например: Люди, Организации, Документы и т.д.

- Класс - это RDF-ресурс, задаваемый по URI.
- Классы описываются средствами RDFS и OWL.
- Принадлежность ресурса классу задается встроенным предикатом rdf:type.



- RDF содержит несколько встроенных классов:
  - rdf:Statement - для создания реификаций (утверждений об утверждениях);
  - rdf:Bag, rdf:Seg, rdf:Alt, rdf:List - для создания контейнеров и списков;
  - и некоторые другие.

Реификация

… - это утверждение об утверждении
https://www.w3.org/TR/rdf-primer/#reification
Для создания утверждения об утверждении создается ресурс относящийся к классу rdf:Statement и имеющий три свойства:

- rdf:subject - субъект утверждения;
- rdf:predicate - предикат;
- rdf:object - объект.

Контейнеры и списки

RDF позволяет группировать ресурсы (и литералы) и использовать их в высказываниях как единое целое.

Поддерживаются следующие виды групп:

- контейнеры
  - bag - неупорядоченный набор;
    Apple основал Стив Джобс, Стив Возняк и Рональд Уэйн.
  - alt - набор возможных альтернатив;
    Столицей XXIII Зимних Олимпийских игр будет Анси, Мюнхен и Пхенчхан.
  - seq - упорядоченный набор;
    В случае отставки президента США его замещают по порядку: вице-президент, спикер Палаты представителей, временный председатель Сената, государственный секретарь.

Неупорядоченный набор

Для создания неупорядоченного набора создается ресурс:

- относящийся к классу rdf:bag
- и имеющий свойства rdf:_1, rdf:_2, …, rdf:_n, указывающие на соответствующие элементы.

Упорядоченный набор

Для создания упорядоченного набора создается ресурс:

- относящийся к классу rdf:seq
- и имеющий свойства rdf:_1, rdf:_2, …, rdf:_n, указывающие на соответствующие элементы.

Коллекция

Коллекция подобна списку в языке Lisp. Для ее создания создается ресурс относящийся к классу rdf:list и имеющий два свойства:

- rdf:first (голова) - первый элемент коллекции;
- rdf:rest (хвост) - ссылка на под-коллекцию, содержащую оставшиеся элементы.

Способы публикации RDF-графа

- VoID - Describing Linked Datasets with the VoID Vocabulary
  - https://www.w3.org/TR/void/
  - См. раздел 3.6 Publishing Linked Data по ссылке https://euclid-project.eu/modules/chapter3.html
  - Другие словари https://lov.linkeddata.es/dataset/lov/vocabs/void
- Ресурсы Linked Open Vocabularies (LOV) и др.
  - https://lov.linkeddata.es/dataset/lov/
  - https://lod-cloud.net

Способы хранения RDF-графа

Два подхода:

- Triplestore + точка доступа (SPARQL endpoint).
- Сериализация в текстовый файл в специальном формате.

Сериализация RDF в файл

… — это представление RDF в машиночитаемом (текстовый, бинарный и т.д.) виде.

Форматы сериализации:

- RDF/XML - стандартный формат на базе XML,
- N-Triples - простой формат,
- Turtle - удобный формат,
- JSON-LD - формат на базе JSON,
- RDF Microdata - формат RDF-разметки HTML-страниц.

Сериализация RDF

Пример RDF/XML:

```
<rdf:RDF
 xmlns:foaf='http://xmlns.com/foaf/0.1/'
 xmlns:rdf='http://www.w3.org/1999/02/22-rdf-syntax-ns#'
 xmlns:rdfs='http://www.w3.org/2000/01/rdf-schema#'>
 <foaf:Person rdf:about="http://kiwitobes.com/toby.rdf#ts">
    <foaf:name>Toby Segaran</foaf:name>
    <foaf:homepage rdf:resource="http://kiwitobes.com/"/>
    <foaf:nick>kiwitobes</foaf:nick>
    <foaf:mbox rdf:resource="mailto:toby@segaran.com"/>
    <foaf:interest>
     <foaf:Document rdf:about="http://semprog.com">
        <rdfs:label>Semantic Programming</rdfs:label>
     </foaf:Document>
    </foaf:interest>
    <foaf:knows>
     <foaf:Person rdf:about="http://semprog.com/people/colin">
        <foaf:name>Colin Evans</foaf:name>
        <foaf:mbox rdf:resource="mailto:colin@semprog.com"/>
     </foaf:Person>
    </foaf:knows>
    <foaf:knows>
     <foaf:Person>
        <foaf:name>Jamie Taylor</foaf:name>
        <foaf:mbox rdf:resource="mailto:jamie@semprog.com"/>
     </foaf:Person>
    </foaf:knows>
 </foaf:Person>
</rdf:RDF>
<!-- (Из книги Segaran, Evans, Taylor «Programming the Semantic Web») -->
```

Сериализация RDF

Пример N-Triples:

```
<http://kiwitobes.com/toby.rdf#ts> <http://xmlns.com/foaf/0.1/homepage> <http://kiwitobes.com/>.
<http://kiwitobes.com/toby.rdf#ts> <http://xmlns.com/foaf/0.1/nick> "kiwitobes".
<http://kiwitobes.com/toby.rdf#ts> <http://xmlns.com/foaf/0.1/name> "Toby Segaran".
<http://kiwitobes.com/toby.rdf#ts> <http://xmlns.com/foaf/0.1/mbox> <mailto:toby@segaran.com>.
<http://kiwitobes.com/toby.rdf#ts> <http://xmlns.com/foaf/0.1/interest> <http://semprog.com>.
<http://kiwitobes.com/toby.rdf#ts> <http://www.w3.org/1999/02/22-rdf-syntax-ns#type> <http://xmlns.com/foaf/0.1/Person>.
<http://kiwitobes.com/toby.rdf#ts> <http://xmlns.com/foaf/0.1/knows> _:jamie .
<http://kiwitobes.com/toby.rdf#ts> <http://xmlns.com/foaf/0.1/knows> <http://semprog.com/people/colin>.
_:jamie <http://xmlns.com/foaf/0.1/name> "Jamie Taylor".
_:jamie <http://xmlns.com/foaf/0.1/mbox> <mailto:jamie@semprog.com>.
_:jamie <http://www.w3.org/1999/02/22-rdf-syntax-ns#type> <http://xmlns.com/foaf/0.1/Person>.
<http://semprog.com/people/colin> <http://xmlns.com/foaf/0.1/name> "Colin Evans".
<http://semprog.com/people/colin> <http://xmlns.com/foaf/0.1/mbox> <mailto:colin@semprog.com>.
<http://semprog.com/people/colin> <http://www.w3.org/1999/02/22-rdf-syntax-ns#type> <http://xmlns.com/foaf/0.1/Person>.
<http://semprog.com> <http://www.w3.org/2000/01/rdf-schema#label> "Semantic Programming".
<http://semprog.com> <http://www.w3.org/1999/02/22-rdf-syntax-ns#type> <http://xmlns.com/foaf/0.1/Document>.

# (Из книги Segaran, Evans, Taylor «Programming the Semantic Web»)
```

Сериализация RDF

Пример Turtle:

```
@prefix foaf: <http://xmlns.com/foaf/0.1/>.
@prefix rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>.
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#>.
@prefix semperp: <http://semprog.com/people/>.
@prefix tobes: <http://kiwitobes.com/toby.rdf#>.

 tobes:ts a foaf:Person;
     foaf:homepage <http://kiwitobes.com/>;
     foaf:interest <http://semprog.com>;
     foaf:knows semperp:colin,
         [ a foaf:Person;
             foaf:mbox <mailto:jamie@semprog.com>;
             foaf:name "Jamie Taylor"];
     foaf:mbox <mailto:toby@segaran.com>;
     foaf:name "Toby Segaran";
     foaf:nick "kiwitobes".
 <http://semprog.com> a foaf:Document;
     rdfs:label "Semantic Programming".
 semperp:colin a foaf:Person;
     foaf:mbox <mailto:colin@semprog.com>;
     foaf:name "Colin Evans".

# (Из книги Segaran, Evans, Taylor «Programming the Semantic Web»)
```

N-Triples

N-Triples состоит из простого перечисления триплетов. Триплет содержит субъект, предикат и объект, разделенные пробелом. В конце триплета ставится точка.

```
<http://example.com/vasya> <http://www.w3.org/1999/02/22-rdf-syntax-ns#type> <http://schema.org/Person>.
<http://example.com/vasya> <http://schema.org/name> "Вася Пупкин".
<http://example.com/vasya> <http://schema.org/email> <mailto:vasya.pupkin@gmail.com>.
<http://example.com/vasya> <http://schema.org/email> <mailto:vasily@example.com>.
<http://example.com/vasya> <http://schema.org/knows> <http://example.com/petya>.
<http://example.com/vasya> <http://schema.org/knows> <http://example.com/masha>.
```

N-Triples

N-Triples состоит из простого перечисления триплетов. Триплет содержит субъект, предикат и объект, разделенные пробелом. В конце триплета ставится точка.

```
<http://example.com/vasya> <http://www.w3.org/1999/02/22-rdf-syntax-ns#type> <http://schema.org/Person>.
<http://example.com/vasya> <http://schema.org/name> "Вася Пупкин".
<http://example.com/vasya> <http://schema.org/email> <mailto:vasya.pupkin@gmail.com>.
<http://example.com/vasya> <http://schema.org/email> <mailto:vasily@example.com>.
<http://example.com/vasya> <http://schema.org/knows> <http://example.com/petya>.
<http://example.com/vasya> <http://schema.org/knows> <http://example.com/masha>.
```

N-Triples

URI’и ресурсов записываются в <...>.

```
<http://example.com/vasya> <http://www.w3.org/1999/02/22-rdf-syntax-ns#type> <http://schema.org/Person>.
<http://example.com/vasya> <http://schema.org/name> "Вася Пупкин".
<http://example.com/vasya> <http://schema.org/email> <mailto:vasya.pupkin@gmail.com>.
<http://example.com/vasya> <http://schema.org/email> <mailto:vasily@example.com>.
<http://example.com/vasya> <http://schema.org/knows> <http://example.com/petya>.
<http://example.com/vasya> <http://schema.org/knows> <http://example.com/masha>.
```

N-Triples

Литералы записываются в кавычках.

```
<http://example.com/vasya> <http://www.w3.org/1999/02/22-rdf-syntax-ns#type> <http://schema.org/Person>.
<http://example.com/vasya> <http://schema.org/name> "Вася Пупкин".
<http://example.com/vasya> <http://schema.org/email> <mailto:vasya.pupkin@gmail.com>.
<http://example.com/vasya> <http://schema.org/email> <mailto:vasily@example.com>.
<http://example.com/vasya> <http://schema.org/knows> <http://example.com/petya>.
<http://example.com/vasya> <http://schema.org/knows> <http://example.com/masha>.
```

N-Triples

Код языка записывается после @.

```
<http://example.com/vasya> <http://www.w3.org/1999/02/22-rdf-syntax-ns#type> <http://schema.org/Person>.
<http://example.com/vasya> <http://schema.org/name> "Вася Пупкин"@ru.
<http://example.com/vasya> <http://schema.org/name> "Vasya Pupkin"@en.
<http://example.com/vasya> <http://schema.org/email> <mailto:vasya.pupkin@gmail.com>.
<http://example.com/vasya> <http://schema.org/email> <mailto:vasily@example.com>.
<http://example.com/vasya> <http://schema.org/knows> <http://example.com/petya>.
<http://example.com/vasya> <http://schema.org/knows> <http://example.com/masha>.
```

N-Triples

Тип данных литерала - после ^^.

```
<http://example.com/vasya> <http://www.w3.org/1999/02/22-rdf-syntax-ns#type> <http://schema.org/Person>.
<http://example.com/vasya> <http://schema.org/name> "Вася Пупкин"@ru.
<http://example.com/vasya> <http://schema.org/name> "Vasya Pupkin"@en.
<http://example.com/vasya> <http://schema.org/birthDate> "1990-04-30"^^<http://www.w3.org/2001/XMLSchema#date>.
<http://example.com/vasya> <http://schema.org/email> <mailto:vasya.pupkin@gmail.com>.
<http://example.com/vasya> <http://schema.org/email> <mailto:vasily@example.com>.
<http://example.com/vasya> <http://schema.org/knows> <http://example.com/petya>.
<http://example.com/vasya> <http://schema.org/knows> <http://example.com/masha>.
```

N-Triples

Для того, чтобы описать анонимный узел, ему дают служебный идентификатор.

Идентификатор записывается так _:идентификатор

Идентификатор - это не URI, на него нельзя сослаться из других

```
<http://example.com/vasya> <http://www.w3.org/1999/02/22-rdf-syntax-ns#type> <http://schema.org/Person>.
<http://example.com/vasya> <http://schema.org/name> "Вася Пупкин"@ru.
<http://example.com/vasya> <http://schema.org/name> "Vasya Pupkin"@en.
<http://example.com/vasya> <http://schema.org/birthDate> "1990-04-30"^^<http://www.w3.org/2001/XMLSchema#date>.
<http://example.com/vasya> <http://schema.org/email> <mailto:vasya.pupkin@gmail.com>.
<http://example.com/vasya> <http://schema.org/email> <mailto:vasily@example.com>.
<http://example.com/vasya> <http://schema.org/knows> <http://example.com/petya>.
<http://example.com/vasya> <http://schema.org/knows> <http://example.com/masha>.
<http://example.com/vasya> <http://schema.org/knows> _:person1.

_:person1 <http://www.w3.org/1999/02/22-rdf-syntax-ns#type> <http://schema.org/Person>.
_:person1 <http://schema.org/name> "Марья Ивановна"@ru.
_:person1 <http://schema.org/name> "Maria Ivanovna"@en.
```

Turtle

Turtle является расширением N-Triples. Поэтому, любой N-Triples-документ также является и Turtle-документом. Turtle позволяет записывать триплеты в более понятном и компактном виде.

```
<http://example.com/vasya> <http://www.w3.org/1999/02/22-rdf-syntax-ns#type> <http://schema.org/Person>.
<http://example.com/vasya> <http://schema.org/name> "Вася Пупкин"@ru.
<http://example.com/vasya> <http://schema.org/name> "Vasya Pupkin"@en.
<http://example.com/vasya> <http://schema.org/birthDate> "1990-04-30"^^<http://www.w3.org/2001/XMLSchema#date>.
<http://example.com/vasya> <http://schema.org/email> <mailto:vasya.pupkin@gmail.com>.
<http://example.com/vasya> <http://schema.org/email> <mailto:vasily@example.com>.
<http://example.com/vasya> <http://schema.org/knows> <http://example.com/petya>.
<http://example.com/vasya> <http://schema.org/knows> <http://example.com/masha>.
<http://example.com/vasya> <http://schema.org/knows> _:person1.

_:person1 <http://www.w3.org/1999/02/22-rdf-syntax-ns#type> <http://schema.org/Person>.
_:person1 <http://schema.org/name> "Марья Ивановна"@ru.
_:person1 <http://schema.org/name> "Maria Ivanovna"@en.
```

Turtle

URI'и с повторяющимися префиксами можно сократить, присвоив префиксу имя.
Имя присваивается так: «@prefix имя-префикса:<префикс> .».

Сокращенный URI записывается так: имя-префикса:остаток-uri (без <...>).

```
@prefix : <http://example.com/>. #Префикс с пустым именем
@prefix rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>.
@prefix schema: <http://schema.org/>.


<http://example.com/vasya> <http://www.w3.org/1999/02/22-rdf-syntax-ns#type> <http://schema.org/Person>.
<http://example.com/vasya> <http://schema.org/name> "Вася Пупкин"@ru.
<http://example.com/vasya> <http://schema.org/name> "Vasya Pupkin"@en.
<http://example.com/vasya> <http://schema.org/birthDate> "1990-04-30"^^<http://www.w3.org/2001/XMLSchema#date>.
<http://example.com/vasya> <http://schema.org/email> <mailto:vasya.pupkin@gmail.com>.
<http://example.com/vasya> <http://schema.org/email> <mailto:vasily@example.com>.
<http://example.com/vasya> <http://schema.org/knows> <http://example.com/petya>.
<http://example.com/vasya> <http://schema.org/knows> <http://example.com/masha>.
<http://example.com/vasya> <http://schema.org/knows> _:person1.

_:person1 <http://www.w3.org/1999/02/22-rdf-syntax-ns#type> <http://schema.org/Person>.
_:person1 <http://schema.org/name> "Марья Ивановна"@ru.
_:person1 <http://schema.org/name> "Maria Ivanovna"@en.
```

Turtle

URI'и с повторяющимися префиксами можно сократить, присвоив префиксу имя.
Имя присваивается так: «@prefix имя-префикса:<префикс> .».

Сокращенный URI записывается так: имя-префикса:остаток-uri (без <...>).

```
@prefix : <http://example.com/>. #Префикс с пустым именем
@prefix rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>.
@prefix schema: <http://schema.org/>.


:vasya rdf:type schema:Person.
:vasya schema:name "Вася Пупкин"@ru.
:vasya schema:name "Vasya Pupkin"@en.
:vasya schema:birthDate "1990-04-30"^^xsd:date.
:vasya schema:email <mailto:vasya.pupkin@gmail.com>.
:vasya schema:email <mailto:vasily@example.com>.
:vasya schema:knows :petya.
:vasya schema:knows :masha.
:vasya schema:knows _:person1.

_:person1 rdf:type schema:Person.
_:person1 schema:name "Марья Ивановна"@ru.
_:person1 schema:name "Maria Ivanovna"@en.
```

Turtle

```
@prefix : <http://example.com/>. #Префикс с пустым именем
@prefix rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>.
@prefix schema: <http://schema.org/>.


:vasya rdf:type schema:Person.
:vasya schema:name "Вася Пупкин"@ru.
:vasya schema:name "Vasya Pupkin"@en.
:vasya schema:birthDate "1990-04-30"^^xsd:date.
:vasya schema:email <mailto:vasya.pupkin@gmail.com>.
:vasya schema:email <mailto:vasily@example.com>.
:vasya schema:knows :petya.
:vasya schema:knows :masha.
:vasya schema:knows _:person1.

_:person1 rdf:type schema:Person.
_:person1 schema:name "Марья Ивановна"@ru.
_:person1 schema:name "Maria Ivanovna"@en.
```

schema:name ↔ http://schema.org/name

Turtle

Повторяющиеся объекты можно опустить.
Триплет с опущенным от предыдущего субъектом отделяется символом ; .

```
@prefix : <http://example.com/>.
@prefix rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>.
@prefix schema: <http://schema.org/>.


:vasya rdf:type schema:Person;
       schema:name "Вася Пупкин"@ru;
       schema:name "Vasya Pupkin"@en;
       schema:birthDate "1990-04-30"^^xsd:date;
       schema:email <mailto:vasya.pupkin@gmail.com>;
       schema:email <mailto:vasily@example.com>;
       schema:knows :petya;
       schema:knows :masha;
       schema:knows _:person1.

_:person1 rdf:type schema:Person;
          schema:name "Марья Ивановна"@ru;
          schema:name "Maria Ivanovna"@en.

#Отступы в примере добавлены для форматирования, они необязательны
```

Turtle

Если повторяется и субъект, и предикат - их также можно опустить. Триплет с опущенным субъектом и предикатом отделяется от предыдущего символом «,».

```
@prefix : <http://example.com/>.
@prefix rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>.
@prefix schema: <http://schema.org/>.


:vasya rdf:type schema:Person;
       schema:name "Вася Пупкин"@ru,
                   "Vasya Pupkin"@en;
       schema:birthDate "1990-04-30"^^xsd:date;
       schema:email <mailto:vasya.pupkin@gmail.com>,
                    <mailto:vasily@example.com>;
       schema:knows :petya,
                    :masha,
                    _:person1.

_:person1 rdf:type schema:Person;
          schema:name "Марья Ивановна"@ru,
                      "Maria Ivanovna"@en.
```

Turtle

Анонимный узел можно записать также в [...].

Субъект в [...] всегда опущен.

```
@prefix : <http://example.com/>.
@prefix rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>.
@prefix schema: <http://schema.org/>.


:vasya rdf:type schema:Person;
       schema:name "Вася Пупкин"@ru,
                   "Vasya Pupkin"@en;
       schema:birthDate "1990-04-30"^^xsd:date;
       schema:email <mailto:vasya.pupkin@gmail.com>,
                    <mailto:vasily@example.com>;
       schema:knows :petya,
                    :masha,
                    [
                    rdf:type schema:Person;
                    schema:name "Марья Ивановна"@ru,
                                "Maria Ivanovna"@en
                    ].
```

Turtle

rdf:type можно заменить на a.

```
@prefix : <http://example.com/>.
@prefix rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>.
@prefix schema: <http://schema.org/>.


:vasya a schema:Person;
       schema:name "Вася Пупкин"@ru,
                   "Vasya Pupkin"@en;
       schema:birthDate "1990-04-30"^^xsd:date;
       schema:email <mailto:vasya.pupkin@gmail.com>,
                    <mailto:vasily@example.com>;
       schema:knows :petya,
                    :masha,
                    [
                    a schema:Person;
                    schema:name "Марья Ивановна"@ru,
                                "Maria Ivanovna"@en
                    ].
```

Turtle

Коллекция записывается в (...).

Ее элементы отделяются пробелом.

```
@prefix : <http://example.com/>.
@prefix rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>.
@prefix schema: <http://schema.org/>.
@prefix xfn: <http://gmpg.org/xfn/1#>.


:vasya rdf:type schema:Person;
       schema:name "Вася Пупкин"@ru,
                   "Vasya Pupkin"@en;
       schema:birthDate "1990-04-30"^^xsd:date;
       schema:email <mailto:vasya.pupkin@gmail.com>,
                    <mailto:vasily@example.com>;
       schema:knows :petya,
                    :masha,
                    [
                    rdf:type schema:Person;
                    schema:name "Марья Ивановна"@ru,
                                "Maria Ivanovna"@en
                    ].

:wolf xfn:crush (:nif-nif :noof-noof :naf-naf). #Волк совсем потерял голову от трех поросят (от всех в целом)
```

RDF/XML

Корнем RDF/XML документа является элемент rdf:RDF

```
<?xml version="1.0"?>
<rdf:RDF xmlns:rdf="http://www.w3.org/1999/02/22-rdf-syntax-ns#">

</rdf:RDF>
```

RDF/XML

В нем можно объявить необходимые пространства имен.

```
<?xml version="1.0"?>
<rdf:RDF xmlns:rdf="http://www.w3.org/1999/02/22-rdf-syntax-ns#"
         xmlns:foaf="http://xmlns.com/foaf/0.1/"
         xmlns:schema="http://schema.org/">
</rdf:RDF>
```

RDF/XML

Описание ресурса задается с помощью элемента rdf:Description.

```
<?xml version="1.0"?>
<rdf:RDF xmlns:rdf="http://www.w3.org/1999/02/22-rdf-syntax-ns#"
         xmlns:foaf="http://xmlns.com/foaf/0.1/"
         xmlns:schema="http://schema.org/">
  <rdf:Description>
  </rdf:Description>
</rdf:RDF>
```

RDF/XML

URI ресурса - через атрибут rdf:about.

```
<?xml version="1.0"?>
<rdf:RDF xmlns:rdf="http://www.w3.org/1999/02/22-rdf-syntax-ns#"
         xmlns:foaf="http://xmlns.com/foaf/0.1/"
         xmlns:schema="http://schema.org/">
  <rdf:Description rdf:about="http://example.com/vasya">
  </rdf:Description>
</rdf:RDF>
```

RDF/XML

Свойства описываются с помощью вложенных элементов.

```
<?xml version="1.0"?>
<rdf:RDF xmlns:rdf="http://www.w3.org/1999/02/22-rdf-syntax-ns#"
         xmlns:foaf="http://xmlns.com/foaf/0.1/"
         xmlns:schema="http://schema.org/">
  <rdf:Description rdf:about="http://example.com/vasya">
    <foaf:name>Вася Пупкин</foaf:name>
    <schema:birthDate>1990-09-30</schema:birthDate>
  </rdf:Description>
</rdf:RDF>
```

RDF/XML

Замечание: URI свойства = URI пространства имен + имя свойства

```
<?xml version="1.0"?>
<rdf:RDF xmlns:rdf="http://www.w3.org/1999/02/22-rdf-syntax-ns#"
         xmlns:foaf="http://xmlns.com/foaf/0.1/"
         xmlns:schema="http://schema.org/">
  <rdf:Description rdf:about="http://example.com/vasya">
    <foaf:name>Вася Пупкин</foaf:name>
    <schema:birthDate>1990-09-30</schema:birthDate>
  </rdf:Description>
</rdf:RDF>
```

foaf:name ↔ http://xmlns.com/foaf/0.1/name

RDF/XML

Значением свойства может быть другой ресурс

```
<?xml version="1.0"?>
<rdf:RDF xmlns:rdf="http://www.w3.org/1999/02/22-rdf-syntax-ns#"
         xmlns:foaf="http://xmlns.com/foaf/0.1/"
         xmlns:schema="http://schema.org/">
  <rdf:Description rdf:about="http://example.com/vasya">
    <foaf:name>Вася Пупкин</foaf:name>
    <schema:birthDate>1990-09-30</schema:birthDate>
    <foaf:knows>
      <rdf:Description rdf:about="http://example.com/masha">
      </rdf:Description>
    </foaf:knows>
  </rdf:Description>
</rdf:RDF>
```

RDF/XML

Он может иметь свои свойства

```
<?xml version="1.0"?>
<rdf:RDF xmlns:rdf="http://www.w3.org/1999/02/22-rdf-syntax-ns#"
         xmlns:foaf="http://xmlns.com/foaf/0.1/"
         xmlns:schema="http://schema.org/">
  <rdf:Description rdf:about="http://example.com/vasya">
    <foaf:name>Вася Пупкин</foaf:name>
    <schema:birthDate>1990-09-30</schema:birthDate>
    <foaf:knows>
      <rdf:Description rdf:about="http://example.com/masha">
        <foaf:name>Маша Пупкина</foaf:name>
      </rdf:Description>
    </foaf:knows>
  </rdf:Description>
</rdf:RDF>
```

RDF/XML

...значением которых могут быть ресурсы (и так до бесконечности)

```
<?xml version="1.0"?>
<rdf:RDF xmlns:rdf="http://www.w3.org/1999/02/22-rdf-syntax-ns#"
         xmlns:foaf="http://xmlns.com/foaf/0.1/"
         xmlns:schema="http://schema.org/">
  <rdf:Description rdf:about="http://example.com/vasya">
    <foaf:name>Вася Пупкин</foaf:name>
    <schema:birthDate>1990-09-30</schema:birthDate>
    <foaf:knows>
      <rdf:Description rdf:about="http://example.com/masha">
        <foaf:name>Маша Пупкина</foaf:name>
        <foaf:interest>
          <rdf:Description rdf:about="http://dbpedia.org/resource/Pop_music"></rdf:Description>
        </foaf:interest>
      </rdf:Description>
    </foaf:knows>
  </rdf:Description>
</rdf:RDF>
```

RDF/XML

Метка языка литерала задается через атрибут xml:lang.

```
<?xml version="1.0"?>
<rdf:RDF xmlns:rdf="http://www.w3.org/1999/02/22-rdf-syntax-ns#"
         xmlns:foaf="http://xmlns.com/foaf/0.1/"
         xmlns:schema="http://schema.org/">
  <rdf:Description rdf:about="http://example.com/vasya">
    <foaf:name xml:lang="ru">Вася Пупкин</foaf:name>
    <schema:birthDate>1990-09-30</schema:birthDate>
    <foaf:knows>
      <rdf:Description rdf:about="http://example.com/masha">
        <foaf:name xml:lang="ru">Маша Пупкина</foaf:name>
        <foaf:interest>
          <rdf:Description rdf:about="http://dbpedia.org/resource/Pop_music"></rdf:Description>
        </foaf:interest>
      </rdf:Description>
    </foaf:knows>
  </rdf:Description>
</rdf:RDF>
```

Microdata

Основная идея Microdata - на web-странице уже есть вся информация, надо только разместить ее в понятном машине виде.

```
<div>
  <h1>ООО «Хабр»</h1>
  <img src="http://habrahabr.ru/images/1432119938/logo.svg">
  <span>
    105082, Москва, Спартаковский переулок, д. 2, стр. 1, секция 3
  </span>
  <span>+7 499 653-59-61</span>
  <time datetime="Mo-Su 6:00-23:00">
    Работаем круглосуточно с 6:00 до 23:00
  </time>
  <a href="mailto:adv@tmtm.ru">Напишете нам!</span>
</div>
```

Microdata

Укажем, что данный HTML-элемент описывает некоторый объект с помощью атрибута itemscope.

```
<div itemscope>
  <h1>ООО «Хабр»</h1>
  <img src="http://habrahabr.ru/images/1432119938/logo.svg">
  <span>
    105082, Москва, Спартаковский переулок, д. 2, стр. 1, секция 3
  </span>
  <span>+7 499 653-59-61</span>
  <time datetime="Mo-Su 6:00-23:00">
    Работаем круглосуточно с 6:00 до 23:00
  </time>
  <a href="mailto:adv@tmtm.ru">Напишете нам!</span>
</div>
```

Microdata

Укажите тип этого объекта с помощью атрибута itemtype.

```
<div itemscope itemtype="http://schema.org/Organization">
  <h1>ООО «Хабр»</h1>
  <img src="http://habrahabr.ru/images/1432119938/logo.svg">
  <span>
    105082, Москва, Спартаковский переулок, д. 2, стр. 1, секция 3
  </span>
  <span>+7 499 653-59-61</span>
  <time datetime="Mo-Su 6:00-23:00">
    Работаем круглосуточно с 6:00 до 23:00
  </time>
  <a href="mailto:adv@tmtm.ru">Напишете нам!</span>
</div>
```

Microdata

Укажите его URI с помощью атрибута itemid.

```
<div itemscope itemtype="http://schema.org/Organization" itemid="http://tmtm.ru/">
  <h1>ООО «Хабр»</h1>
  <img src="http://habrahabr.ru/images/1432119938/logo.svg">
  <span>
    105082, Москва, Спартаковский переулок, д. 2, стр. 1, секция 3
  </span>
  <span>+7 499 653-59-61</span>
  <time datetime="Mo-Su 6:00-23:00">
    Работаем круглосуточно с 6:00 до 23:00
  </time>
  <a href="mailto:adv@tmtm.ru">Напишете нам!</span>
</div>
```

Microdata

Разметим его свойства с помощью атрибута itemprop.

```
<div itemscope itemtype="http://schema.org/Organization" itemid="http://tmtm.ru/">
  <h1 itemprop="http://schema.org/name">ООО «Хабр»</h1>
  <img itemprop="http://schema.org/logo" src="http://habrahabr.ru/images/1432119938/logo.svg">
  <span itemprop="http://schema.org/address">
    105082, Москва, Спартаковский переулок, д. 2, стр. 1, секция 3
  </span>
  <span itemprop="http://schema.org/telephone">+7 499 653-59-61</span>
  <time itemprop="http://schema.org/openingHours" datetime="Mo-Su 6:00-23:00">
    Работаем круглосуточно с 6:00 до 23:00
  </time>
  <a itemprop="http://schema.org/email" href="mailto:adv@tmtm.ru">Напишете нам!</span>
</div>
```

Microdata

URI свойств можно сократить.

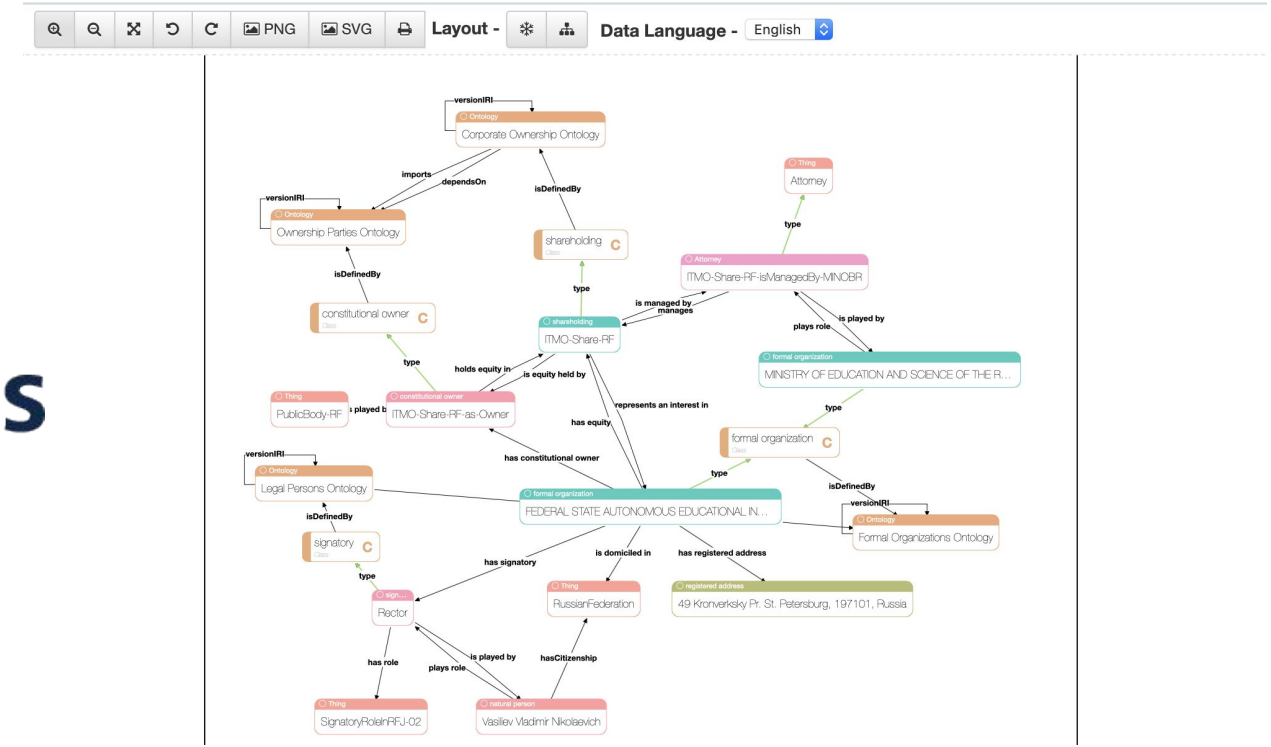
См. подробности:

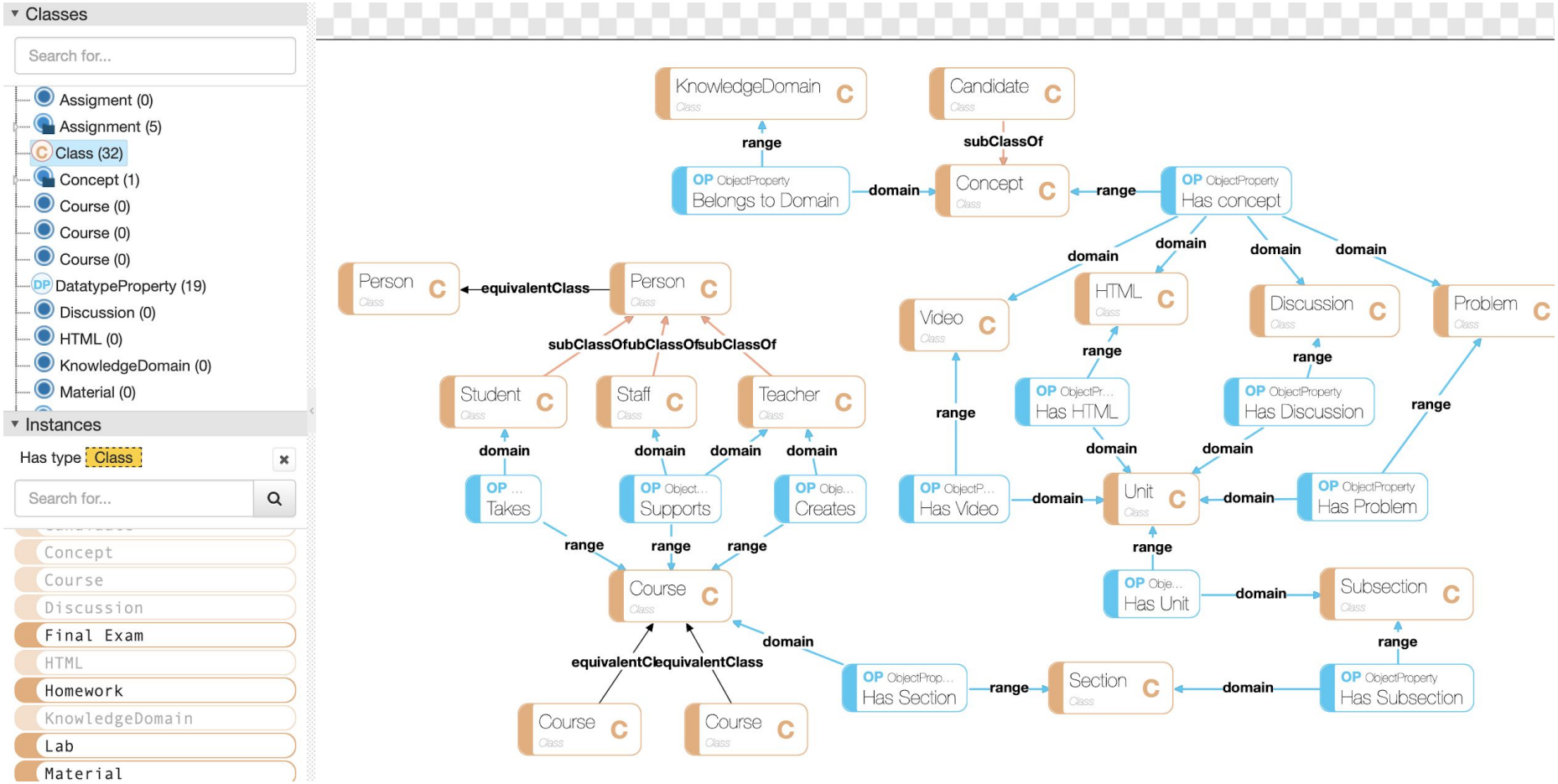
http://www.w3.org/TR/microdata-rdf/#property-uri-generation

```
<div itemscope itemtype="http://schema.org/Organization" itemid="http://tmtm.ru/">
  <h1 itemprop="http://schema.org/name">ООО «Хабр»</h1>
  <img itemprop="http://schema.org/logo" src="http://habrahabr.ru/images/1432119938/logo.svg">
  <span itemprop="http://schema.org/address">
    105082, Москва, Спартаковский переулок, д. 2, стр. 1, секция 3
  </span>
  <span itemprop="http://schema.org/telephone">+7 499 653-59-61</span>
  <time itemprop="http://schema.org/openingHours" datetime="Mo-Su 6:00-23:00">
    Работаем круглосуточно с 6:00 до 23:00
  </time>
  <a itemprop="http://schema.org/email" href="mailto:adv@tmtm.ru">Напишете нам!</span>
</div>
```

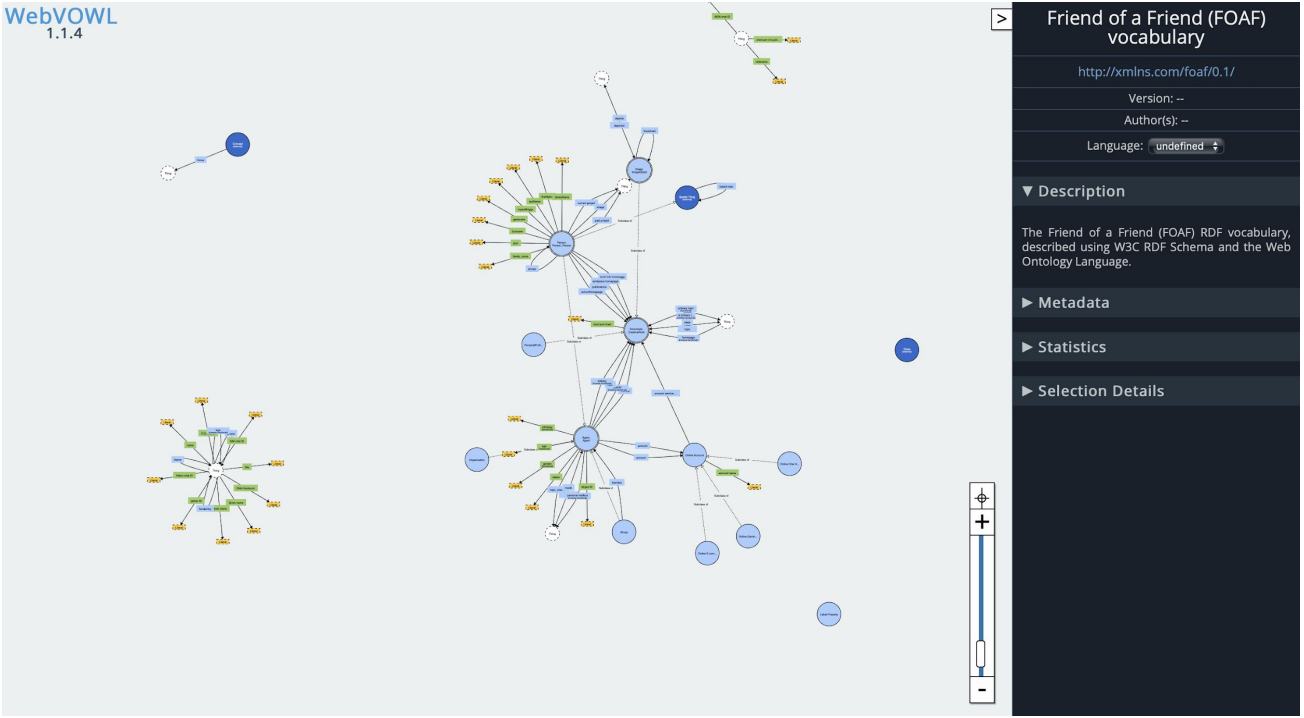
https://www.metaphacts.com

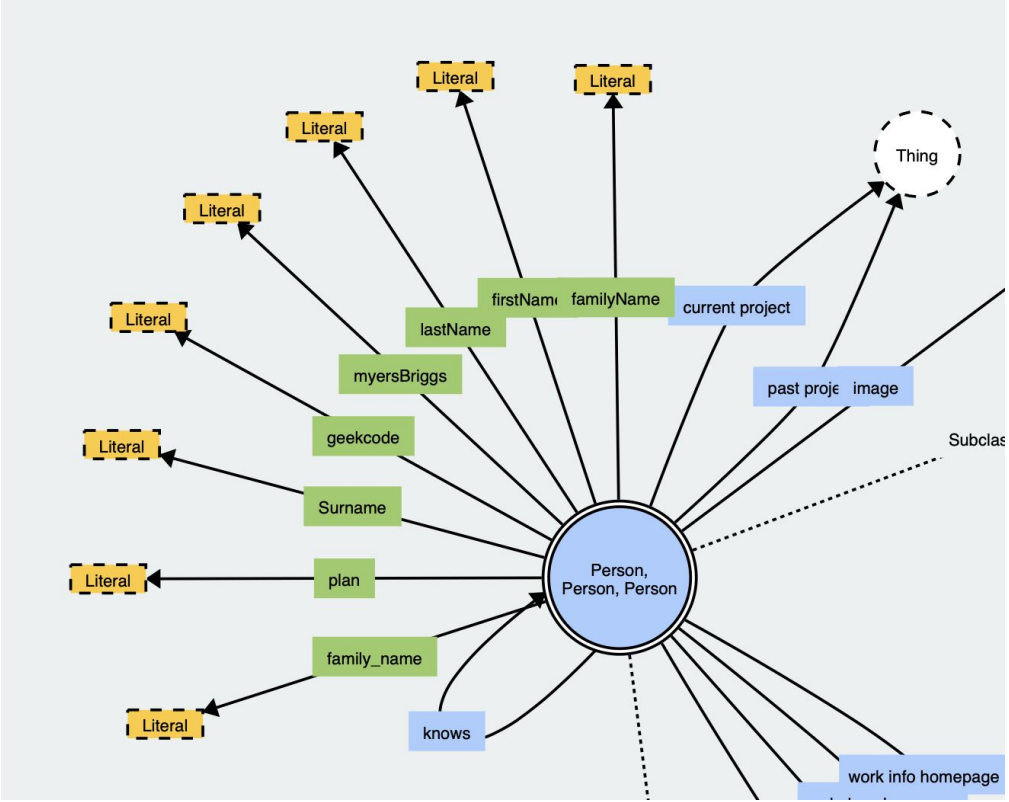
metaphacts



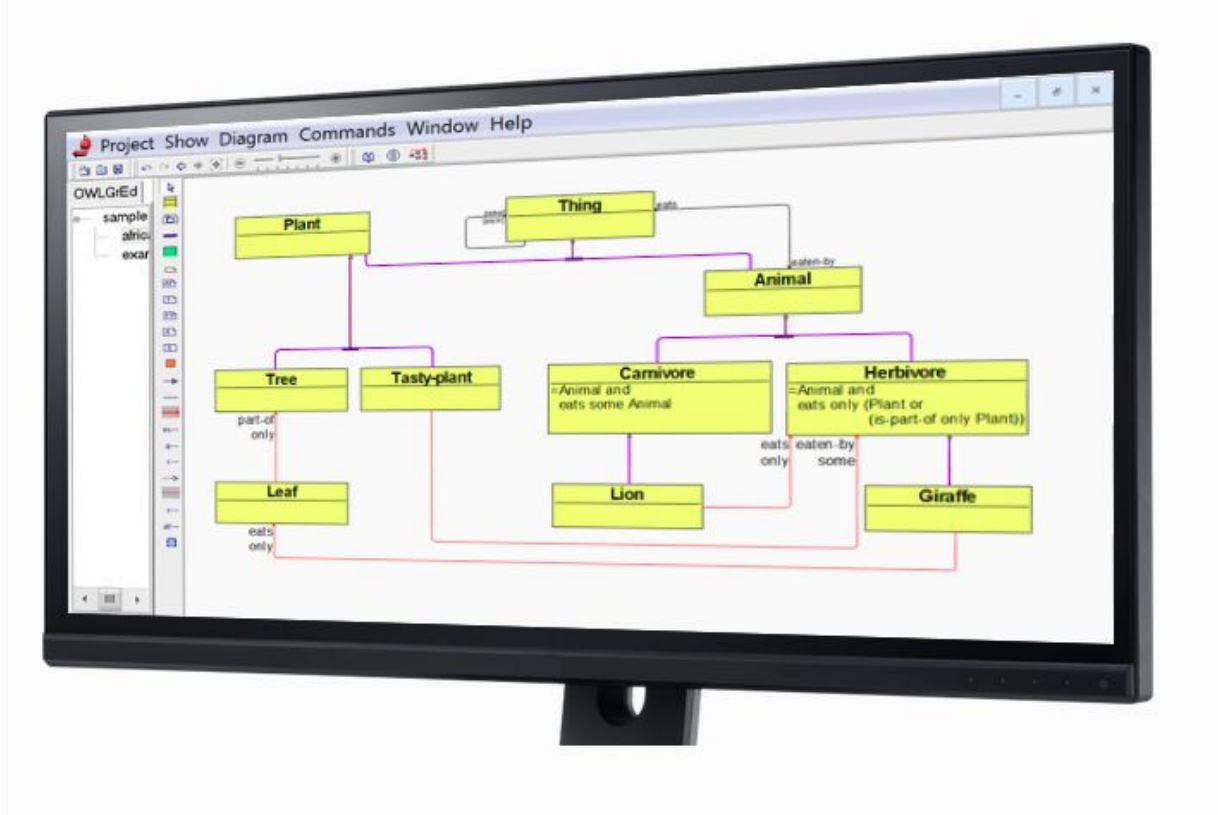

Classes
Search for...
Assigment (0)
Assignment (5)
Class (32)
Concept (1)
Course (0)
Course (0)
Course (0)
DatatypeProperty (19)
Discussion (0)
HTML (0)
KnowledgeDomain (0)
Material (0)
Instances
Has type Class
Search for...
Concept
Course
Discussion
Final Exam
HTML
Homework
KnowledgeDomain
Lab
Material
KnowledgeDomain
Candidate
range
subClassOf
OP ObjectProperty
Belongs to Domain
domain
Concept
range
OP ObjectProperty
Has concept
domain
domain
domain
domain
Person
equivalentClass
Person
Video
HTML
Discussion
Problem
subClassOf
subClassOf
subClassOf
range
range
range
Student
Staff
Teacher
OP ObjectPr...
Has HTML
OP ObjectProperty
Has Discussion
range
domain
domain
domain
domain
domain
domain
OP ...
Takes
OP Object...
Supports
OP Obje...
Creates
OP ObjectP...
Has Video
Unit
OP ObjectProperty
Has Problem
range
range
range
range
Course
OP Obje...
Has Unit
domain
Subsection
domain
range
equivalentClass
equivalentClass
OP ObjectProp...
Has Section
range
Section
domain
OP ObjectProperty
Has Subsection
Course
Course

http://vowl.visualdataweb.org/webvowl.html




Literal
Literal
Literal
Literal
Literal
Literal
Literal
Literal
Thing
firstNam
familyName
current project
lastName
myersBriggs
past proje
image
geekcode
Subclas
Surname
plan
Person,
Person, Person
family_name
knows
work info homepage

http://owlgred.lumii.lv



Как создавать онтологии?

Продолжение после обеда

Создание онтологий OWL (Protégé)

Что такое RDF Schema?

Мы можем использовать RDF для описания таких фактов как:

```
ex:AlbertEinstein    ex:discovered    ex:TheoryOfRelativity .
```

Но как мы можем уточнить данный факт?

- Как задать, что ex:discovered предикат может иметь в качестве субъекта *человека*, а в качестве объекта *теорию*?
- Как задать, что Альберт Энштейн является *исследователем* и что каждый исследователь является *человеком*?

Такую информацию называют схемой знаний или терминологическими знаниями.

RDF Schema позволяет такие знания моделировать

RDF Schema (RDFS)

- является частью семейства RDF-стандартов разработанных W3C
- используется для схемы/терминологических знаний
- сама по себе является RDF-графом
- словарь RDFS не привязан к какой-либо предметной области
- позволяет описывать семантику словарей/онтологий создаваемых пользователями

URI пространства имен: http://www.w3.org/2000/01/rdf-schema#

Префикс: rdfs

Ограничения RDFS

- нет возможности описать отрицание какого-то выражения, например, нельзя задать, что доменом/объектом предиката не является какой-либо определенный класс
- нельзя задать кардинальное число, например ограничить классу Person кол-во возможных связей (0 или 1) ex:isMarriedTo
- нельзя задать, что объектом предиката может является один из нескольких классов
- нельзя задать метаданные словаря/онтологии, например её версию и т.д.

Это только небольшой список наиболее важных ограничений RDFS. Язык OWL не имеет таких ограничений.

prefix.cc

- поиск популярных префиксов для существующих словарей/онтологий
- добавление префикса своей онтологии
- для определения префикса онтологии используется метод краудсорсинга

Онтология (философии)

… - учение о сущем; учение о бытии как таковом; раздел философии, изучающий фундаментальные принципы бытия, его наиболее общие сущности и категории, структуру и закономерности. (Википедия)

Онтология (информатика)

Gruber (1993)[1]: Онтология - это формальная спецификация общей коцептуализации предметной области.

- машино-читаемая
- создана на основе коценсусе
- описывает концепты
- относится к какой-либо предметной области

[1] http://tomgruber.org/writing/ontolingua-kaj-1993.pdf

Онтология (на практике)

- создание инстансов/экземпляров на базе классов
- иерархия концептов (таксономии, наследование): классы и термины
- бинарные отношения между инстансами: свойства и роли
- свойства связей/предикатов: range, transitive
- типы данных: конкретные предметные области
- логические средства выражения
- четкая семантика

Создание онтологий OWL (Protégé)

OWL (1)

- OWL - Web Ontology Language, произноситься легче, чем WOL
- семейство языков для создания онтологий
- стандартизован в 2004, в 2009 стандартизована OWL 2.0
- основан на логике первого порядка

OWL (2)

- состоит из набора аксиом
- аксиомы описываются с помощью RDF-триплов
- для сериализации используются те же форматы, что и для RDF. Т.е. XML, Turtle и т.д.
- Например: FOAF ontology (http://xmlns.com/foaf/spec/)
  - http://vowl.visualdataweb.org/webvowl/

OWL: Классы, Индивиды, Свойства

Класс и индивид

- Класс - ресурс, который связан с owl:Class через rdf:type предикат.
- Существующие классы:
  - owl:Thing - описывает, то что “существует”
  - owl:Nothing - описывает, то чего “не существует”

```
ex:Person rdfs:subClassOf owl:Thing .
```

- Индивид - ресурс, имеющий принадлежность к какому-либо классу

```
ex:Mike rdf:type ex:Person .
```

Объектные свойства (Object Properties)

- owl:ObjectProperty
- подкласс rdf:Property
- связывает пары индивидов

```
ex:isFriendOf    a              owl:ObjectProperty ;
                 rdfs:domain    ex:Person ;
                 rdfs:range     ex:Person .
ex:Mike          ex:isFriendOf  ex:John .
```

Свойства данных (Data Properties)

- owl:DatatypeProperty
- подкласс rdf:Property
- связывает индивиды с литералами

```
ex:hasName      rdf:type        owl:DatatypeProperty ;
                rdfs:domain     ex:Person ;
                rdfs:range      xsd:string .
ex:Mike         ex:hasName      “Mike”^^xsd:string .
```

Пример модели

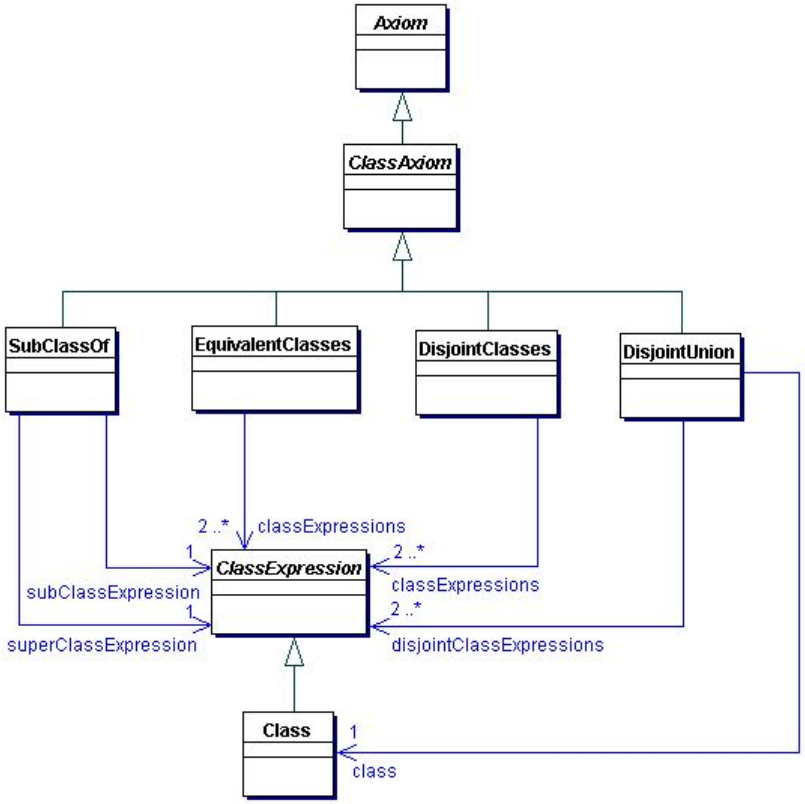
- Есть женщины и мужчины. У людей есть имена.

Пример модели

- Есть женщины и мужчины. У людей есть имена.

```
ex:Person                rdf:type                owl:Class .
ex:MalePerson            rdfs:subClassOf         ex:Person .
ex:FemalePerson  rdfs:subClassOf          ex:Person .
ex:hasName               rdf:type        owl:DatatypeProperty ;
                         rdfs:domain     ex:Person ;
                         rdfs:range xsd:string .
ex:Mike                  rdf:type        ex:MalePerson ;
                         ex:hasName      “Mike”^^xsd:string .
ex:Anna                  rdf:type        ex:FemalePerson ;
                         ex:hasName      “Anna”^^xsd:string .
```

Классы и их взаимосвязи



owl:equivalentClass

- Классы, связанные друг с другом через свойство owl:equivalentClass, являются синонимами

```
ex:Person          rdf:type        owl:Class .
ex:Individual      rdf:type        owl:Class .

ex:Person          owl:equivalentClassex:Individual .
```

Разработка онтологий

Такой же артефакт как исходный код ПО

Онтологии должны проходит такой же цикл разработки как и программный код, т.е.:

- формирование требований,
- проектирование, включая разбивку на модули,
- кодирование,
- ревью (рецензирование),
- тестирование,
- документирование,
- развертывание.

Базовые принципы

1. Не существует единственно правильного способа моделирования предметной области, всегда существуют жизнеспособные альтернативы.
   a. наилучшее решение почти всегда зависит от его применения и тех расширений, которые ожидаются.
2. Разработка онтологий - неизбежно итеративный процесс.
3. Концепты в онтологии должны быть как можно близки к объектам и связям в рассматриваемой предметной области.

Методология разработки онтологий

1. Формирование функциональных и нефункциональных требований
2. Составление предварительного глоссария терминов (классы и свойства)
3. Поиск существующих онтологий и не онтологических ресурсов (спецификации)
   a. найти то что потенциально можно переиспользовать
4. Разработка онтологии
5. Верификация онтологии

Например, http://neon-toolkit.org/wiki/Main_Page.html

Нефункциональные требования

Примеры:

- правила именования,
- правила аннотирования,
- правила публикации онтологии,
- правила версионнирования,
- и т.д.

Функциональные требования

Компетентностные вопросы - способ описания функциональных требований и верификации онтологий.

Пример:

“У каких измерительных приборов, установленных на скважинах на участке X, дата поверки приходится на текущий месяц?”

“Найти координаты участков, на которых в ходе спектрального анализа керна был обнаружен элемент X в концентрации более Y”

Переиспользование существующих онтологий

Цель: переиспользовать как есть или использовать как паттерн проектирования.

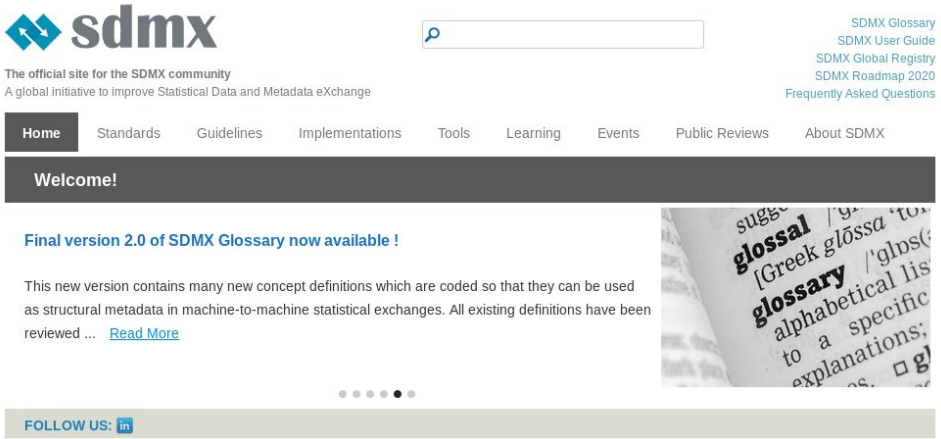
Рекомендации:

- искать стандартизованные онтологии (W3C, ISO)
  - они не обязаны покрывать все случаи применения
- пытаться понять смысловую нагрузку каждого концепта
  - по названию может подходить, но по типа связей и контексту использования - нет

Переиспользование *не* онтологий

Пример: онтология RDF Data Cube основана на ISO стандарте SDMX.

- переиспользует наименования,
- переиспользует некоторые таксономии (списки кодов, атрибуты и т.д.),
- переиспользует паттерны проектирования.



Верификация онтологии

Компетентностные вопросы -> SPARQL запросы

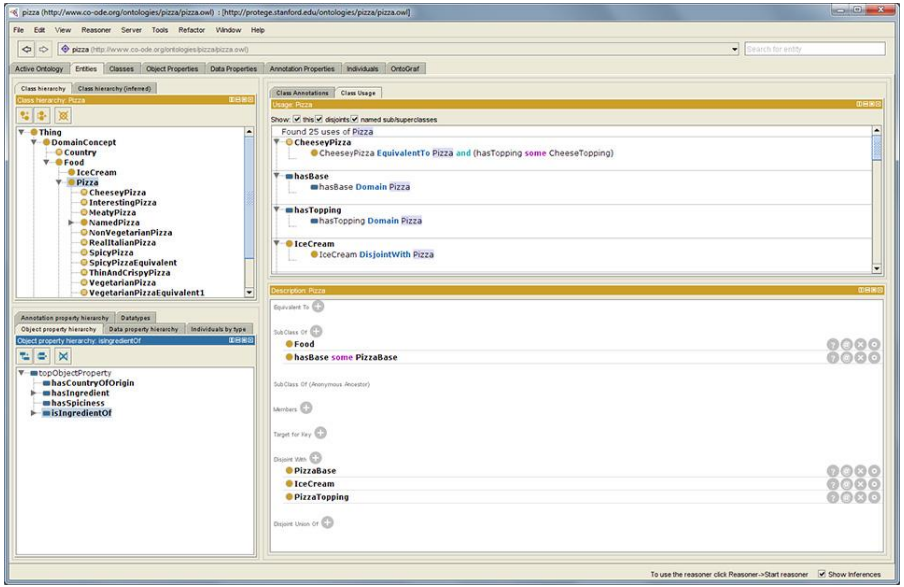
Если получилось переписать вопрос в SPARQL запрос, то онтология отвечает требованию.

Рекомендации:

- (по необходимости) проводить проверку на наличие типичных ошибок,
- писать примеры использования.

Редакторы

- Protege - классический десктопный редактор,
- WebProtege - больше подходит для совместной разработки, но имеет меньшую функциональность,
- TopBraid Composer - коммерческий редактор с большим набором функций.



- http://owl.cs.manchester.ac.uk/publications/talks-and-tutorials/protg-owl-tutorial/
- https://cgi.csc.liv.ac.uk/~frank/teaching/comp08/protege_tutorial.pdf

Документация

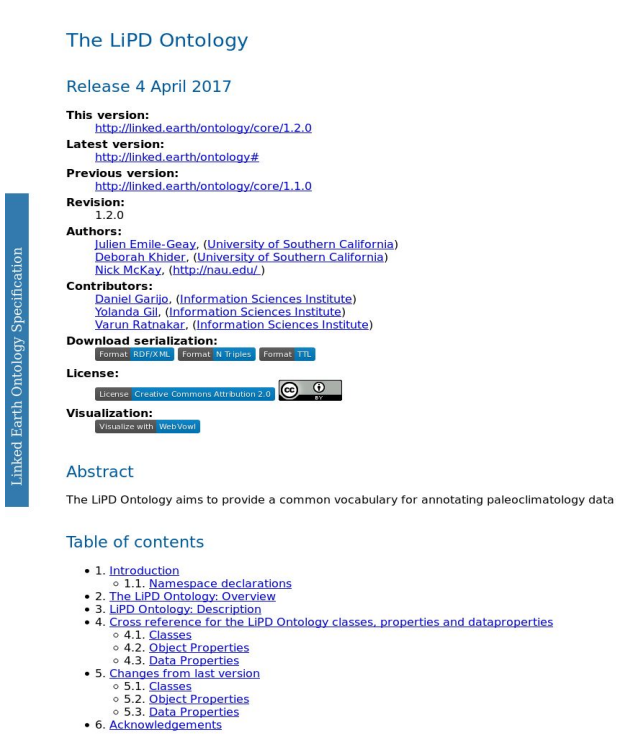
Документация может быть оформлена вручную, а также сгенерирована автоматически.

ПО:

- Widoco - генератор документации,
- LODE - ещё один генератор документации,
- OnToology - интегрированный с Github, генератор документации, диаграмм и валидатор синтаксиса.

Пример сгенерированной документации

http://linked.earth/ontology/core/1.2.0/index-en.html



Linked Earth Ontology Specification

The LiPD Ontology

Release 4 April 2017

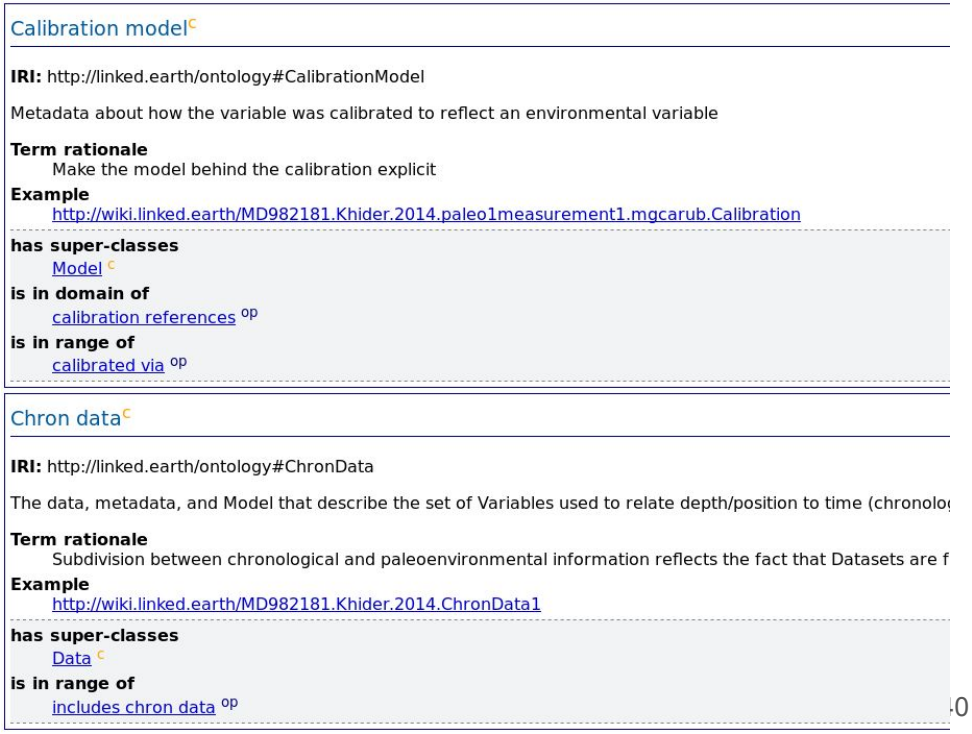
This version:
http://linked.earth/ontology/core/1.2.0
Latest version:
http://linked.earth/ontology#
Previous version:
http://linked.earth/ontology/core/1.1.0
Revision:
1.2.0
Authors:
Julien Emile-Geay, (University of Southern California)
Deborah Khider, (University of Southern California)
Nick McKay, (http://nau.edu/)
Contributors:
Daniel Garijo, (Information Sciences Institute)
Yolanda Gil, (Information Sciences Institute)
Varun Ratnakar, (Information Sciences Institute)
Download serialization:
Format RDF/XML Format N Triples Format TTL
License:
License Creative Commons Attribution 2.0
Visualization:
Visualize with WebVowl

Abstract

The LiPD Ontology aims to provide a common vocabulary for annotating paleoclimatology data

Table of contents

- 1. Introduction
  - 1.1. Namespace declarations
- 2. The LiPD Ontology: Overview
- 3. LiPD Ontology: Description
- 4. Cross reference for the LiPD Ontology classes, properties and dataproperties
  - 4.1. Classes
  - 4.2. Object Properties
  - 4.3. Data Properties
- 5. Changes from last version
  - 5.1. Classes
  - 5.2. Object Properties
  - 5.3. Data Properties
- 6. Acknowledgements



Calibration model[C]

IRI: http://linked.earth/ontology#CalibrationModel

Metadata about how the variable was calibrated to reflect an environmental variable

Term rationale
Make the model behind the calibration explicit
Example
http://wiki.linked.earth/MD982181.Khider.2014.paleo1measurement1.mgcarub.Calibration

has super-classes
Model [C]
is in domain of
calibration references [OP]
is in range of
calibrated via [OP]

Chron data[C]

IRI: http://linked.earth/ontology#ChronData

The data, metadata, and Model that describe the set of Variables used to relate depth/position to time (chronolo

Term rationale
Subdivision between chronological and paleoenvironmental information reflects the fact that Datasets are f
Example
http://wiki.linked.earth/MD982181.Khider.2014.ChronData1

has super-classes
Data [C]
is in range of
includes chron data [OP]

Именование

- классы
  - LoanContract
- свойства
  - hasBorrower

- “осмысленные” vs “бессмысленные” URI
  - ex:DataSystem
  - ex:123jhf12

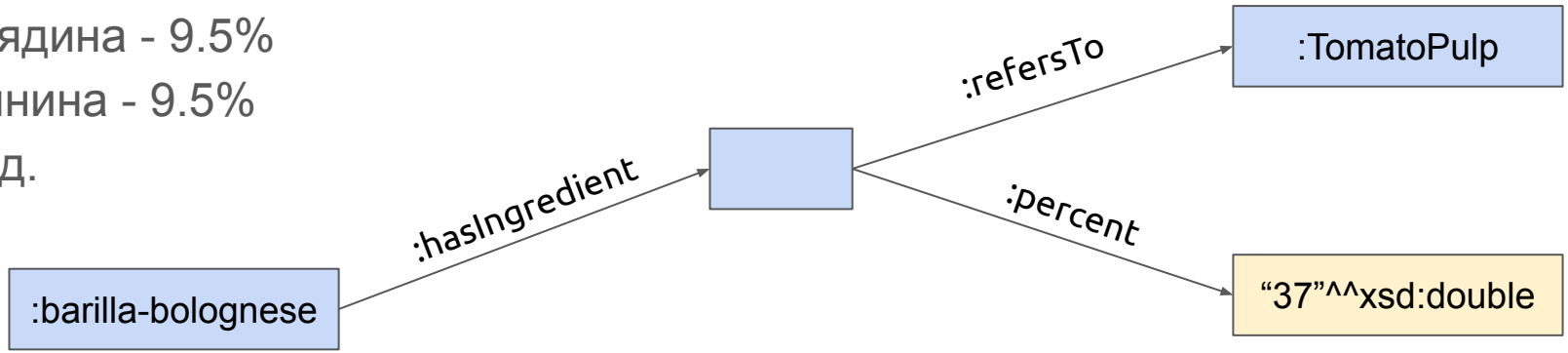
N-арные связи

Имя: Соус Barilla Bolognese

Состав:

- мякоть томата - 37%
- лук - 12,8%
- говядина - 9.5%
- свинина - 9.5%
- и т.д.

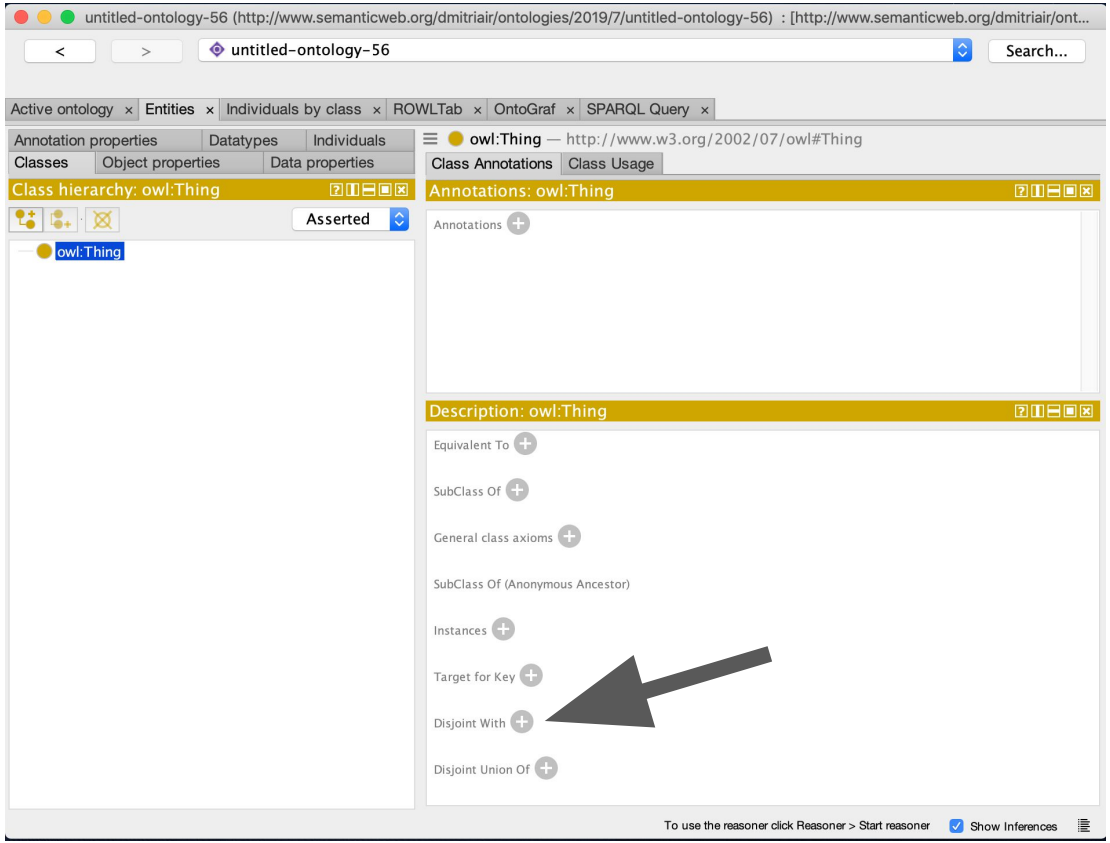
```
@prefix : <http://example.com/> .

:barilla-bolognese a :Sauce ;
        :hasIngredient [
                :refersTo :TomatoPulp ;
                :percent “37”^^xsd:double
        ] .
```



rdfs:domain и rdfs:range

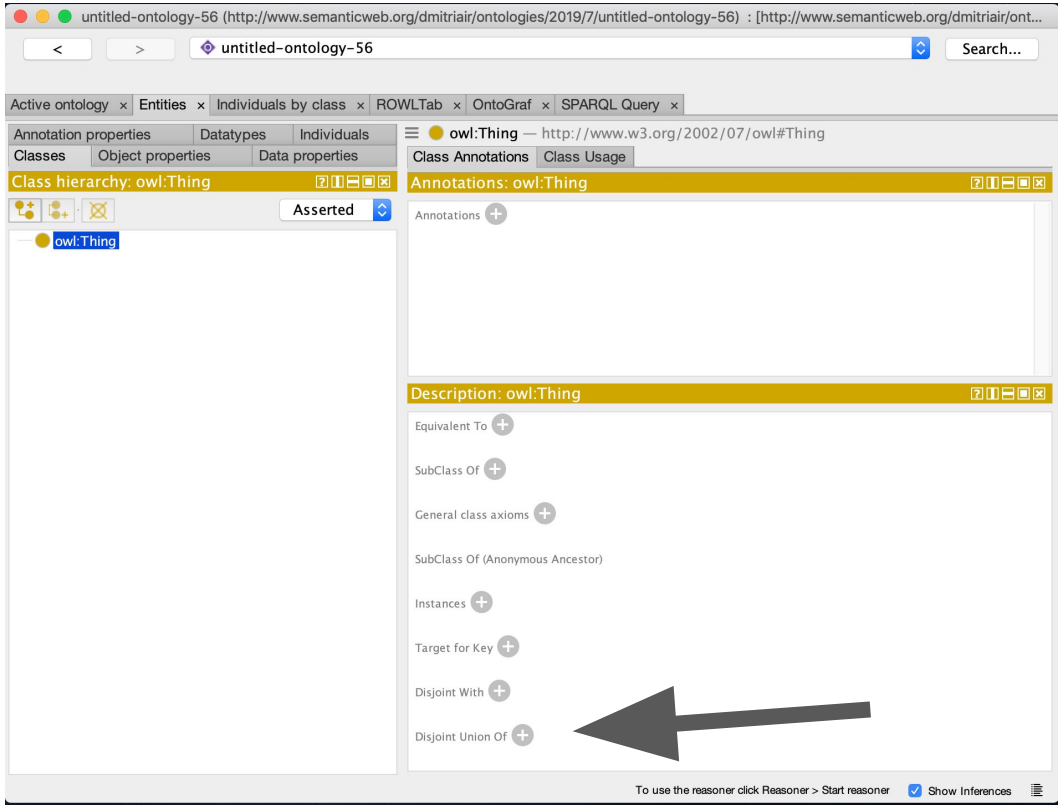
- не являются ограничениями,
- указывают на то к каким классам относятся инстансы.

owl:disjointWith



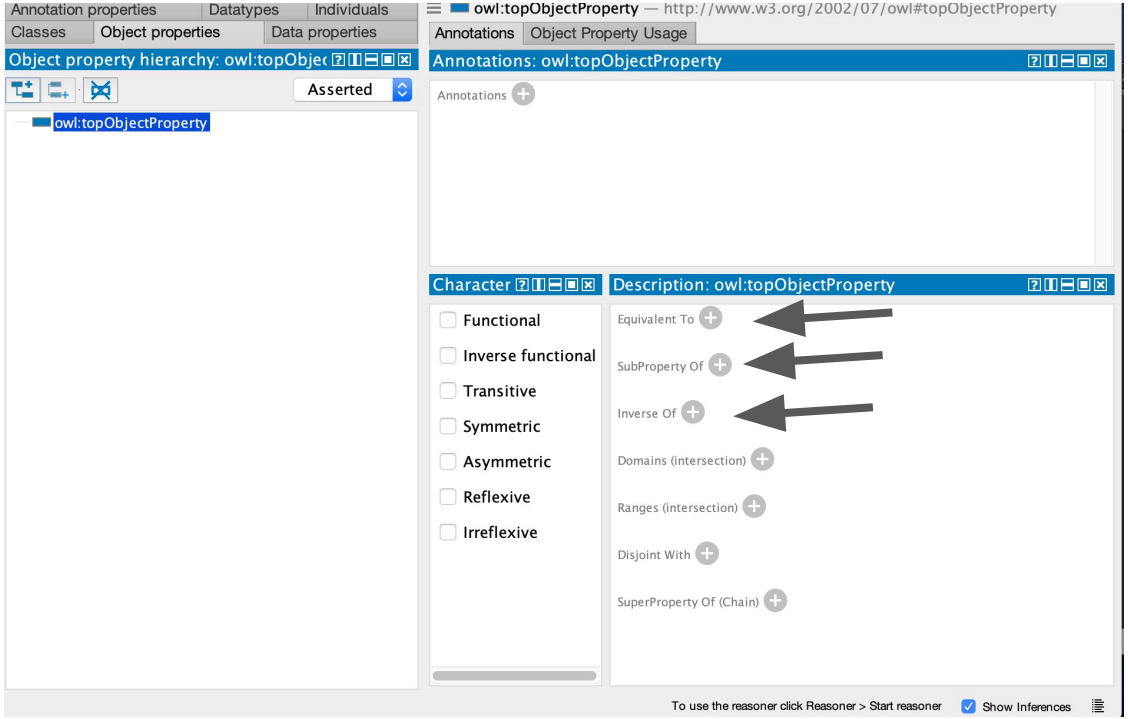
- Если классы связаны друг с другом через свойство **owl:disjointWith**, то это означает, что не должно существовать индивидов, которые относятся одновременно к обоим классам

owl:disjointUnionOf



- Если классы связаны через свойство **owl:disjointUnionOf**, то это означает, что индивиды должны относиться к одному из данных классов. Например: ex:Child owl:disjointUnionOf (ex:Boy ex:Girl)

Свойства и их взаимосвязи

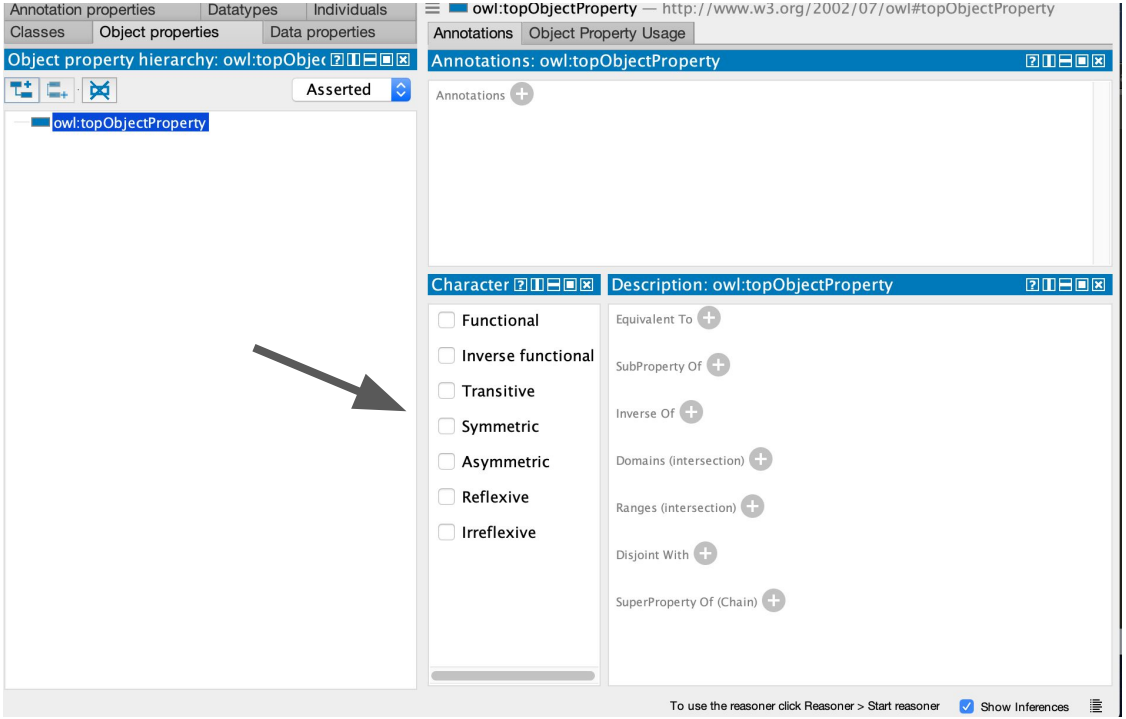


- По аналогии с классами, для свойств существуют такие связи как **rdfs:subPropertyOf** и **owl:equivalentProperty**. Однако свойства так же могут быть инверсными (**owl:inverseOf**).

Например:

```
ex:examinedBy owl:inverseOf ex:examinerOf .
```

Свойства и их взаимосвязи



- Транзитивные свойства (owl:TransitiveProperty): a->b и b->c = a->c
- Симметричные свойства (owl:SymmetricProperty): a->b = b->a
- Функциональные свойства (owl:FunctionalProperty): a->b и a->c = b == c
- Инверсионно-функциональные свойства (owl:InverseFunctionalProperty): a->b и c->b = a == c

Ограничения свойств

- **Экзистенциальные ограничения (Some)**. Эти ограничения описывают классы индивидов, которые участвуют в по крайней мере в одном отношении по указанному свойству для отдельных индивидов, которые являются членами указанного класса.

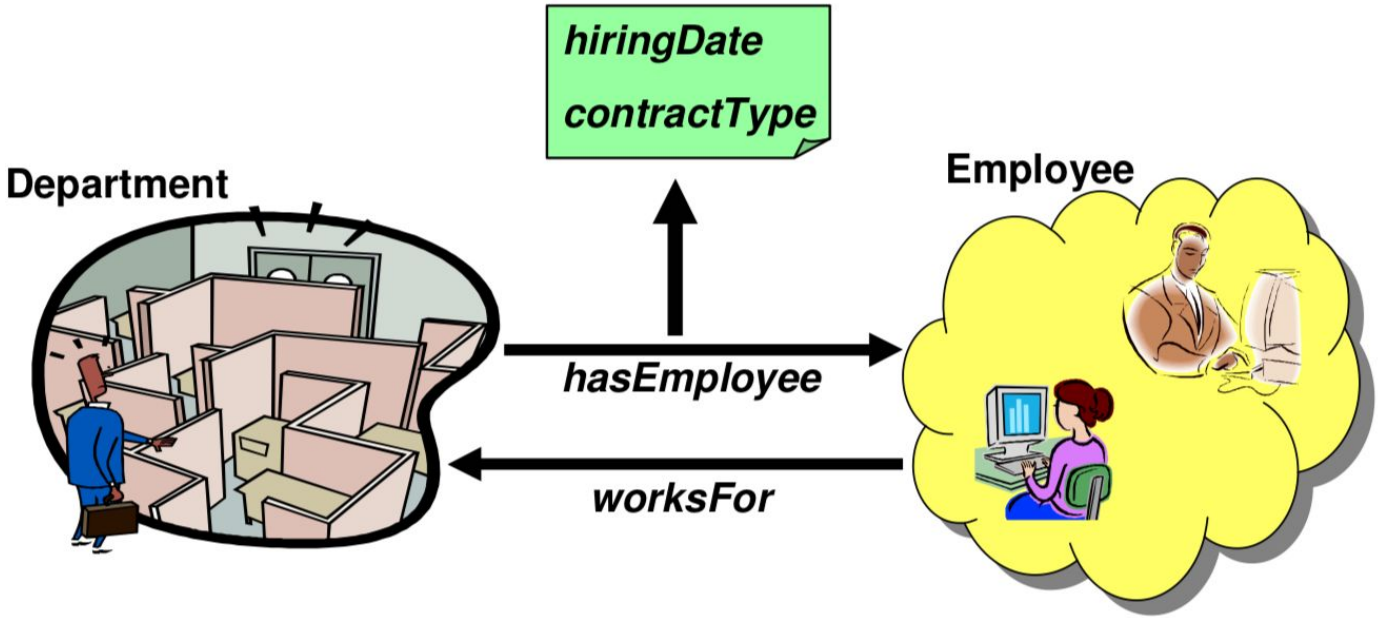
- **Универсальные ограничения.** Эти ограничения описывают классы индивидов, имеющих «только - **only**» данное отношение к отдельным индивидам, которые являются членами конкретного класса.

Индивиды и их взаимосвязи

- owl:sameAs - индивиды являются синонимами
- owl:differentFrom - индивиды не являются синонимами

```
ex:Mike     rdf:type     ex:Doctor .
ex:Mike     owl:sameAs   ex:DrMike .
```

Овеществление(Reification)



Отдел имеет **Сотрудник**ов

В **отдел**е у каждого **сотрудник**а есть **дата найма** и **тип контракта**

Если **отдел** имеет **сотрудник**а, то **сотрудник работает в** этом **отдел**е

Что такое овеществление?

- Представление абстрактного объекта, как если бы это была конкретная, существующая вещь;
- Используется для представления фактов, над которыми необходимо выполнять действия;
- Утверждение, прикрепленное как свойство к другому утверждению

Reasoning

Что умеют Ризонеры?

- Проверка классификации и иерархии классов
- Проверка согласованности
- Построение дополнительных связей на основе свойств

Дополнительные материалы:

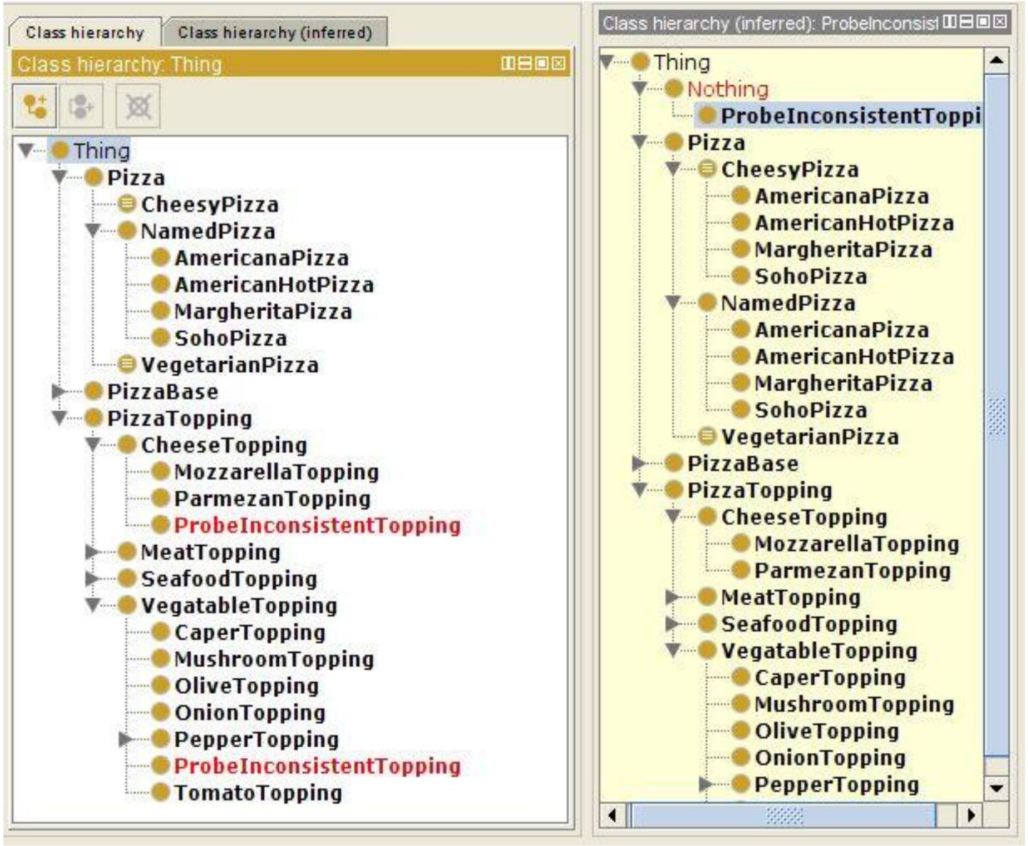
https://protege.stanford.edu/conference/2007/slides/owlTutorial-reasoning_dameron.pdf

http://owl.man.ac.uk/2003/why/latest/

https://perso.liris.cnrs.fr/lionel.medini/enseignement/M2_IA/KnowDyn/CM_Reasoning.pdf

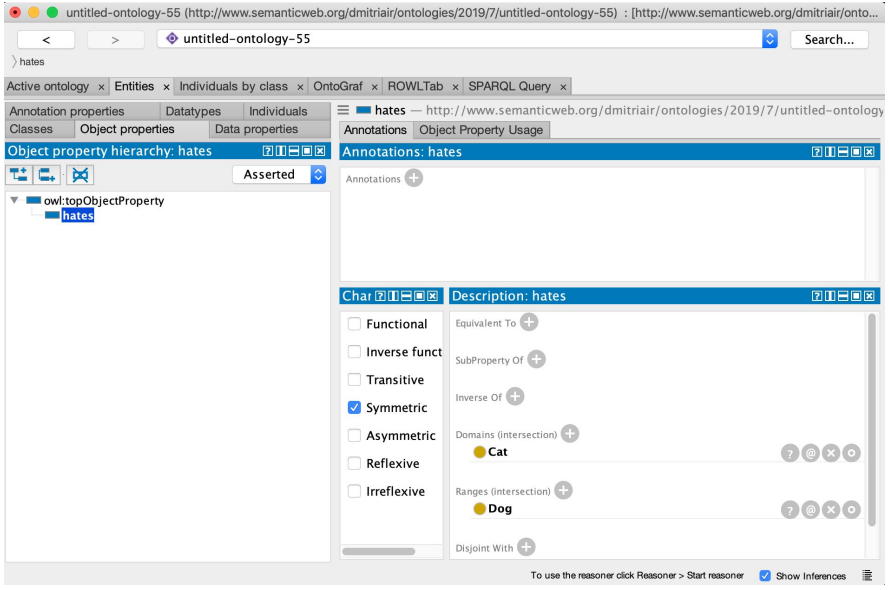
Онтология пиццы:
https://github.com/owlcs/pizza-ontology/blob/master/pizza.owl

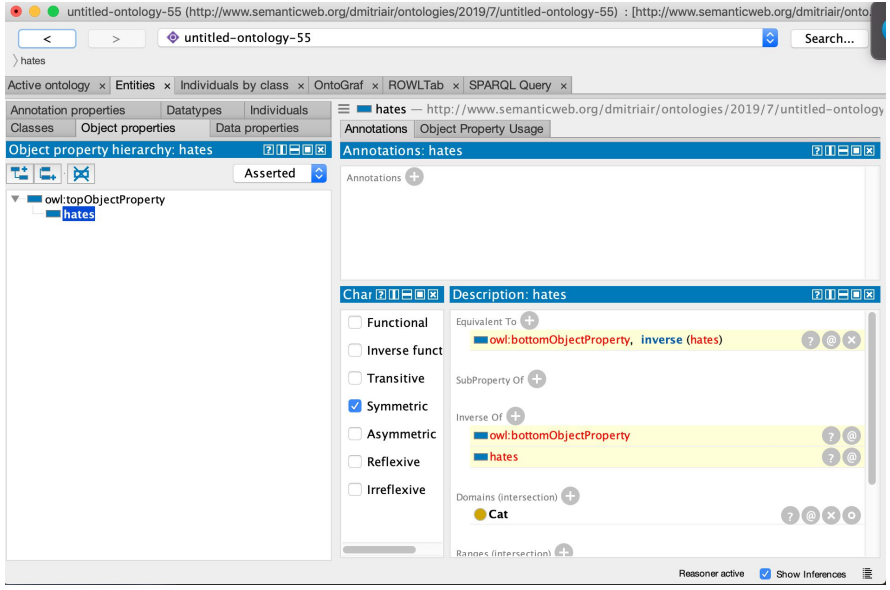
Проверка классификации согласованности



Если вычисляемая иерархия изменяет существующую, то имя изменяющего положение класса появится в синем цвете на изображении иерархии. Если класс признан несовместимым, то его иконка будут выделена красным цветом.

Проверка классификации согласованности



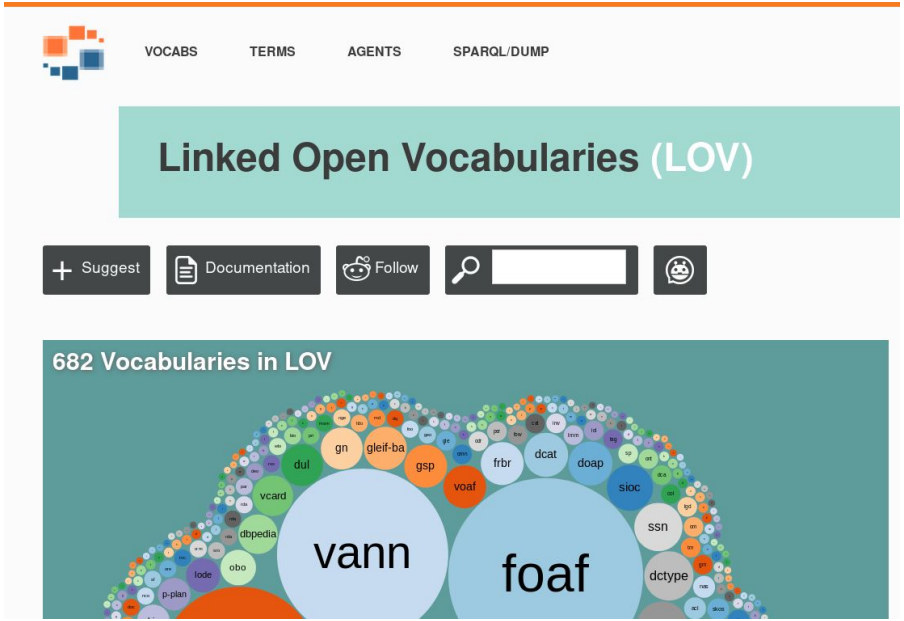


Слева отношения Кошка - ненавидит - Собак с установленным свойством симметричности. Справа дополнительные связи, построенные ризонером.

Онтологии, ставшие стандартами

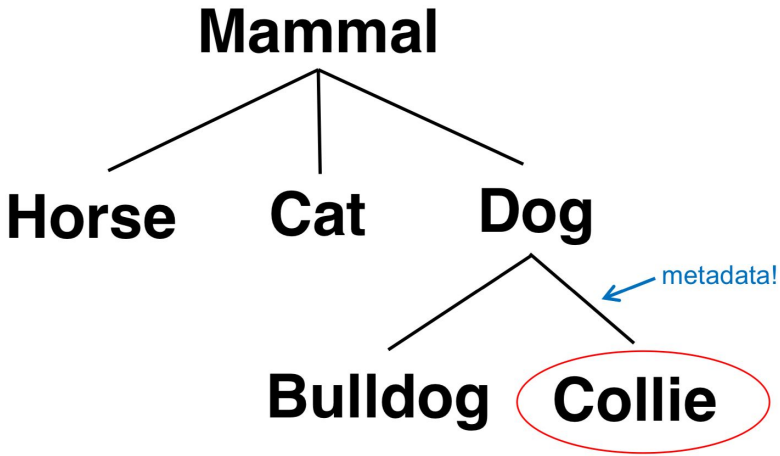
Поиск существующих онтологий

- Linked Open Vocabularies
  - https://lov.linkeddata.es
  - поддерживается сообществом
  - и Политехом Мадрида



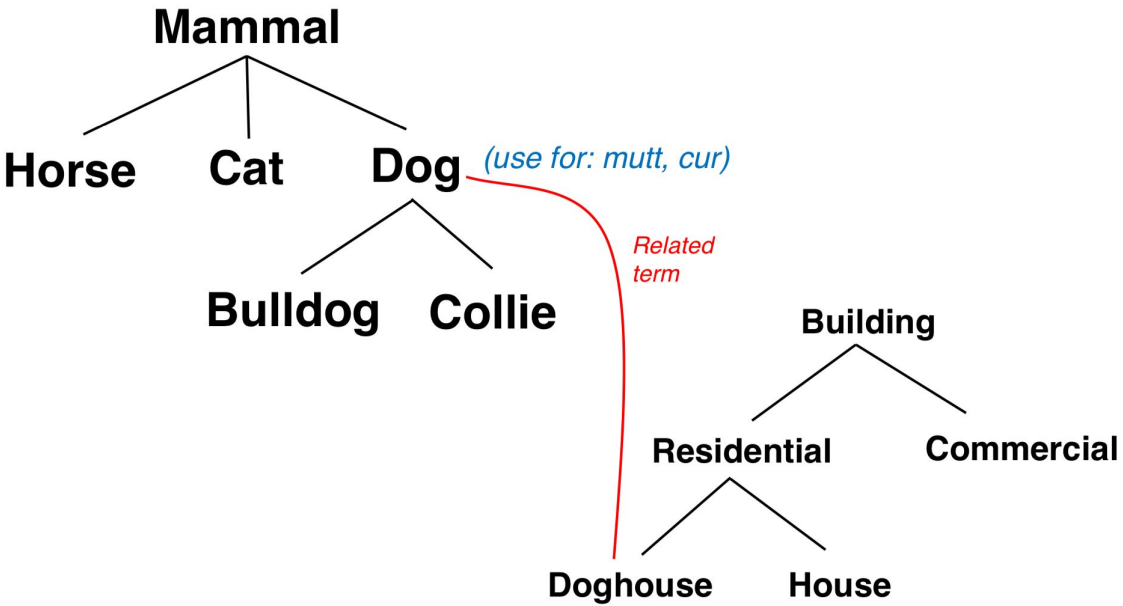
Simple Knowledge Organization System

- **SKOS:** OWL-онтология W3C для создания тезаурусов, таксономий и контролируемых словарей.



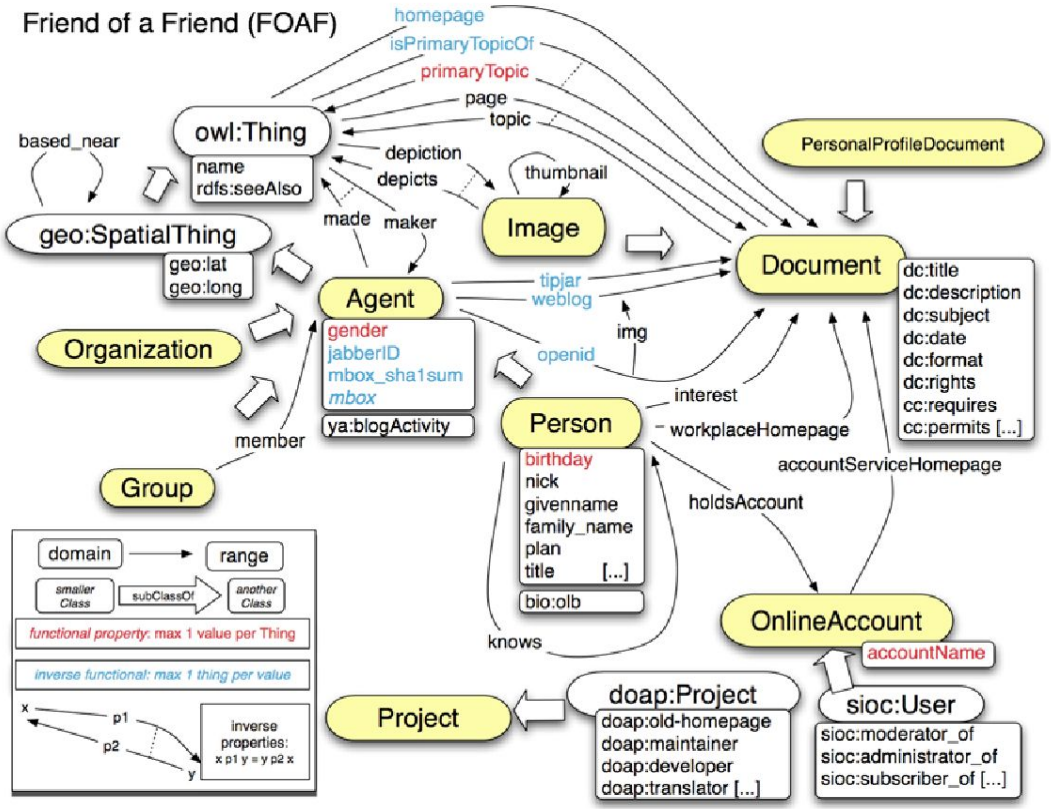
https://www.w3.org/2004/02/skos/

Тезаурус в SKOS



Тезаурус позволяет выстраивать семантические отношения между лексическими единицами не находящимися в иерархическом отношении (синонимы, антонимы, паронимы, гипонимы, гиперонимы и так далее)

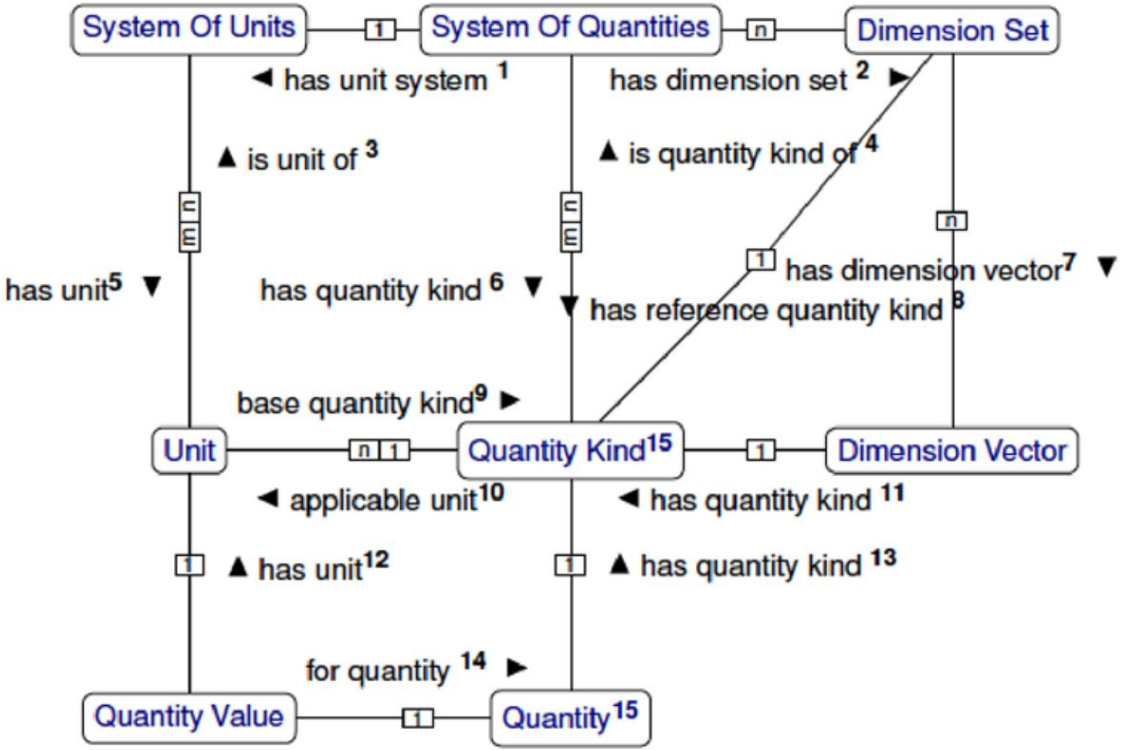
Friend Of A Friend



- **FOAF.** Объединяет три вида сетей: социальные сети, дружба и ассоциации. Предоставляет классы и свойства для описания людей, и их взаимоотношений.

http://xmlns.com/foaf/spec/

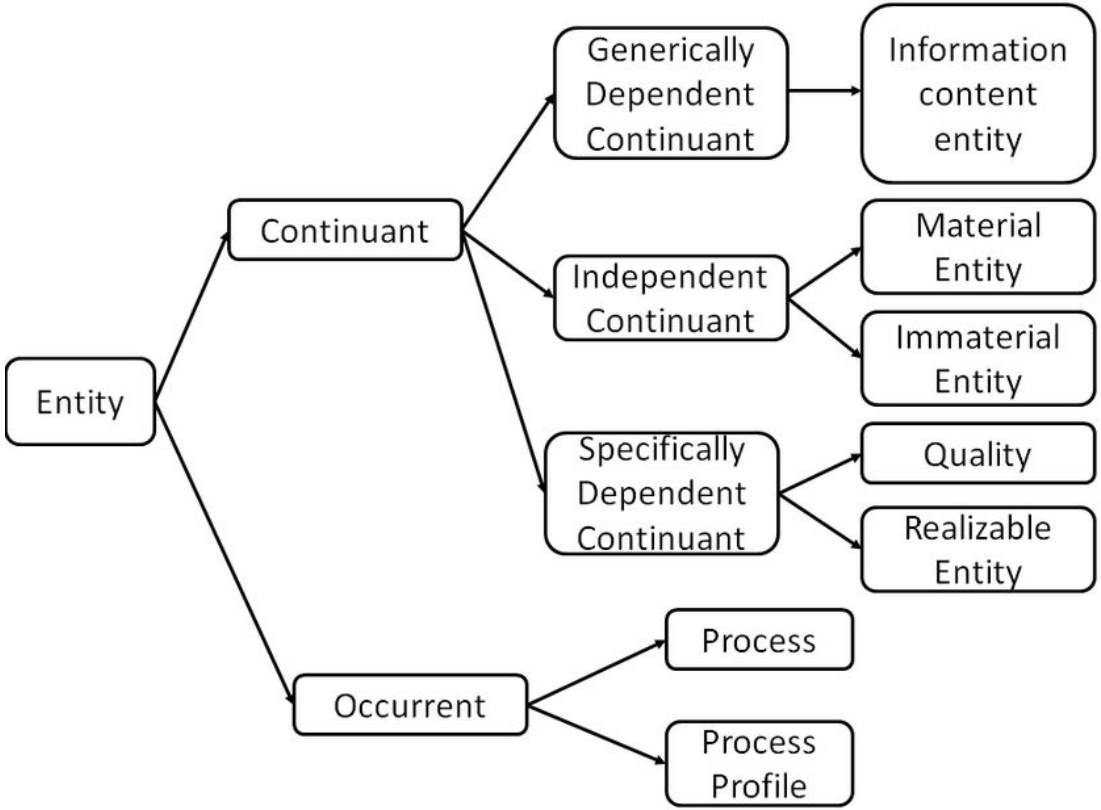
Quantity, Unit, Dimension and Type



- **QUDT**: предоставляет семантические спецификации для единиц измерения, размерностей и типов данных.

http://www.qudt.org

The Basic Formal Ontology



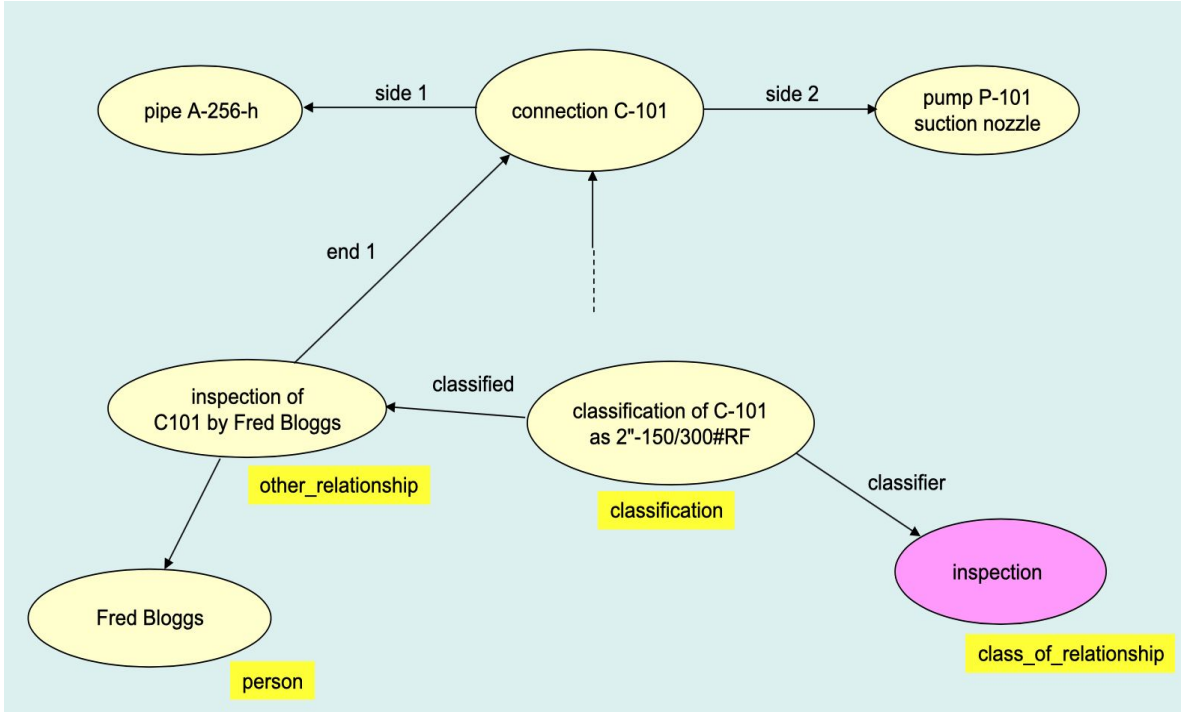
- **BFO**: это небольшая онтология верхнего уровня, предназначенная для использования при поддержке поиска, анализа и интеграции информации в научной и других областях.

https://basic-formal-ontology.org

The Unified Foundational Ontology

http://dev.nemo.inf.ufes.br/seon/UFO.html

ISO 15926-2



- **ISO 15926-2** Модель данных. Специфицирует обобщенную концептуальную модель данных, поддерживающую представление всех аспектов жизненного цикла установки непрерывного производства.

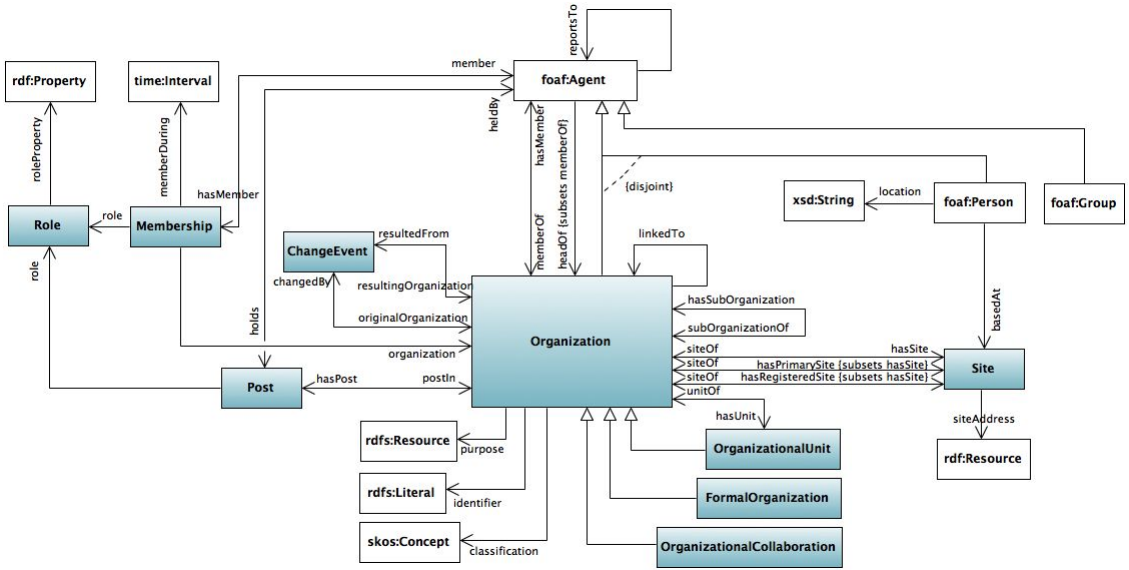
http://15926.org

W3C Recommendation

The Organization Ontology

W3C Recommendation 16 January 2014

This version:
http://www.w3.org/TR/2014/REC-vocab-org-20140116/
Latest published version:
http://www.w3.org/TR/vocab-org/
Implementation report:
http://www.w3.org/2011/gld/wiki/ORG_Implementations
Previous version:
http://www.w3.org/TR/2013/PR-vocab-org-20131217/
Editor:
Dave Reynolds, Epimorphics Ltd.



Semantic Sensor Network Ontology

- 2-я версия онтологии
- концепты
  - устройства
  - измерения
  - актуации
  - и т.д.

Semantic Sensor Network Ontology

W3C Recommendation 19 October 2017 (Link errors corrected 08 December 2017)

This version:
https://www.w3.org/TR/2017/REC-vocab-ssn-20171019/

Latest published version:
https://www.w3.org/TR/vocab-ssn/

Latest editor's draft:
https://w3c.github.io/sdw/ssn/

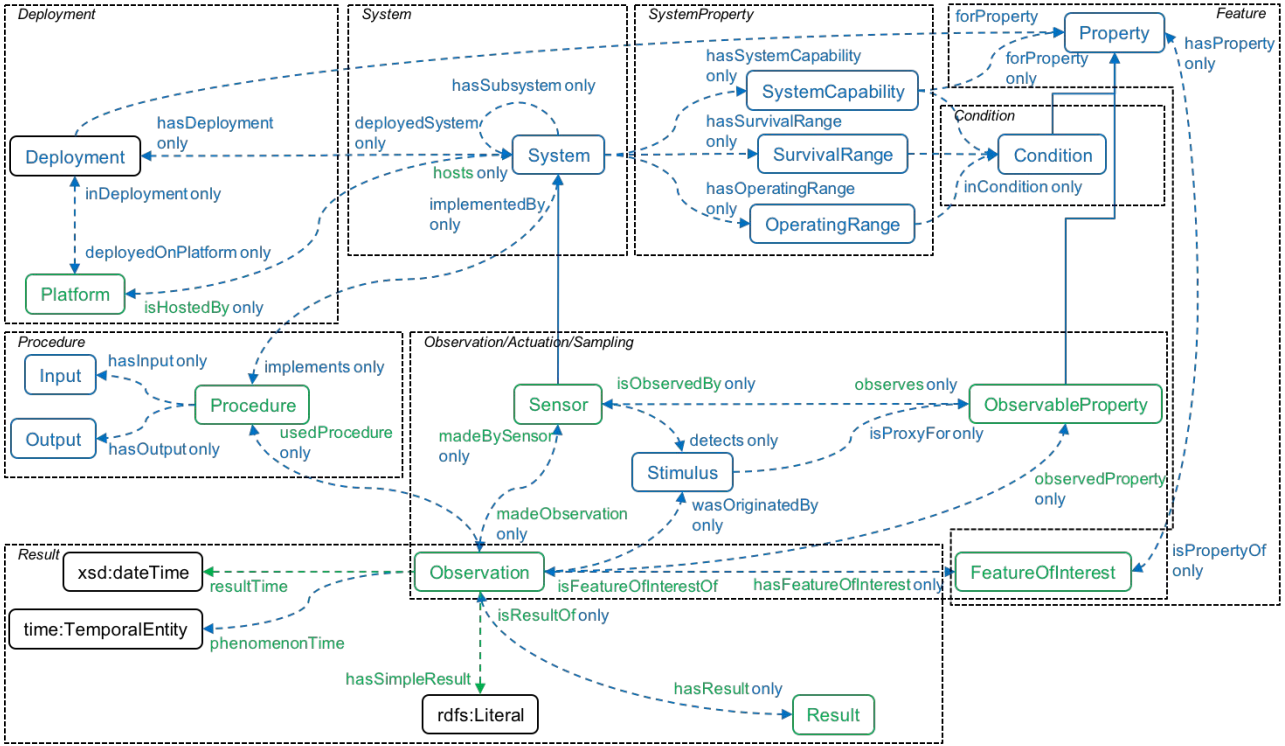
Implementation report:
https://w3c.github.io/sdw/ssn-usage/

Previous version:
https://www.w3.org/TR/2017/PR-vocab-ssn-20170907/

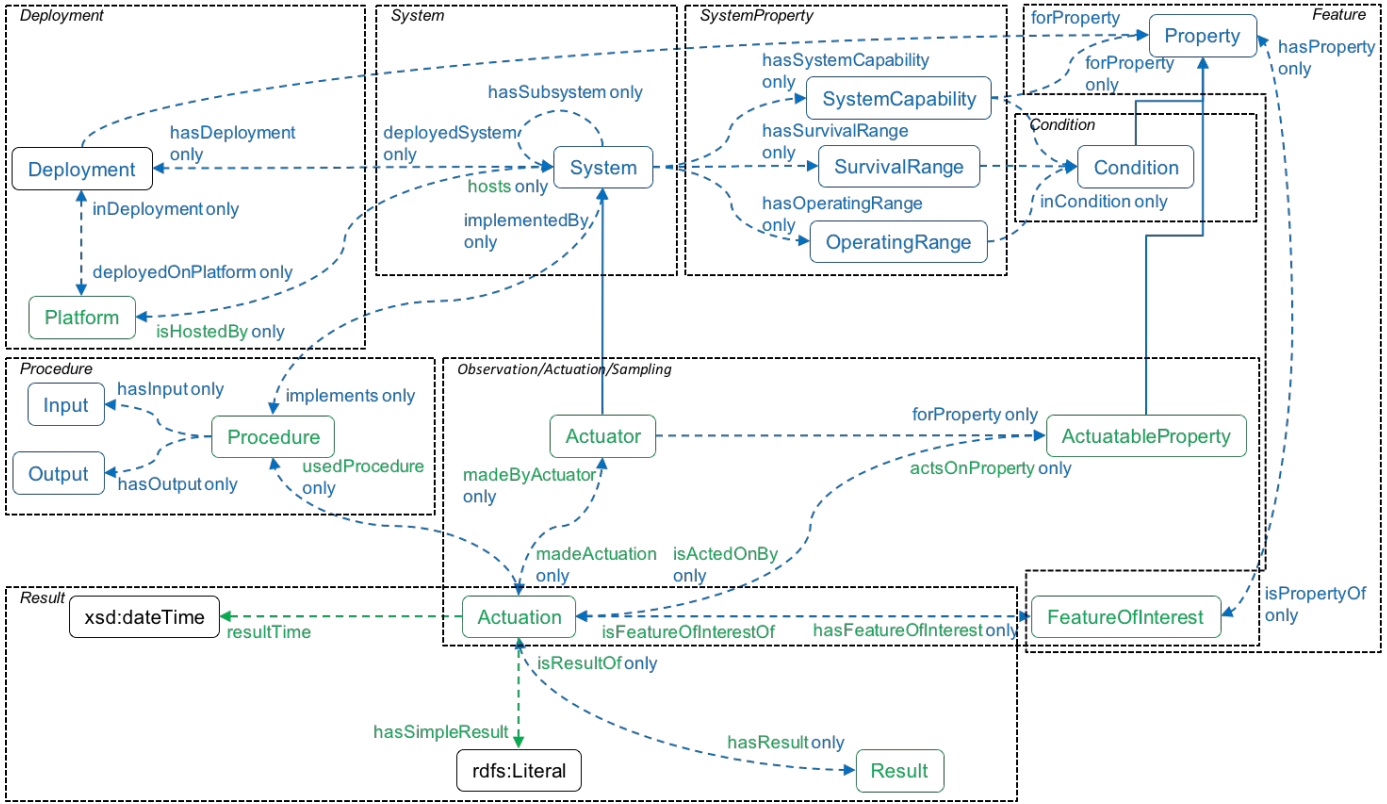
Editors:
Armin Haller, Australian National University
Krzysztof Janowicz, University of California, Santa Barbara
Simon Cox, CSIRO
Danh Le Phuoc, Technical University of Berlin
Kerry Taylor, Australian National University
Maxime Lefrançois, École Nationale Supérieure des Mines de Saint-Étienne

Contributors (ordered alphabetically):
Rob Atkinson, Metalinkage
Raúl García-Castro, Universidad Politécnica de Madrid
Joshua Lieberman, Tumbling Walls
Claus Stadler, Universität Leipzig

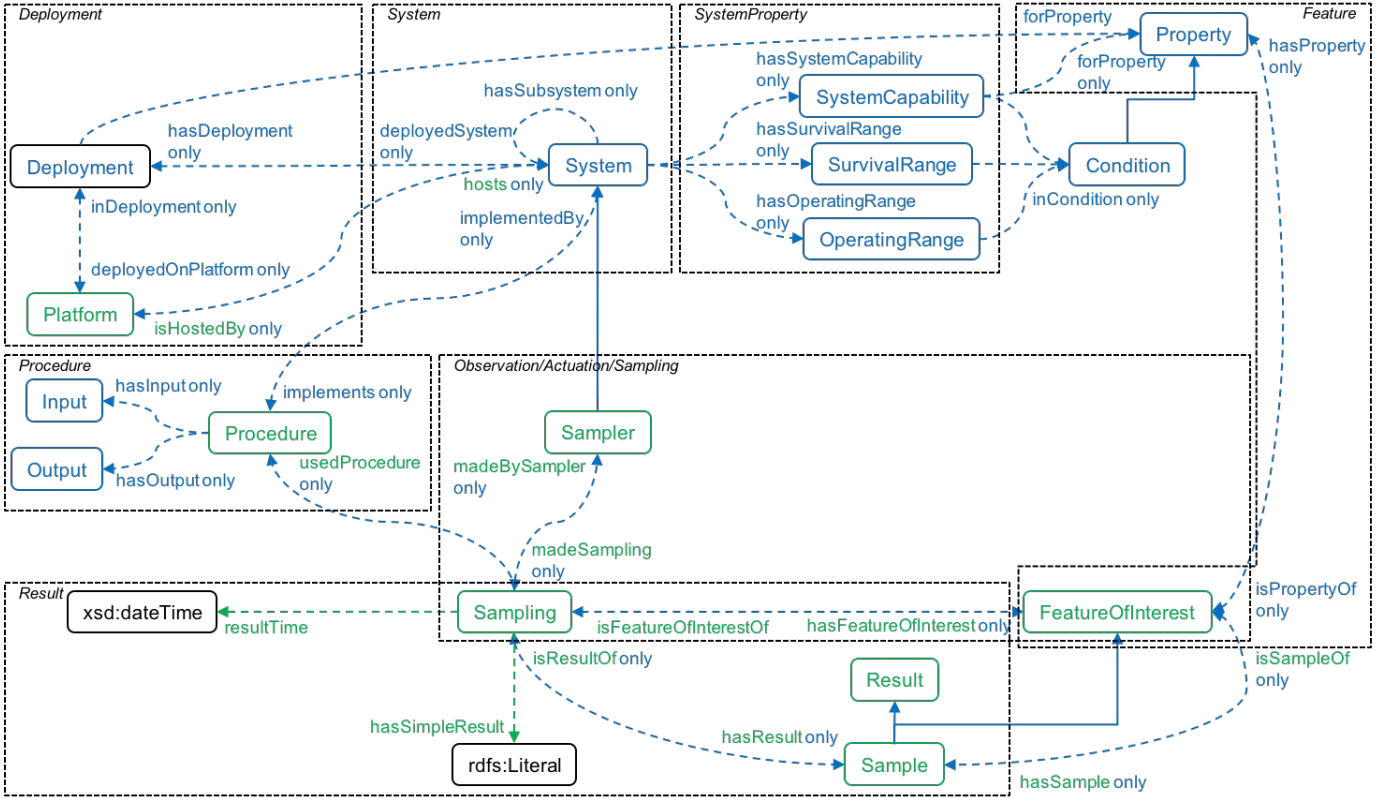
Semantic Sensor Network Ontology (1)



Semantic Sensor Network Ontology (2)

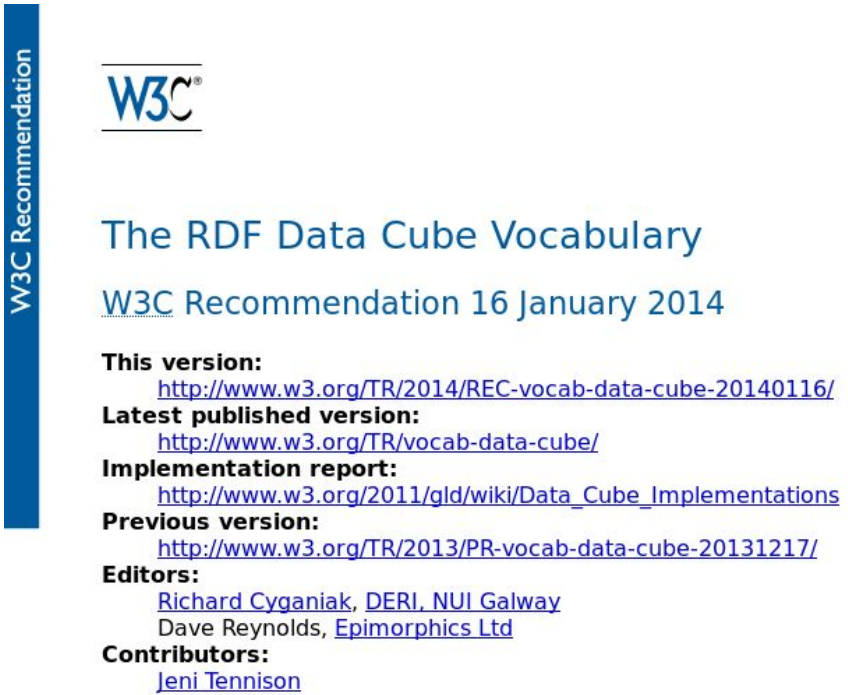


Semantic Sensor Network Ontology (3)

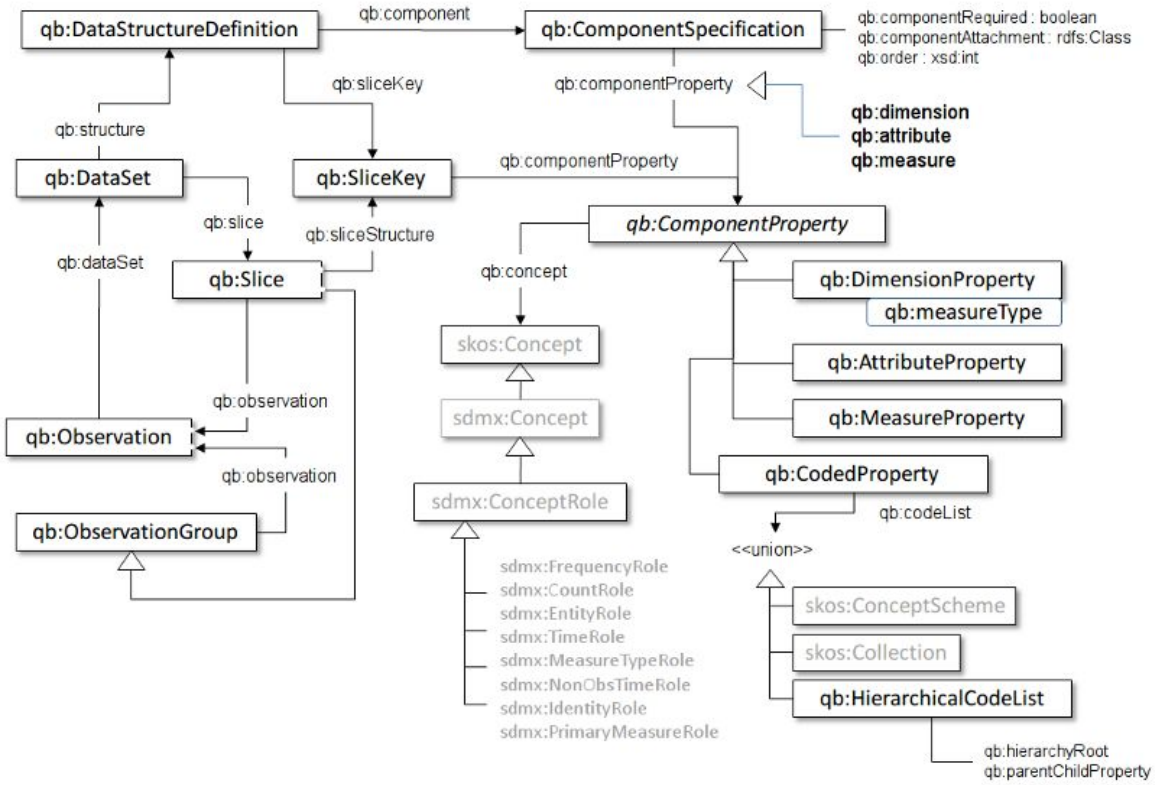


RDF Data Cube (1)

- онтология для описания статистических данных
- Примеры:
  - финансовые отчеты
  - результаты измерений
  - различные агрегированные данные



RDF Data Cube (2)



Введение в SPARQL

Что такое SPARQL?

… - SPARQL Protocol and RDF Query Language.

Вдобавок к языку W3C-стандарт определяет:

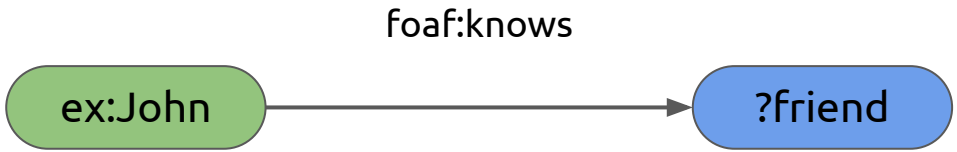
- протокол для отправки SPARQL-запросов и получения ответов
- формат ответов на SPARQL-запросы на основе XML

Triplestore и SPARQL-точка доступа

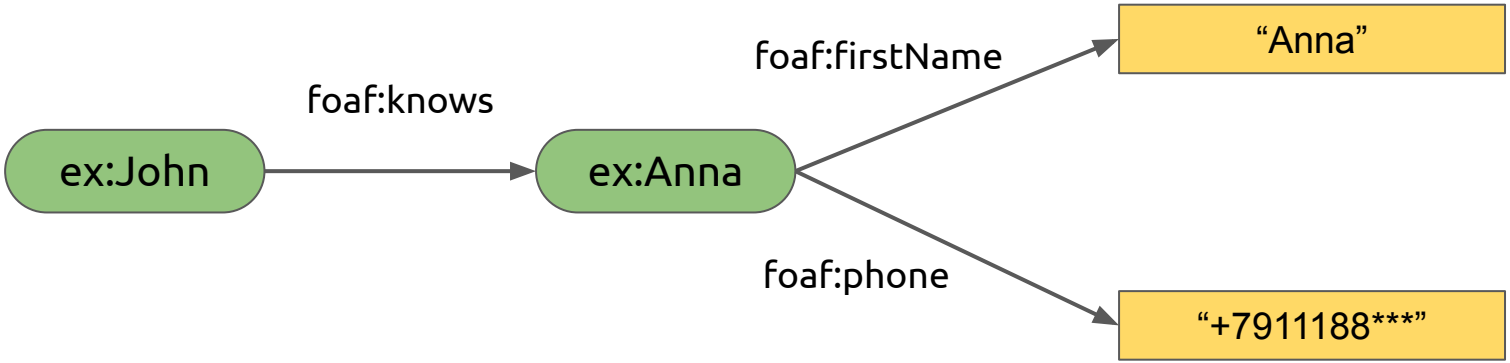
- Triplestore, Триплстор - база данных для хранения RDF-данных
- SPARQL-endpoint - программный интерфейс выполнения запросов

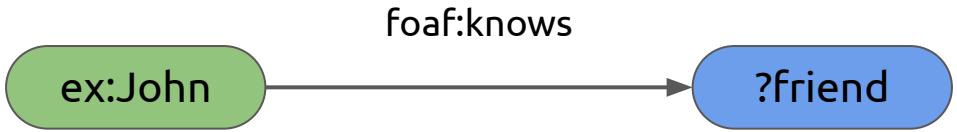
Например: dbpedia.org/sparql

Простой запрос

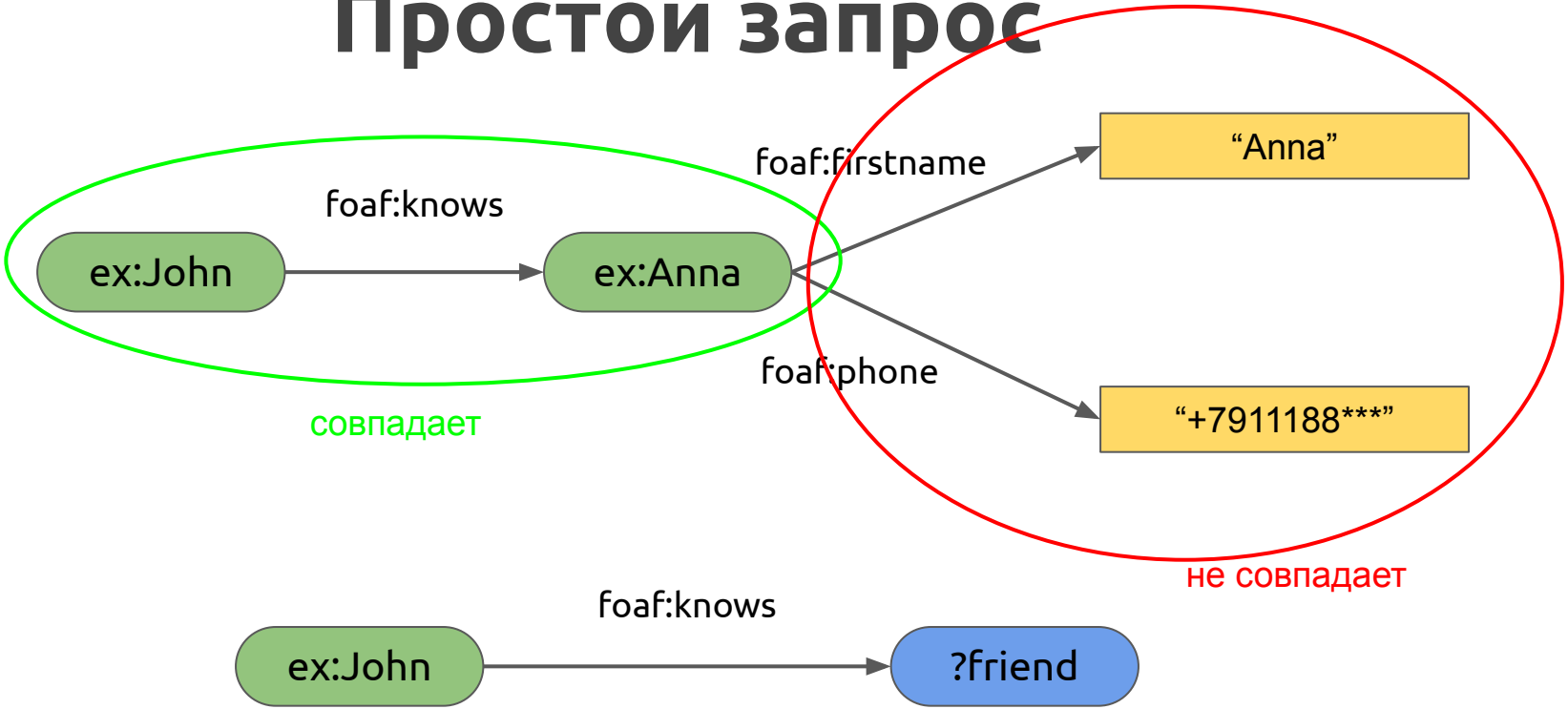


Простой запрос





Простой запрос

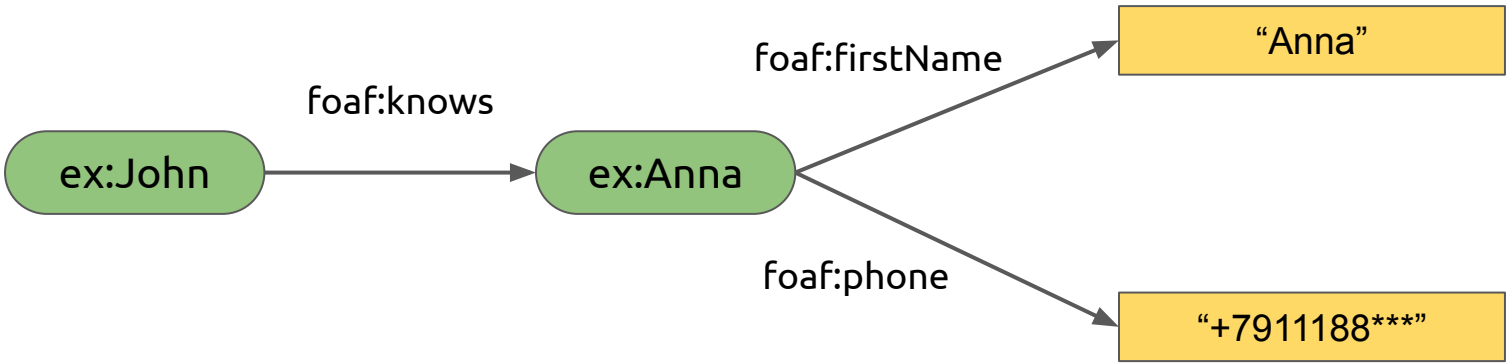


Более сложный запрос

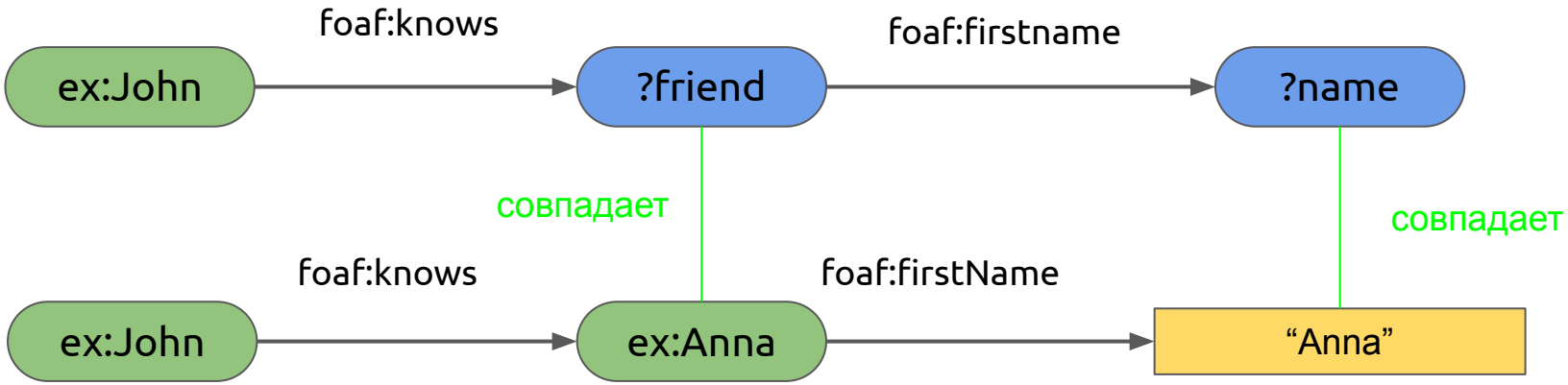
Более сложный запрос

```
SELECT ?friend, ?name WHERE {
   ex:John foaf:knows ?friend .
   ?friend foaf:firstname ?name .
}
```

Более сложный запрос



Более сложный запрос




```
?friend | ?friendname
--------+------------
ex:Anna | “Anna”
```

Структура запросов

```
# префиксы
PREFIX foaf: <http://xmlns.com/foaf/0.1/>
# тип запроса  # проекции   # датасет
SELECT         ?x ?y        FROM
# шаблоны графа
WHERE {
   ?x a ?y
}
# модификаторы запроса
ORDER BY ?y
```

Типы запросов

SELECT - возвращает таблицу результатов

ASK - возвращает true или false, если шаблон совпадает

CONSTRUCT - создает триплы на основе шаблонов графов

DESCRIBE - возвращает описание ресурса в виде триплов

Модификаторы запроса

Изменяют результаты запросов.

- LIMIT and OFFSET делит результаты запроса на страницы, например: SELECT * WHERE {...} LIMIT 10
- ORDER BY сортирует результаты запроса, например: SELECT * WHERE {...} ORDER BY ASC(...) LIMIT 10

SPARQL на примерах

Далее будут рассмотрены несколько примеров SPARQL-запросов иллюстрирующих работу различных конструкций.

Для этих примеров используется http://dbpedia.org/sparql

SPARQL на примерах

```
PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
PREFIX yago: <http://dbpedia.org/class/yago/>
PREFIX dbo: <http://dbpedia.org/ontology/>
PREFIX dbr: <http://dbpedia.org/resource/>

SELECT * WHERE {
    ?city rdf:type yago:City108524735 ;
    dbo:country dbr:Germany .
}
```

city
http://dbpedia.org/resource/Freiburg,_Lower_Saxony
http://dbpedia.org/resource/Horka,_Saxony
http://dbpedia.org/resource/Sosa,_Germany
http://dbpedia.org/resource/Eisenhüttenstadt
http://dbpedia.org/resource/Göttingen
http://dbpedia.org/resource/Uhry_(Königslutter)
http://dbpedia.org/resource/Suurhusen
http://dbpedia.org/resource/Bramstedt
http://dbpedia.org/resource/Freital
http://dbpedia.org/resource/Grevesmühlen
http://dbpedia.org/resource/Haselbachtal
http://dbpedia.org/resource/Kappeln

е в SPARQL

SPARQL на примерах

```
PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
PREFIX yago: <http://dbpedia.org/class/yago/>
PREFIX dbo: <http://dbpedia.org/ontology/>
PREFIX dbr: <http://dbpedia.org/resource/>
SELECT * WHERE {
        ?city rdf:type yago:City108524735 ;
                dbo:country dbr:Germany ;
                dbo:populationTotal ?population .
}
LIMIT 100
```

city	population
http://dbpedia.org/resource/Freiburg,_Lower_Saxony	"1864"^^<http://www.w3.org/2001/XMLSchema#nonNegativeInteger>
http://dbpedia.org/resource/Horka,_Saxony	"1984"^^<http://www.w3.org/2001/XMLSchema#nonNegativeInteger>
http://dbpedia.org/resource/Sosa,_Germany	"2069"^^<http://www.w3.org/2001/XMLSchema#nonNegativeInteger>
http://dbpedia.org/resource/Göttingen	"116052"^^<http://www.w3.org/2001/XMLSchema#nonNegativeInteger>
http://dbpedia.org/resource/Uhry_(Königslutter)	"300"^^<http://www.w3.org/2001/XMLSchema#nonNegativeInteger>
http://dbpedia.org/resource/Suurhusen	"1200"^^<http://www.w3.org/2001/XMLSchema#nonNegativeInteger>
http://dbpedia.org/resource/Bramstedt	"1892"^^<http://www.w3.org/2001/XMLSchema#nonNegativeInteger>
http://dbpedia.org/resource/Freital	"39267"^^<http://www.w3.org/2001/XMLSchema#nonNegativeInteger>
http://dbpedia.org/resource/Grevesmühlen	"10945"^^<http://www.w3.org/2001/XMLSchema#nonNegativeInteger>
http://dbpedia.org/resource/Haselbachtal	"4582"^^<http://www.w3.org/2001/XMLSchema#nonNegativeInteger>
http://dbpedia.org/resource/Kappeln	"11393"^^<http://www.w3.org/2001/XMLSchema#nonNegativeInteger>
http://dbpedia.org/resource/Mirow	"4144"^^<http://www.w3.org/2001/XMLSchema#nonNegativeInteger>
http://dbpedia.org/resource/Stößen	"1036"^^<http://www.w3.org/2001/XMLSchema#nonNegativeInteger>
http://dbpedia.org/resource/Achim	"30059"^^<http://www.w3.org/2001/XMLSchema#nonNegativeInteger>
http://dbpedia.org/resource/Alzenau	"18971"^^<http://www.w3.org/2001/XMLSchema#nonNegativeInteger>
http://dbpedia.org/resource/Arzberg,_Bavaria	"5893"^^<http://www.w3.org/2001/XMLSchema#nonNegativeInteger>
http://dbpedia.org/resource/Augustusburg	"5152"^^<http://www.w3.org/2001/XMLSchema#nonNegativeInteger>

SPARQL на примерах

```
SELECT * WHERE {
      ?city rdf:type yago:City108524735 ;
            dbo:country dbr:Germany ;
            dbo:populationTotal ?population .
      OPTIONAL { ?city foaf:homepage ?homepage }
}
LIMIT 100
```

city	population	homepage
http://dbpedia.org/resource/Freiburg,_Lower_Saxony	"1864"^^<http://www.w3.org/2001/XMLSchema#nonNegativeInteger>	http://www.nordkehdingen.de/
http://dbpedia.org/resource/Horka,_Saxony	"1984"^^<http://www.w3.org/2001/XMLSchema#nonNegativeInteger>	http://www.horka.de/
http://dbpedia.org/resource/Sosa,_Germany	"2069"^^<http://www.w3.org/2001/XMLSchema#nonNegativeInteger>	http://www.sosa.de/
http://dbpedia.org/resource/Göttingen	"116052"^^<http://www.w3.org/2001/XMLSchema#nonNegativeInteger>	http://www.goettingen.de/
http://dbpedia.org/resource/Uhry_(Königslutter)	"300"^^<http://www.w3.org/2001/XMLSchema#nonNegativeInteger>	http://www.koenigslutter.de
http://dbpedia.org/resource/Suurhusen	"1200"^^<http://www.w3.org/2001/XMLSchema#nonNegativeInteger>	
http://dbpedia.org/resource/Bramstedt	"1892"^^<http://www.w3.org/2001/XMLSchema#nonNegativeInteger>	http://www.hagen-cux.de/
http://dbpedia.org/resource/Freital	"39267"^^<http://www.w3.org/2001/XMLSchema#nonNegativeInteger>	http://www.freital.de/
http://dbpedia.org/resource/Grevesmühlen	"10945"^^<http://www.w3.org/2001/XMLSchema#nonNegativeInteger>	http://www.grevesmuehlen.de/
http://dbpedia.org/resource/Haselbachtal	"4582"^^<http://www.w3.org/2001/XMLSchema#nonNegativeInteger>	http://www.haselbachtal.de/
http://dbpedia.org/resource/Kappeln	"11393"^^<http://www.w3.org/2001/XMLSchema#nonNegativeInteger>	http://www.stadt-kappeln.de/startseite.phtml/
http://dbpedia.org/resource/Mirow	"4144"^^<http://www.w3.org/2001/XMLSchema#nonNegativeInteger>	http://www.amt-mecklenburgische-kleinseenplatte.de/
http://dbpedia.org/resource/Stößen	"1036"^^<http://www.w3.org/2001/XMLSchema#nonNegativeInteger>	http://www.stoessen-online.de
http://dbpedia.org/resource/Achim	"30059"^^<http://www.w3.org/2001/XMLSchema#nonNegativeInteger>	http://www.achim.de/
http://dbpedia.org/resource/Alzenau	"18971"^^<http://www.w3.org/2001/XMLSchema#nonNegativeInteger>	http://www.alzenau.de/
http://dbpedia.org/resource/Arzberg,_Bavaria	"5893"^^<http://www.w3.org/2001/XMLSchema#nonNegativeInteger>	http://www.arzberg.de/
http://dbpedia.org/resource/Augustusburg	"5152"^^<http://www.w3.org/2001/XMLSchema#nonNegativeInteger>	http://www.augustusburg.de/
http://dbpedia.org/resource/Bad_Düben	"8702"^^<http://www.w3.org/2001/XMLSchema#nonNegativeInteger>	http://www.bad-dueben.de
http://dbpedia.org/resource/Bad_Eilsen	"2254"^^<http://www.w3.org/2001/XMLSchema#nonNegativeInteger>	http://www.bad-eilsen.de/
http://dbpedia.org/resource/Bad_Essen	"15807"^^<http://www.w3.org/2001/XMLSchema#nonNegativeInteger>	http://www.badessen.de/
http://dbpedia.org/resource/Bad_Grund	"8957"^^<http://www.w3.org/2001/XMLSchema#nonNegativeInteger>	http://www.bad-grund.de/
http://dbpedia.org/resource/Bad_Iburg	"11652"^^<http://www.w3.org/2001/XMLSchema#nonNegativeInteger>	http://www.badiburg.de/
http://dbpedia.org/resource/Bad_Laer	"9242"^^<http://www.w3.org/2001/XMLSchema#nonNegativeInteger>	http://www.bad-laer.de/

SPARQL на примерах

```
SELECT * WHERE {
      ?city rdf:type yago:City108524735 ;
            dbo:country dbr:Germany ;
            dbo:populationTotal ?population .
      OPTIONAL { ?city foaf:homepage ?homepage }
}
ORDER BY DESC(?population)
LIMIT 100
```

city	population	homepage
http://dbpedia.org/resource/Berlin	"3610156"^^<http://www.w3.org/2001/XMLSchema#nonNegativeInteger>	http://www.berlin.de/en/
http://dbpedia.org/resource/Hamburg	"1774242"^^<http://www.w3.org/2001/XMLSchema#nonNegativeInteger>	http://www.hamburg.de/
http://dbpedia.org/resource/Munich	"1520408"^^<http://www.w3.org/2001/XMLSchema#nonNegativeInteger>	http://www.muenchen.de/
http://dbpedia.org/resource/Cologne	"1057327"^^<http://www.w3.org/2001/XMLSchema#nonNegativeInteger>	http://www.stadt-koeln.de
http://dbpedia.org/resource/Frankfurt	"731095"^^<http://www.w3.org/2001/XMLSchema#nonNegativeInteger>	http://www.frankfurt.de/
http://dbpedia.org/resource/Bochum	"670320"^^<http://www.w3.org/2001/XMLSchema#nonNegativeInteger>	http://www.bochum.de/
http://dbpedia.org/resource/Bremen_(state)	"661000"^^<http://www.w3.org/2001/XMLSchema#nonNegativeInteger>	http://www.bremen.de/
http://dbpedia.org/resource/Düsseldorf	"653678"^^<http://www.w3.org/2001/XMLSchema#nonNegativeInteger>	http://www.duesseldorf.de/
http://dbpedia.org/resource/Stuttgart	"623738"^^<http://www.w3.org/2001/XMLSchema#nonNegativeInteger>	http://www.stuttgart.de/
http://dbpedia.org/resource/Essen	"589075"^^<http://www.w3.org/2001/XMLSchema#nonNegativeInteger>	http://www.essen.de/
http://dbpedia.org/resource/Dortmund	"575944"^^<http://www.w3.org/2001/XMLSchema#nonNegativeInteger>	http://www.dortmund.de/en/
http://dbpedia.org/resource/Dortmund	"575944"^^<http://www.w3.org/2001/XMLSchema#nonNegativeInteger>	http://www.dortmund.de/
http://dbpedia.org/resource/Bremen	"548475"^^<http://www.w3.org/2001/XMLSchema#nonNegativeInteger>	http://www.bremen.de/
http://dbpedia.org/resource/Dresden	"536107"^^<http://www.w3.org/2001/XMLSchema#nonNegativeInteger>	http://www.dresden.de/
http://dbpedia.org/resource/Hanover	"518386"^^<http://www.w3.org/2001/XMLSchema#nonNegativeInteger>	http://www.hannover.de/
http://dbpedia.org/resource/Nuremberg	"498876"^^<http://www.w3.org/2001/XMLSchema#nonNegativeInteger>	http://www.nuernberg.de/internet/portal_e/index.html
http://dbpedia.org/resource/Duisburg	"488005"^^<http://www.w3.org/2001/XMLSchema#nonNegativeInteger>	http://www.duisburg.de/
http://dbpedia.org/resource/Wuppertal	"349950"^^<http://www.w3.org/2001/XMLSchema#nonNegativeInteger>	http://www.wuppertal.de/
http://dbpedia.org/resource/Bielefeld	"327199"^^<http://www.w3.org/2001/XMLSchema#nonNegativeInteger>	http://www.bielefeld.de/en/index.html
http://dbpedia.org/resource/Bonn	"311287"^^<http://www.w3.org/2001/XMLSchema#nonNegativeInteger>	http://www.bonn.de/
http://dbpedia.org/resource/Mannheim	"311142"^^<http://www.w3.org/2001/XMLSchema#nonNegativeInteger>	http://www.mannheim.de/
http://dbpedia.org/resource/Münster	"300000"^^<http://www.w3.org/2001/XMLSchema#nonNegativeInteger>	http://www.muenster.de/en/
http://dbpedia.org/resource/Karlsruhe	"288917"^^<http://www.w3.org/2001/XMLSchema#nonNegativeInteger>	http://www.karlsruhe.de
http://dbpedia.org/resource/Augsburg	"286374"^^<http://www.w3.org/2001/XMLSchema#nonNegativeInteger>	http://www2.augsburg.de/index.php?id=12307

SPARQL на примерах

```
SELECT * WHERE {
      ?city rdf:type yago:City108524735 ;
            dbo:country dbr:Germany ;
            dbo:populationTotal ?population .
      OPTIONAL { ?city foaf:homepage ?homepage }
}
ORDER BY DESC(?population)
LIMIT 100
OFFSET 200
```

city	population	homepage
http://dbpedia.org/resource/Rheinbach	"26710"^^<http://www.w3.org/2001/XMLSchema#nonNegativeInteger>	http://www.rheinbach.de/
http://dbpedia.org/resource/Limbach-Oberfrohna	"26597"^^<http://www.w3.org/2001/XMLSchema#nonNegativeInteger>	http://www.limbach-oberfrohna.de/
http://dbpedia.org/resource/Delitzsch	"26344"^^<http://www.w3.org/2001/XMLSchema#nonNegativeInteger>	http://www.delitzsch.de/
http://dbpedia.org/resource/Nordenham	"26325"^^<http://www.w3.org/2001/XMLSchema#nonNegativeInteger>	http://www.nordenham.de/
http://dbpedia.org/resource/Strausberg	"26229"^^<http://www.w3.org/2001/XMLSchema#nonNegativeInteger>	http://www.stadt-strausberg.de/
http://dbpedia.org/resource/Petershagen	"26171"^^<http://www.w3.org/2001/XMLSchema#nonNegativeInteger>	http://www.petershagen.de/
http://dbpedia.org/resource/Garmisch-Partenkirchen	"26112"^^<http://www.w3.org/2001/XMLSchema#nonNegativeInteger>	http://www.garmisch-partenkirchen.de/
http://dbpedia.org/resource/Glauchau	"25760"^^<http://www.w3.org/2001/XMLSchema#nonNegativeInteger>	http://www.glauchau.de/
http://dbpedia.org/resource/Zirndorf	"25509"^^<http://www.w3.org/2001/XMLSchema#nonNegativeInteger>	http://www.zirndorf.de/
http://dbpedia.org/resource/Helmstedt	"25389"^^<http://www.w3.org/2001/XMLSchema#nonNegativeInteger>	http://www.stadt-helmstedt.de
http://dbpedia.org/resource/Bad_Honnef	"25326"^^<http://www.w3.org/2001/XMLSchema#nonNegativeInteger>	http://www.bad-honnef.de/
http://dbpedia.org/resource/Norden,_Lower_Saxony	"25097"^^<http://www.w3.org/2001/XMLSchema#nonNegativeInteger>	http://www.norden.de/
http://dbpedia.org/resource/Varel	"25084"^^<http://www.w3.org/2001/XMLSchema#nonNegativeInteger>	http://www.varel.de/
http://dbpedia.org/resource/Bedburg	"24926"^^<http://www.w3.org/2001/XMLSchema#nonNegativeInteger>	http://www.bedburg.de/
http://dbpedia.org/resource/Lindau	"24537"^^<http://www.w3.org/2001/XMLSchema#nonNegativeInteger>	http://www.lindau.de/
http://dbpedia.org/resource/Korbach	"24537"^^<http://www.w3.org/2001/XMLSchema#nonNegativeInteger>	http://www.korbach.de
http://dbpedia.org/resource/Walsrode	"24433"^^<http://www.w3.org/2001/XMLSchema#nonNegativeInteger>	http://www.walsrode.de/
http://dbpedia.org/resource/Rottweil	"24378"^^<http://www.w3.org/2001/XMLSchema#nonNegativeInteger>	http://www.rottweil.de/
http://dbpedia.org/resource/Eisleben	"24284"^^<http://www.w3.org/2001/XMLSchema#nonNegativeInteger>	http://www.eisleben.eu
http://dbpedia.org/resource/Ingelheim_am_Rhein	"24283"^^<http://www.w3.org/2001/XMLSchema#nonNegativeInteger>	http://www.ingelheim.de/
http://dbpedia.org/resource/Döbeln	"24034"^^<http://www.w3.org/2001/XMLSchema#nonNegativeInteger>	http://www.doebeln.de/
http://dbpedia.org/resource/Markkleeberg	"24030"^^<http://www.w3.org/2001/XMLSchema#nonNegativeInteger>	http://www.markkleeberg.de/
http://dbpedia.org/resource/Freudenstadt	"23942"^^<http://www.w3.org/2001/XMLSchema#nonNegativeInteger>	http://www.freudenstadt.de/
http://dbpedia.org/resource/Warburg	"23688"^^<http://www.w3.org/2001/XMLSchema#nonNegativeInteger>	http://www.warburg.de
http://dbpedia.org/resource/Hann._Münden	"23668"^^<http://www.w3.org/2001/XMLSchema#nonNegativeInteger>	http://www.hann.muenden.de/
http://dbpedia.org/resource/Holzminden	"23502"^^<http://www.w3.org/2001/XMLSchema#nonNegativeInteger>	http://www.holzminden.de/

SPARQL на примерах

```
SELECT * WHERE {
      ?city rdf:type yago:City108524735 ;
            dbo:country dbr:Germany ;
            dbo:populationTotal ?population .
      OPTIONAL { ?city foaf:homepage ?homepage }
      FILTER (?population > 500000)
}
ORDER BY DESC(?population)
```

city	population	homepage
http://dbpedia.org/resource/Berlin	"3610156"^^<http://www.w3.org/2001/XMLSchema#nonNegativeInteger>	http://www.berlin.de/en/
http://dbpedia.org/resource/Hamburg	"1774242"^^<http://www.w3.org/2001/XMLSchema#nonNegativeInteger>	http://www.hamburg.de/
http://dbpedia.org/resource/Munich	"1520408"^^<http://www.w3.org/2001/XMLSchema#nonNegativeInteger>	http://www.muenchen.de/
http://dbpedia.org/resource/Cologne	"1057327"^^<http://www.w3.org/2001/XMLSchema#nonNegativeInteger>	http://www.stadt-koeln.de
http://dbpedia.org/resource/Frankfurt	"731095"^^<http://www.w3.org/2001/XMLSchema#nonNegativeInteger>	http://www.frankfurt.de/
http://dbpedia.org/resource/Bochum	"670320"^^<http://www.w3.org/2001/XMLSchema#nonNegativeInteger>	http://www.bochum.de/
http://dbpedia.org/resource/Bremen_(state)	"661000"^^<http://www.w3.org/2001/XMLSchema#nonNegativeInteger>	http://www.bremen.de/
http://dbpedia.org/resource/Düsseldorf	"653678"^^<http://www.w3.org/2001/XMLSchema#nonNegativeInteger>	http://www.duesseldorf.de/
http://dbpedia.org/resource/Stuttgart	"623738"^^<http://www.w3.org/2001/XMLSchema#nonNegativeInteger>	http://www.stuttgart.de/
http://dbpedia.org/resource/Essen	"589075"^^<http://www.w3.org/2001/XMLSchema#nonNegativeInteger>	http://www.essen.de/
http://dbpedia.org/resource/Dortmund	"575944"^^<http://www.w3.org/2001/XMLSchema#nonNegativeInteger>	http://www.dortmund.de/en/
http://dbpedia.org/resource/Dortmund	"575944"^^<http://www.w3.org/2001/XMLSchema#nonNegativeInteger>	http://www.dortmund.de/
http://dbpedia.org/resource/Bremen	"548475"^^<http://www.w3.org/2001/XMLSchema#nonNegativeInteger>	http://www.bremen.de/
http://dbpedia.org/resource/Dresden	"536107"^^<http://www.w3.org/2001/XMLSchema#nonNegativeInteger>	http://www.dresden.de/
http://dbpedia.org/resource/Hanover	"518386"^^<http://www.w3.org/2001/XMLSchema#nonNegativeInteger>	http://www.hannover.de/

SPARQL на примерах

```
SELECT * WHERE {
    ?city rdf:type yago:City108524735 ;
        dbo:country dbr:Germany ;
        dbo:populationTotal ?population ;
        rdfs:label ?label .
    OPTIONAL { ?city foaf:homepage ?homepage }
    FILTER (?population > 500000)
}
ORDER BY DESC(?population)
```

city	population	label	homepage
http://dbpedia.org/resource/Berlin	"3610156"^^<http://www.w3.org/2001/XMLSchema#nonNegativeInteger>	"Berlin"@en	http://www.berlin.de/en/
http://dbpedia.org/resource/Berlin	"3610156"^^<http://www.w3.org/2001/XMLSchema#nonNegativeInteger>	"برلين"@ar	http://www.berlin.de/en/
http://dbpedia.org/resource/Berlin	"3610156"^^<http://www.w3.org/2001/XMLSchema#nonNegativeInteger>	"Berlin"@de	http://www.berlin.de/en/
http://dbpedia.org/resource/Berlin	"3610156"^^<http://www.w3.org/2001/XMLSchema#nonNegativeInteger>	"Berlín"@es	http://www.berlin.de/en/
http://dbpedia.org/resource/Berlin	"3610156"^^<http://www.w3.org/2001/XMLSchema#nonNegativeInteger>	"Berlin"@fr	http://www.berlin.de/en/
http://dbpedia.org/resource/Berlin	"3610156"^^<http://www.w3.org/2001/XMLSchema#nonNegativeInteger>	"Berlino"@it	http://www.berlin.de/en/
http://dbpedia.org/resource/Berlin	"3610156"^^<http://www.w3.org/2001/XMLSchema#nonNegativeInteger>	"Berlijn"@nl	http://www.berlin.de/en/
http://dbpedia.org/resource/Berlin	"3610156"^^<http://www.w3.org/2001/XMLSchema#nonNegativeInteger>	"ベルリン"@ja	http://www.berlin.de/en/
http://dbpedia.org/resource/Berlin	"3610156"^^<http://www.w3.org/2001/XMLSchema#nonNegativeInteger>	"Berlin"@pl	http://www.berlin.de/en/
http://dbpedia.org/resource/Berlin	"3610156"^^<http://www.w3.org/2001/XMLSchema#nonNegativeInteger>	"Berlim"@pt	http://www.berlin.de/en/
http://dbpedia.org/resource/Berlin	"3610156"^^<http://www.w3.org/2001/XMLSchema#nonNegativeInteger>	"Берлин"@ru	http://www.berlin.de/en/
http://dbpedia.org/resource/Berlin	"3610156"^^<http://www.w3.org/2001/XMLSchema#nonNegativeInteger>	"柏林"@zh	http://www.berlin.de/en/
http://dbpedia.org/resource/Hamburg	"1774242"^^<http://www.w3.org/2001/XMLSchema#nonNegativeInteger>	"Hamburg"@en	http://www.hamburg.de/
http://dbpedia.org/resource/Hamburg	"1774242"^^<http://www.w3.org/2001/XMLSchema#nonNegativeInteger>	"هامبورغ"@ar	http://www.hamburg.de/
http://dbpedia.org/resource/Hamburg	"1774242"^^<http://www.w3.org/2001/XMLSchema#nonNegativeInteger>	"Hamburg"@de	http://www.hamburg.de/
http://dbpedia.org/resource/Hamburg	"1774242"^^<http://www.w3.org/2001/XMLSchema#nonNegativeInteger>	"Hamburgo"@es	http://www.hamburg.de/
http://dbpedia.org/resource/Hamburg	"1774242"^^<http://www.w3.org/2001/XMLSchema#nonNegativeInteger>	"Hambourg"@fr	http://www.hamburg.de/
http://dbpedia.org/resource/Hamburg	"1774242"^^<http://www.w3.org/2001/XMLSchema#nonNegativeInteger>	"Amburgo"@it	http://www.hamburg.de/
http://dbpedia.org/resource/Hamburg	"1774242"^^<http://www.w3.org/2001/XMLSchema#nonNegativeInteger>	"ハンブルク"@ja	http://www.hamburg.de/
http://dbpedia.org/resource/Hamburg	"1774242"^^<http://www.w3.org/2001/XMLSchema#nonNegativeInteger>	"Hamburg"@nl	http://www.hamburg.de/
http://dbpedia.org/resource/Hamburg	"1774242"^^<http://www.w3.org/2001/XMLSchema#nonNegativeInteger>	"Hamburg"@pl	http://www.hamburg.de/
http://dbpedia.org/resource/Hamburg	"1774242"^^<http://www.w3.org/2001/XMLSchema#nonNegativeInteger>	"Hamburgo"@pt	http://www.hamburg.de/
http://dbpedia.org/resource/Hamburg	"1774242"^^<http://www.w3.org/2001/XMLSchema#nonNegativeInteger>	"Гамбург"@ru	http://www.hamburg.de/
http://dbpedia.org/resource/Hamburg	"1774242"^^<http://www.w3.org/2001/XMLSchema#nonNegativeInteger>	"汉堡"@zh	http://www.hamburg.de/
http://dbpedia.org/resource/Munich	"1520408"^^<http://www.w3.org/2001/XMLSchema#nonNegativeInteger>	"Munich"@en	http://www.muenchen.de/
http://dbpedia.org/resource/Munich	"1520408"^^<http://www.w3.org/2001/XMLSchema#nonNegativeInteger>	"ميونخ"@ar	http://www.muenchen.de/
http://dbpedia.org/resource/Munich	"1520408"^^<http://www.w3.org/2001/XMLSchema#nonNegativeInteger>	"München"@de	http://www.muenchen.de/
http://dbpedia.org/resource/Munich	"1520408"^^<http://www.w3.org/2001/XMLSchema#nonNegativeInteger>	"Múnich"@es	http://www.muenchen.de/
http://dbpedia.org/resource/Munich	"1520408"^^<http://www.w3.org/2001/XMLSchema#nonNegativeInteger>	"Munich"@fr	http://www.muenchen.de/
http://dbpedia.org/resource/Munich	"1520408"^^<http://www.w3.org/2001/XMLSchema#nonNegativeInteger>	"Monaco di Baviera"@it	http://www.muenchen.de/

SPARQL на примерах

```
SELECT * WHERE {
?city rdf:type yago:City108524735 ;
        dbo:country dbr:Germany ;
        dbo:populationTotal ?population ;
    rdfs:label ?label .
    OPTIONAL { ?city foaf:homepage ?homepage }
    FILTER (?population > 500000
        && langmatches(lang(?label), "RU"))
}
ORDER BY DESC(?population)
```

city	population	label	homepage
http://dbpedia.org/resource/Berlin	"3610156"^^<http://www.w3.org/2001/XMLSchema#nonNegativeInteger>	"Берлин"@ru	http://www.berlin.de/en/
http://dbpedia.org/resource/Hamburg	"1774242"^^<http://www.w3.org/2001/XMLSchema#nonNegativeInteger>	"Гамбург"@ru	http://www.hamburg.de/
http://dbpedia.org/resource/Munich	"1520408"^^<http://www.w3.org/2001/XMLSchema#nonNegativeInteger>	"Мюнхен"@ru	http://www.muenchen.de/
http://dbpedia.org/resource/Cologne	"1057327"^^<http://www.w3.org/2001/XMLSchema#nonNegativeInteger>	"Кёльн"@ru	http://www.stadt-koeln.de
http://dbpedia.org/resource/Frankfurt	"731095"^^<http://www.w3.org/2001/XMLSchema#nonNegativeInteger>	"Франкфурт-на-Майне"@ru	http://www.frankfurt.de/
http://dbpedia.org/resource/Bochum	"670320"^^<http://www.w3.org/2001/XMLSchema#nonNegativeInteger>	"Бохум"@ru	http://www.bochum.de/
http://dbpedia.org/resource/Bremen_(state)	"661000"^^<http://www.w3.org/2001/XMLSchema#nonNegativeInteger>	"Бремен (земля)"@ru	http://www.bremen.de/
http://dbpedia.org/resource/Düsseldorf	"653678"^^<http://www.w3.org/2001/XMLSchema#nonNegativeInteger>	"Дюссельдорф"@ru	http://www.duesseldorf.de/
http://dbpedia.org/resource/Stuttgart	"623738"^^<http://www.w3.org/2001/XMLSchema#nonNegativeInteger>	"Штутгарт"@ru	http://www.stuttgart.de/
http://dbpedia.org/resource/Essen	"589075"^^<http://www.w3.org/2001/XMLSchema#nonNegativeInteger>	"Эссен"@ru	http://www.essen.de/
http://dbpedia.org/resource/Dortmund	"575944"^^<http://www.w3.org/2001/XMLSchema#nonNegativeInteger>	"Дортмунд"@ru	http://www.dortmund.de/en/
http://dbpedia.org/resource/Dortmund	"575944"^^<http://www.w3.org/2001/XMLSchema#nonNegativeInteger>	"Дортмунд"@ru	http://www.dortmund.de/
http://dbpedia.org/resource/Bremen	"548475"^^<http://www.w3.org/2001/XMLSchema#nonNegativeInteger>	"Бремен"@ru	http://www.bremen.de/
http://dbpedia.org/resource/Dresden	"536107"^^<http://www.w3.org/2001/XMLSchema#nonNegativeInteger>	"Дрезден"@ru	http://www.dresden.de/
http://dbpedia.org/resource/Hanover	"518386"^^<http://www.w3.org/2001/XMLSchema#nonNegativeInteger>	"Ганновер"@ru	http://www.hannover.de/

SPARQL на примерах

```
SELECT * WHERE {
      ?city rdf:type yago:City108524735 ;
            dbo:country dbr:England ;
            dbo:populationTotal ?population .
}
ORDER BY DESC(?population)
LIMIT 10
```

city	population
http://dbpedia.org/resource/Milton_Keynes	"229941"^^<http://www.w3.org/2001/XMLSchema#nonNegativeInteger>
http://dbpedia.org/resource/Middlesbrough	"174700"^^<http://www.w3.org/2001/XMLSchema#nonNegativeInteger>
http://dbpedia.org/resource/Huddersfield	"162949"^^<http://www.w3.org/2001/XMLSchema#nonNegativeInteger>
http://dbpedia.org/resource/Guildford	"137200"^^<http://www.w3.org/2001/XMLSchema#nonNegativeInteger>
http://dbpedia.org/resource/Maidstone	"113137"^^<http://www.w3.org/2001/XMLSchema#nonNegativeInteger>
http://dbpedia.org/resource/Basildon	"107123"^^<http://www.w3.org/2001/XMLSchema#nonNegativeInteger>
http://dbpedia.org/resource/Sutton_Coldfield	"95107"^^<http://www.w3.org/2001/XMLSchema#nonNegativeInteger>
http://dbpedia.org/resource/Stockton-on-Tees	"83490"^^<http://www.w3.org/2001/XMLSchema#nonNegativeInteger>
http://dbpedia.org/resource/Wakefield	"76886"^^<http://www.w3.org/2001/XMLSchema#nonNegativeInteger>
http://dbpedia.org/resource/South_Shields	"75337"^^<http://www.w3.org/2001/XMLSchema#nonNegativeInteger>

SPARQL на примерах

```
SELECT * WHERE {
        {
                ?city rdf:type yago:City108524735 ;
                        dbo:country dbr:England ;
                        dbo:populationTotal ?population .
        } UNION {
                ?city rdf:type yago:City108524735 ;
                        dbo:country dbr:United_Kingdom ;
                        dbo:populationTotal ?population .
        }
}
ORDER BY DESC(?population)
LIMIT 10
```

city	population
http://dbpedia.org/resource/London	"8673713"^^<http://www.w3.org/2001/XMLSchema#nonNegativeInteger>
http://dbpedia.org/resource/Glasgow	"2178000"^^<http://www.w3.org/2001/XMLSchema#nonNegativeInteger>
http://dbpedia.org/resource/Birmingham	"1101360"^^<http://www.w3.org/2001/XMLSchema#nonNegativeInteger>
http://dbpedia.org/resource/Bradford	"528155"^^<http://www.w3.org/2001/XMLSchema#nonNegativeInteger>
http://dbpedia.org/resource/Edinburgh	"464990"^^<http://www.w3.org/2001/XMLSchema#nonNegativeInteger>
http://dbpedia.org/resource/Teesside	"376633"^^<http://www.w3.org/2001/XMLSchema#nonNegativeInteger>
http://dbpedia.org/resource/Cardiff	"346100"^^<http://www.w3.org/2001/XMLSchema#nonNegativeInteger>
http://dbpedia.org/resource/Nottingham	"318900"^^<http://www.w3.org/2001/XMLSchema#nonNegativeInteger>
http://dbpedia.org/resource/City_of_Sunderland	"275300"^^<http://www.w3.org/2001/XMLSchema#nonNegativeInteger>
http://dbpedia.org/resource/Southampton	"253651"^^<http://www.w3.org/2001/XMLSchema#nonNegativeInteger>

SPARQL на примерах

```
SELECT * WHERE {
      ?city rdf:type yago:City108524735 ;
                  dbo:populationTotal ?population .
      { ?city dbo:country dbr:United_Kingdom  }
      UNION
      {?city dbo:country dbr:England}
}
ORDER BY DESC(?population)
LIMIT 10
```

city	population
http://dbpedia.org/resource/London	"8673713"^^<http://www.w3.org/2001/XMLSchema#nonNegativeInteger>
http://dbpedia.org/resource/Glasgow	"2178000"^^<http://www.w3.org/2001/XMLSchema#nonNegativeInteger>
http://dbpedia.org/resource/Birmingham	"1101360"^^<http://www.w3.org/2001/XMLSchema#nonNegativeInteger>
http://dbpedia.org/resource/Bradford	"528155"^^<http://www.w3.org/2001/XMLSchema#nonNegativeInteger>
http://dbpedia.org/resource/Edinburgh	"464990"^^<http://www.w3.org/2001/XMLSchema#nonNegativeInteger>
http://dbpedia.org/resource/Teesside	"376633"^^<http://www.w3.org/2001/XMLSchema#nonNegativeInteger>
http://dbpedia.org/resource/Cardiff	"346100"^^<http://www.w3.org/2001/XMLSchema#nonNegativeInteger>
http://dbpedia.org/resource/Nottingham	"318900"^^<http://www.w3.org/2001/XMLSchema#nonNegativeInteger>
http://dbpedia.org/resource/City_of_Sunderland	"275300"^^<http://www.w3.org/2001/XMLSchema#nonNegativeInteger>
http://dbpedia.org/resource/Southampton	"253651"^^<http://www.w3.org/2001/XMLSchema#nonNegativeInteger>

SPARQL на примерах

```
ASK WHERE {
        <http://dbpedia.org/resource/London> dbo:country dbr:England
}
```

false

SPARQL на примерах

```
ASK WHERE {
        <http://dbpedia.org/resource/London> dbo:country dbr:United_Kingdom ;
                dbo:populationTotal ?population .
        FILTER (?population > 1000000)
}
```

true

SPARQL на примерах

DESCRIBE <http://dbpedia.org/resource/Russia>

This HTML5 document contains 18124 embedded RDF statements represented using HTML+Microdata notation.

The embedded RDF content will be recognized by any processor of HTML5 Microdata.

Prefix	Namespace IRI
n18	http://dbpedia.org/resource/Mikhaylovsky_District,
n82	http://dbpedia.org/resource/Spassky_District,
n877	http://dbpedia.org/resource/2006_St.
dbo	http://dbpedia.org/ontology/
n804	http://dbpedia.org/resource/2012â13_Chelsea_F.C.
n148	http://dbpedia.org/resource/Seymchan_(urban-type_settlement)
n747	http://dbpedia.org/resource/Magomed_Omarov_(boxer)
n880	http://dbpedia.org/resource/Pavlovka_(meteorite)
n486	http://dbpedia.org/resource/Ivan_Kupala_(band)
n360	http://dbpedia.org/resource/Klimovo,
n428	http://dbpedia.org/resource/Petropavlovsky_District,
n785	http://dbpedia.org/resource/Umba_River_(Russia)
n396	http://dbpedia.org/resource/1992â93_Liverpool_F.C.
n35	http://dbpedia.org/resource/Trinity_Lavra_of_St.
n533	http://dbpedia.org/resource/Malaya_Tsilna,
n790	http://dbpedia.org/resource/Sergey_Morozov_(racewalker)
n334	http://dbpedia.org/resource/Petrovsky_District,
n990	http://dbpedia.org/resource/Zherdevka,
n392	http://dbpedia.org/resource/Volokonovka,_Volokonovsky_District,
n52	http://dbpedia.org/resource/Red_Room_(comics)
n76	http://dbpedia.org/resource/Tomtor,_Tattinsky_District,

SPARQL на примерах

```
DESCRIBE ?city WHERE {
      ?city dbo:country dbr:United_Kingdom ;
            dbo:populationTotal ?population .
      FILTER (?population > 1000000)
}
```

The embedded RDF content will be recognized by any processor of HTML5 Microdata.

Prefix	Namespace IRI
n584	http://dbpedia.org/resource/Five_Suns_(album)
n880	http://dbpedia.org/resource/1984â85_Cardiff_City_F.C.
n358	http://dbpedia.org/resource/Wind_of_Change_(album)
n260	http://dbpedia.org/resource/2014â15_Leicester_City_F.C.
n250	http://dbpedia.org/resource/David_Walsh_(Donegal_Gaelic_footballer)
n843	http://dbpedia.org/resource/Rob_Skipper_&
n627	http://dbpedia.org/resource/2016â17_Motherwell_F.C.
n45	http://dbpedia.org/resource/Oliver_Lyttelton,
n525	http://dbpedia.org/resource/1994â95_Fulham_F.C.
n351	http://dbpedia.org/resource/William_Smith_(abolitionist)
n756	http://dbpedia.org/resource/Discovery_World_(TV_channel)
n774	http://dbpedia.org/resource/Hourglass_(Clannad_song)
n497	http://dbpedia.org/resource/Joseph_Ward_(VC)
n187	http://dbpedia.org/resource/International_Federation_of_Shipmasters'
n9	http://dbpedia.org/resource/Beenz.
n957	http://dbpedia.org/resource/Michael_Andrews_(artist)
n561	http://dbpedia.org/resource/Academy_of_Medical_Sciences,

SPARQL на примерах

```
CONSTRUCT {
        ?city rdf:type <http://example.com/City> ;
                <http://example.com/inCountry> ?country ;
                <http://example.com/population> ?population .
} WHERE {
        ?city rdf:type yago:City108524735 ;
                dbo:country dbr:Germany ;
                dbo:country ?country ;
               dbo:populationTotal ?population .
}
```

```
@prefix rdf:    <http://www.w3.org/1999/02/22-rdf-syntax-ns#> .
@prefix ns1:    <http://example.com/> .
<http://dbpedia.org/resource/Freiburg,_Lower_Saxony>    rdf:type        ns1:City .
@prefix dbr:    <http://dbpedia.org/resource/> .
<http://dbpedia.org/resource/Freiburg,_Lower_Saxony>    ns1:inCountry   dbr:Germany .
@prefix xsd:    <http://www.w3.org/2001/XMLSchema#> .
<http://dbpedia.org/resource/Freiburg,_Lower_Saxony>    ns1:population  "1864"^^xsd:nonNegativeInteger .
<http://dbpedia.org/resource/Horka,_Saxony>     rdf:type        ns1:City ;
        ns1:inCountry   dbr:Germany ;
        ns1:population  "1984"^^xsd:nonNegativeInteger .
<http://dbpedia.org/resource/Sosa,_Germany>     rdf:type        ns1:City ;
        ns1:inCountry   dbr:Germany ;
        ns1:population  "2069"^^xsd:nonNegativeInteger .
<http://dbpedia.org/resource/G\u00F6ttingen>    rdf:type        ns1:City ;
        ns1:inCountry   dbr:Germany ;
        ns1:population  "116052"^^xsd:nonNegativeInteger .
<http://dbpedia.org/resource/Uhry_(K\u00F6nigslutter)>  rdf:type        ns1:City ;
        ns1:inCountry   dbr:Germany ;
        ns1:population  "300"^^xsd:nonNegativeInteger .
dbr:Suurhusen   rdf:type        ns1:City ;
        ns1:inCountry   dbr:Germany ;
        ns1:population  "1200"^^xsd:nonNegativeInteger .
dbr:Bramstedt   rdf:type        ns1:City ;
        ns1:inCountry   dbr:Germany ;
        ns1:population  "1892"^^xsd:nonNegativeInteger .
dbr:Freital     rdf:type        ns1:City ;
        ns1:inCountry   dbr:Germany ;
        ns1:population  "39267"^^xsd:nonNegativeInteger .
<http://dbpedia.org/resource/Grevesm\u00FChlen> rdf:type        ns1:City ;
        ns1:inCountry   dbr:Germany ;
        ns1:population  "10945"^^xsd:nonNegativeInteger .
```

Агрегатные функции

- Агрегатные функции были введены в SPARQL 1.1[1]
- Похожи на агрегатные функции из SQL: min, max, avg, sum, count и другие

[1] http://www.w3.org/TR/sparql11-query/

Агрегатные функции

:John :age 32 .
:John :gender :male .
:Tim :age 20.
:Tim :gender :male.
:Jane :gender :female.
:Jane :age 23.

SELECT **avg**(?age) WHERE {?person :age ?age}
--> 25

SELECT ?gender **min**(?age) { ?person :age ?age. ?person :gender ?gender}
GROUP BY ?gender

--> :male 20
--> :female 23

Запросы о структуре датасета

- Список всех классов

```
SELECT DISTINCT ?class WHERE { ?x a ?class }
```

- Список всех предикатов

```
SELECT DISTINCT ?pred WHERE {?x ?pred ?y}
```

- Все классы, к которым относиться ресурс

```
SELECT ?class WHERE { <...> a ?class }
```

Логический вывод и SPARQL

- Логический вывод (Reasoning) не является функцией SPARQL
- Но частично он может быть симулирован с помощью SPARQL

Property Paths[1]

- Специальный синтаксис для работы с предикатами
- Является...
  - “синтаксическим сахаром”
  - и способом навигации по графу

[1] http://www.w3.org/TR/sparql11-query/#propertypaths

Property Paths

- Альтернативные предикаты

```
SELECT * WHERE { :book1 dc:title | rdfs:title ?nameofBook }
```

Property Paths

- Последовательность предикатов

Найти имена всех, кого знает Alice.

```
SELECT ?name WHERE {
    ?x foaf:mbox <mailto:alice@example> .
    ?x foaf:knows/foaf:name ?name .
}
```

Property Paths

- Последовательность предикатов

Найти имена всех людей, кого знают люди, известные Alice.

```
SELECT ?name WHERE {
    ?x foaf:mbox <mailto:alice@example> .
    ?x foaf:knows/foaf:knows/foaf:name ?name .
}
```

Property Paths

- Последовательность предикатов

Найти имена всех людей, кого знают люди, известные Alice. Без использования Property Paths:

```
SELECT ?x ?name
 {
   ?x  foaf:mbox <mailto:alice@example> .
   ?x  foaf:knows ?a1 .
   ?a1 foaf:knows ?a2 .
   ?a2 foaf:name ?name .
 }
```

Property Paths

- Последовательность предикатов

Найти имена всех людей, которые связаны с Alice через одно или несколько “рукопожатий”.

```
SELECT * WHERE {
   ?x foaf:mbox <mailto:alice@example> .
   ?x foaf:knows+/foaf:name ?name .
 }
```

Property Paths

- Пример симуляции reasoning’a

Найти все классы, к которым принадлежит ресурс.

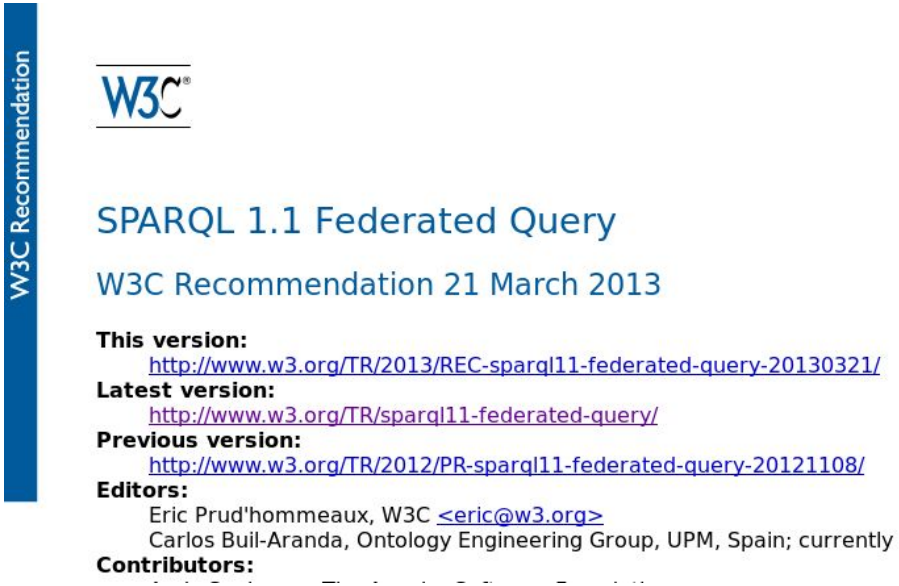
```
SELECT * WHERE {
    :thing rdf:type/rdfs:subClassOf* ?type
}
```

^elt	Инверсивный путь (от объекта к субъекту).
(elt)	Групповой путь, скобки иллюстрируют приоритет.
elt1 / elt2	Путь последовательности elt1, за которым следует elt2
elt1 ^ elt2	Сокращение для elt1 / ^ elt2, то есть elt1, за которым следует инверсия elt2.
elt1 | elt2	Альтернативный путь для elt1 или elt2 (все возможные варианты).
elt*	Путь для нуля или большего числа вхождений elt.
elt+	Путь для одного или большего числа вхождений elt.
elt?	Путь для нуля или одного вхождения elt.
elt{n,m}	Путь от n до m вхождений elt.
elt{n}	Ровно n вхождений elt.
elt{n,}	n и более вхождений elt.
elt{,n}	От 0 до n вхождений elt.

Федеративный SPARQL

Часть стандарта SPARQL 1.1

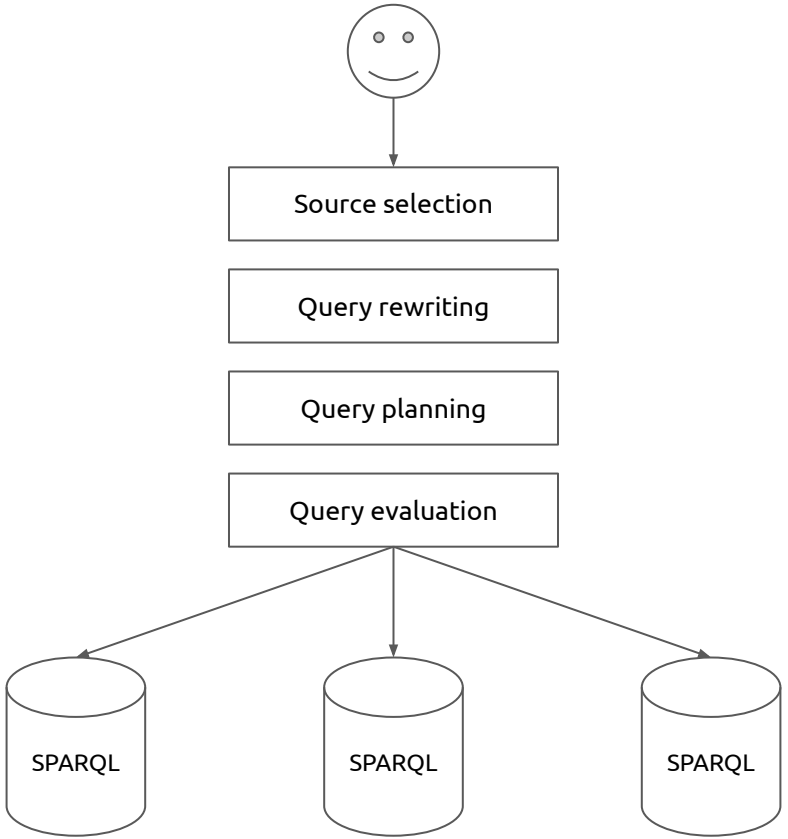
- Поддерживается всеми RDF хранилищами соответствующие стандарту SPARQL 1.1
- Существует специализированное ПО для федеративного SPARQL



https://www.w3.org/TR/sparql11-federated-query

Алгоритм работы

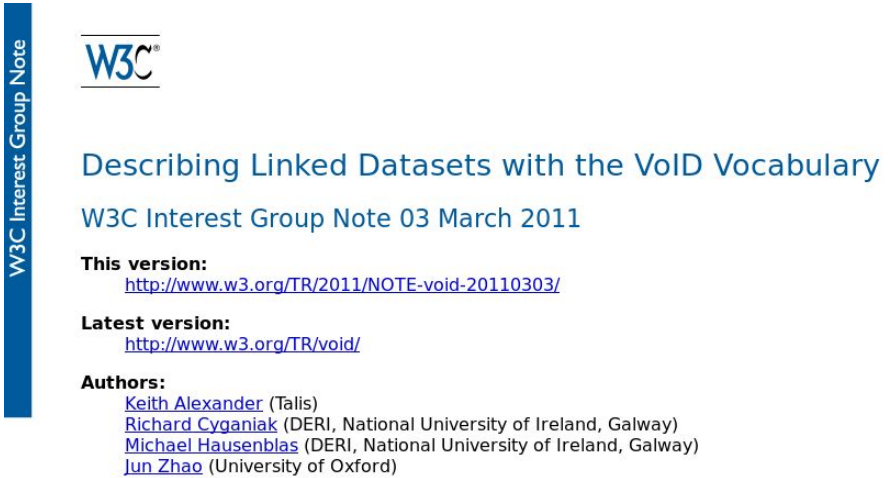
- выбирает куда отправлять запрос,
- переписывает запрос под каждый источник,
- устанавливает порядок выполнения подзапросов,
- объединяет результаты.



Источник: https://hal.archives-ouvertes.fr/tel-01974118/document

Шаг: Source selection

- каталог структурированных описаний
  - Service descriptions,
  - VoID.
- результаты sampling-запросов
  - 14% VoID,
  - 25% Service descriptions,
  - ASK запросы на BGP.



https://www.w3.org/TR/void/

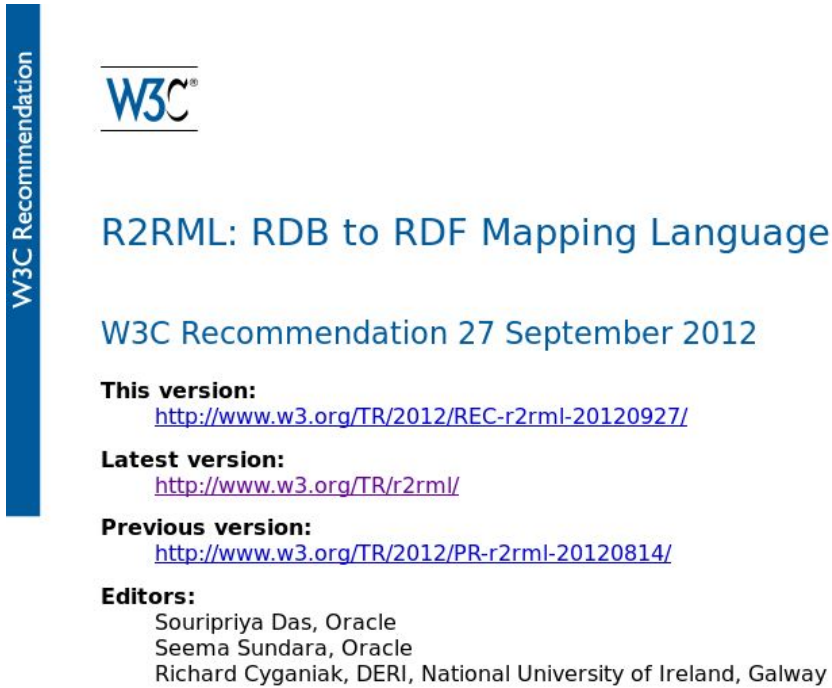
Hack: SQL запрос внутри федеративного SPARQL

```
SELECT ?locationId (?orderDate AS ?date) ?numOrders ?numPeople (?numOrders / ?numPeople AS ?peopleToOrdersRate) {
    SERVICE <http://104.155.121.206:5000/sparql-to-sql> {
        [
            <urn:sparql-to-sql:queryTemplate> """
                SELECT location_id AS locationId, DATE(order_closed) AS orderDate, COUNT(order_number) AS numOrders
                FROM orders
                WHERE order_closed > :start_date AND order_closed < :end_date
                GROUP BY location_id, DATE(order_closed)
            """ ;
            <urn:sparql-to-sql:parameters> [
                <urn:param:start_date> "2019-01-01T00:00:00+03:00"^^xsd:dateTime ;
                <urn:param:end_date> "2019-02-01T23:59:59+03:00"^^xsd:dateTime
            ] ;
            <urn:sparql-to-sql:projection> ?locationId, ?orderDate, ?numOrders
        ]
    }

    hint:Prior hint:runFirst true .

    ?obsUri a qb:Observation ;
        qb:dataSet wvri:peoplecountstats ;
        sdmx-dimension:refTime ?obsDate ;
        pcs:refPoS/schema:identifier ?locationId ;
        pcs:measure-peopleamount ?numPeople .

    FILTER(?obsDate = ?orderDate)
}
```

Расширения RDF и SPARQL

Стандарт RDB to RDF Mapping Language (R2RML)

- декларативный маппинг в формате Turtle
- таблица.столбец -> RDF трипл
- SQL запрос -> RDF трипл



https://www.w3.org/TR/r2rml/

RDF* and SPARQL*-аннотирование утверждений RDF

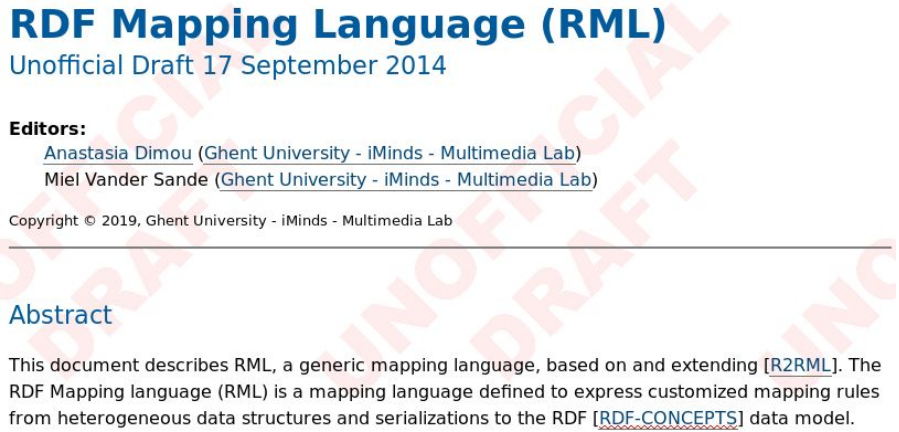
- Альтернатива RDF reification
- Непосредственно содержит триплет как субъект или объект
- https://blog.liu.se/olafhartig/2019/01/10/position-statement-rdf-star-and-sparql-star/
- https://github.com/RDFstar/RDFstarTools

```
<<:bob foaf:age 23>> ex:certainty 0.9 .
```

```
SELECT ?p ?a ?c WHERE {
  <<?p foaf:age ?a>> ex:certainty ?c .
}
```

RDF Mapping Language

- расширение R2RML с поддержкой дополнительных типов источников
- CSV, XML, и т.д.
- Web API, SPARQL



RDF Mapping Language (RML)

Unofficial Draft 17 September 2014

Editors:

Anastasia Dimou (Ghent University - iMinds - Multimedia Lab)

Miel Vander Sande (Ghent University - iMinds - Multimedia Lab)

Copyright © 2019, Ghent University - iMinds - Multimedia Lab

Abstract

This document describes RML, a generic mapping language, based on and extending [R2RML]. The RDF Mapping language (RML) is a mapping language defined to express customized mapping rules from heterogeneous data structures and serializations to the RDF [RDF-CONCEPTS] data model.

https://rml.io/specs/rml/

Стандарт GeoSPARQL

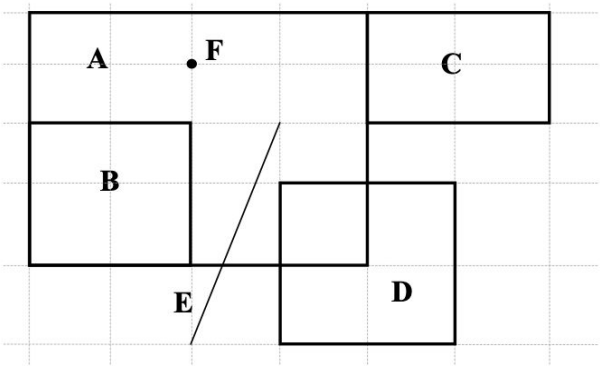
- отдельный стандарт с расширениями для SPARQL,
- форматы для указания координат,
- доп. функции

https://www.ogc.org/standards/geosparql

Стандарт GeoSPARQL - пример

Example 2: *Find all features that are within a transient bounding box geometry, where spatial calculations are based on* `my:hasPointGeometry`.

```
PREFIX my:   <http://example.org/ApplicationSchema#>
PREFIX geo:  <http://www.opengis.net/ont/geosparql#>
PREFIX geof: <http://www.opengis.net/def/function/geosparql/>

SELECT ?f
WHERE { ?f my:hasPointGeometry ?fGeom .
        ?fGeom geo:asWKT ?fWKT .
        FILTER (geof:sfWithin(?fWKT,
                 "<http://www.opengis.net/def/crs/OGC/1.3/CRS84>
                  Polygon ((-83.4 34.0, -83.1 34.0,
                            -83.1 34.2, -83.4 34.2,
                            -83.4 34.0))"^^geo:wktLiteral))
}
```

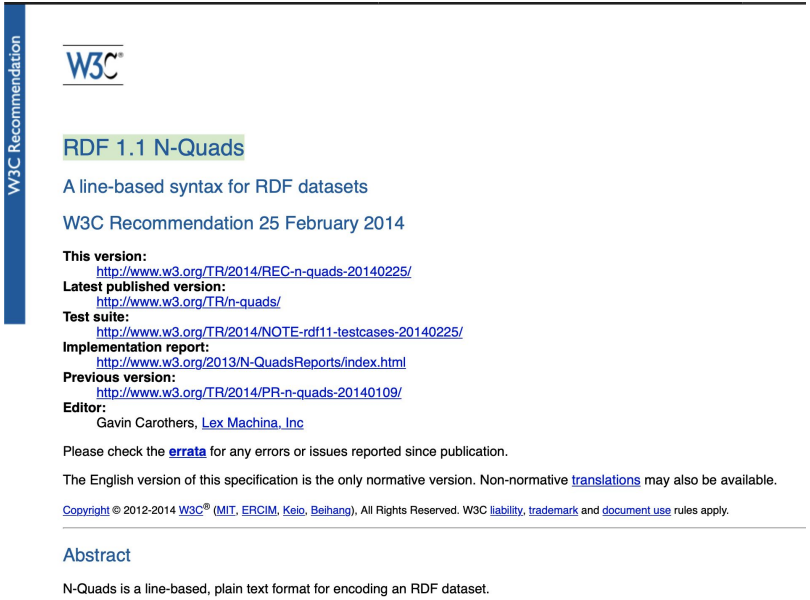


Result:

?f
my:D

Стандарт RDF 1.1 N-Quads

- N-Quads - это последовательности терминов RDF, представляющих субъект, предикат, объект и метку графа RDF Triple,
- Граф в данном случае также является частью датасета.



W3C Recommendation

W3C

RDF 1.1 N-Quads

A line-based syntax for RDF datasets

W3C Recommendation 25 February 2014

This version:
http://www.w3.org/TR/2014/REC-n-quads-20140225/
Latest published version:
http://www.w3.org/TR/n-quads/
Test suite:
http://www.w3.org/TR/2014/NOTE-rdf11-testcases-20140225/
Implementation report:
http://www.w3.org/2013/N-QuadsReports/index.html
Previous version:
http://www.w3.org/TR/2014/PR-n-quads-20140109/
Editor:
Gavin Carothers, Lex Machina, Inc

Please check the **errata** for any errors or issues reported since publication.

The English version of this specification is the only normative version. Non-normative translations may also be available.

Copyright © 2012-2014 W3C® (MIT, ERCIM, Keio, Beihang), All Rights Reserved. W3C liability, trademark and document use rules apply.

Abstract

N-Quads is a line-based, plain text format for encoding an RDF dataset.

EXAMPLE 3

```
_:alice <http://xmlns.com/foaf/0.1/knows> _:bob <http://example.org/graphs/john> .
_:bob <http://xmlns.com/foaf/0.1/knows> _:alice <http://example.org/graphs/james> .
```

https://www.w3.org/TR/n-quads/

Способы валидации RDF графов

Стандарт SHACL

- декларативный язык для описания и валидации RDF данных,
- дополняет RDFS и OWL тем что позволяет описывать классы как сущности,
- демо: https://shacl.org/playground

Shapes Constraint Language (SHACL)

W3C Recommendation 20 July 2017

This version:
https://www.w3.org/TR/2017/REC-shacl-20170720/

Latest published version:
https://www.w3.org/TR/shacl/

Latest editor's draft:
https://w3c.github.io/data-shapes/shacl/

Implementation report:
https://w3c.github.io/data-shapes/data-shapes-test-suite/

Previous version:
https://www.w3.org/TR/2017/PR-shacl-20170608/

Editors:
Holger Knublauch, TopQuadrant, Inc.
Dimitris Kontokostas, University of Leipzig

Repository:
GitHub
Issues

Test Suite:
SHACL Test Suite

https://www.w3.org/TR/shacl/

SHACL: Пример

Примеры и туториал:

- https://www.topquadrant.com/technology/shacl/tutorial-sparql-constraints/
- https://www.topquadrant.com/technology/shacl/tutorial/

Data Graph

```
ex:Alice
   a ex:Person ;
   ex:ssn "987-65-432A" .

ex:Bob
   a ex:Person ;
   ex:ssn "123-45-6789" ;
   ex:ssn "124-35-6789" .

ex:Calvin
   a ex:Person ;
   ex:birthDate "1971-07-07"^^xsd:date ;
   ex:worksFor ex:UntypedCompany .
```

Shapes Graph

```
ex:PersonShape
   a sh:NodeShape ;
   sh:targetClass ex:Person ;    # Applies
   sh:property [                 # _:b1
      sh:path ex:ssn ;           # constrai
      sh:maxCount 1 ;
      sh:datatype xsd:string ;
      sh:pattern "^\\d{3}-\\d{2}-\\d{4}$" ;
   ] ;
   sh:property [                 # _:b2
      sh:path ex:worksFor ;
      sh:class ex:Company ;
      sh:nodeKind sh:IRI ;
   ] ;
   sh:closed true ;
   sh:ignoredProperties ( rdf:type ) .
```

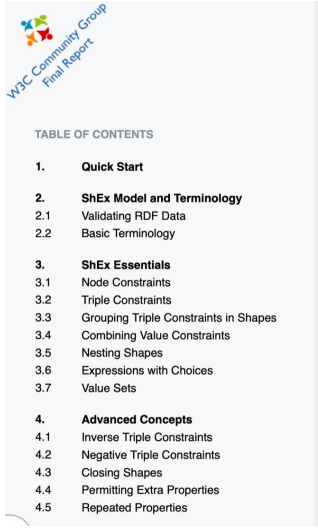
pySHACL: библиотека

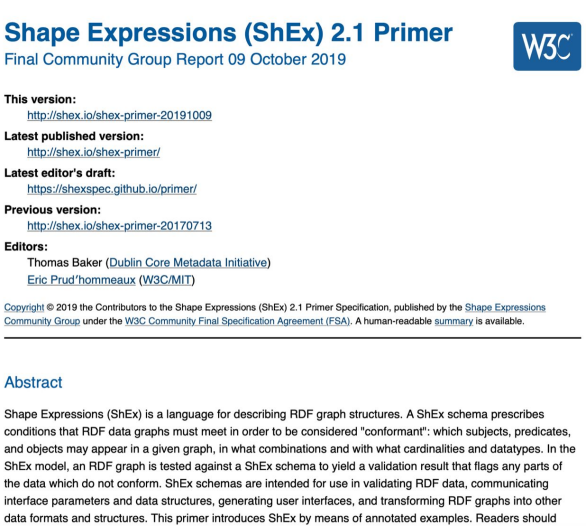
- Документация https://pypi.org/project/pyshacl/0.8.2/
- Репо https://github.com/RDFLib/pySHACL
- Примеры https://derwen.ai/docs/kgl/ex5_0/

```
import pyshacl

results = pyshacl.validate(
    data_graph,
    shacl_graph=shapes_graph,
    data_graph_format="json-ld",
    shacl_graph_format="ttl",
    inference="rdfs",
    debug=True,
    serialize_report_graph="ttl",
    )

conforms, report_graph, report_text = results

print("conforms", conforms)
```

Язык SHEX

- декларативный язык для описания профилей RDF данных
- Демо и туториал: https://rawgit.com/shexSpec/shex.js/wikidata/doc/micro-tutorial.html
- http://rdfshape.herokuapp.com





https://shex.io

Краткий обзор RDF хранилищ

Триплсторы

Список некоторых популярных триплсторов:

- OpenLink Virtuoso, http://virtuoso.openlinksw.com/
- Fuseki, http://jena.apache.org/
- Sesame, http://rdf4j.org/
- Stardog, http://stardog.com/
- Blazegraph, http://blazegraph.com/

RDF хранилища с открытым исходных кодом

- Apache Fuseki
- Eclipse RDF4J
- Blazegraph
  - до пары миллиардов триплов
  - скорость записи падает с кол-вом триплов
- Halyard
  - Apache HBase
- Openlink Virtuoso Open Source
  - бесплатная версия

Коммерческие RDF хранилища

Сравнивать и оценивать любой класс/тип СУБД дело непростое, поэтому это только поверхностный обзор.

Название	LPG	Full-text поиск	Вирт. графы	GeoSPARQL	Reasoning
Ontotext GraphDB	Нет	Да	Нет	Да	Да
Stardog	Нет	Да	Да	Да	Да
Amazon Neptune	Да	Нет	Нет	Нет	Нет
AllegroGraph	Нет	Да	Нет	Нет (?)	Да

В это обзоре не упомянуты: MarkLogic Database, AnzoGraph DB.

Практическое задание 1

- Выполнить поиск для РФ и США в сервисах dbpedia и wikidata следующей информации:
  - Население страны;
  - Количество городов/районов/областей;
  - Общая площадь;

Практическое задание 2

- Зарегистрироваться в сервисе https://ontodia.org
- Продолжение задания после 3_2

Практическое задание 3_1

- Построить онтологическую модель любой предметной области, состоящей из не менее 8 классов, 6 отношений (Object Properties) и 10 атрибутов (Data Properties). Обязательно использовать одно или несколько свойств связей (инверсия, транзитивность и т.п.)
- Наполнить онтологическую модель экземплярами и определить связи
- Выполнить Ризонинг для построения новых связей на основе свойств связей.

Практическое задание 3_2

- Загрузить построенную онтологическую модель, наполненную экземплярами и связями в сервис ontodia;
- Проверить корректность модели в визуальном редакторе;
- Загрузить построенную модель в сервис VOWL.

Практическое задание 4_1

- Необходимо построить запросы и проверить их работу. (http://dbpedia.org/sparql). Для РФ и США:
  - Население(dbo:populationTotal);
  - Количество городов/районов/областей (dbo:PopulatedPlace);
  - Общая площадь (dbo:areaTotal)
  - Список городов с населением свыше 1 млн (опционально)

Практическое задание 4_2

- Необходимо построить запросы и проверить их работу. (https://query.wikidata.org). Сначала приводится пример, а затем абсолютно аналогичное задание.
  - Список всех нефтяных месторождений
    - Список моделей бренда BMW
  - Изображения всех котов в wikidata
    - Изображения всех собак в wikidata

Список всех нефтяных месторождений

```
SELECT ?item ?itemLabel

WHERE

{

  ?item wdt:P31 wd:Q211748.

  SERVICE wikibase:label { bd:serviceParam wikibase:language
"[AUTO_LANGUAGE],en". }

}

ORDER BY ?itemLabel
```

Список моделей под брендом BMW

Структуру запроса можно взять из предыдущего слайда. Принципиальные отличия:

wdt:P1716 - это свойство brand.

wd:Q796364 - BMW

Изображения всех котов

```
#defaultView:ImageGrid

SELECT ?item ?itemLabel ?pic

WHERE

{

?item wdt:P31 wd:Q146 .

?item wdt:P18 ?pic

SERVICE wikibase:label { bd:serviceParam wikibase:language
"[AUTO_LANGUAGE],en" }

}
```

Изображения всех собак

Структуру запроса можно взять из предыдущего слайда.
Принципиальное отличие:

Собака в wikidata имеет URI **Q144**